



Certification professionnelle

Titre professionnel: **Concepteur développeur d'applications**

RNCP: **37873**

Dossier Projet

Projet : M&C Billetterie

Rédacteur : Coulet Corentin

Date : 26/11/2025

SOMMAIRE

1. LISTE DES COMPÉTENCES DU RÉFÉRENTIEL COUVERTES PAR LE PROJET

1.1. Développer une application sécurisée

- 1.1.1. Installer et configurer son environnement de travail en fonction du projet
- 1.1.2. Développer des interfaces utilisateur
- 1.1.3. Développer des composants métier
- 1.1.4. Contribuer à la gestion d'un projet informatique

1.2. Concevoir et développer une application sécurisée organisée en couches

- 1.2.1. Analyser les besoins et maquetter une application
- 1.2.2. Définir l'architecture logicielle d'une application
- 1.2.3. Concevoir et mettre en place une base de données relationnelle
- 1.2.4. Développer des composants d'accès aux données SQL et NoSQL

1.3. Préparer le déploiement d'une application sécurisée

- 1.3.1. Préparer et exécuter les plans de tests d'une application
- 1.3.2. Préparer et documenter le déploiement d'une application
- 1.3.3. Contribuer à la mise en production dans une démarche DevOps

2. CAHIER DES CHARGES

- 2.1. Description de l'existant
- 2.2. Reprise de l'existant
- 2.3. Principes de référencement
- 2.4. Exigences de performances et de volumétrie
- 2.5. Multilinguisme & adaptations pour un public spécifique

- 2.6. Description graphique et ergonomique
 - 2.6.1. Composants de la charte graphique
 - 2.6.2. Design, Responsive design et autres exigences liées au design
- 2.7. Besoins fonctionnels « métier »
 - 2.7.1. Utilisateurs du projet
 - 2.7.2. Informations relatives aux contenus
 - 2.7.3. Inventaire des besoins fonctionnels
- 2.8. Budget

3. PRÉSENTATION DE L'ENTREPRISE ET DU SERVICE

- 3.1. Présentation de l'entreprise et du service
- 3.2. Objectifs du projet
- 3.3. Cible adressée par le projet
- 3.4. Processus utilisateur impacté

4. GESTION DE PROJET

- 4.1. Intervenants sur le projet
- 4.2. Méthodologie
- 4.3. Outils, planning et suivi
- 4.4. Objectifs de qualité

5. SPÉCIFICATIONS FONCTIONNELLES

- 5.1. Contraintes du projet et livrables attendus
 - 5.1.1. Criticité de l'application
 - 5.1.2. Applications connexes
 - 5.1.3. Services tiers
 - 5.1.4. Livrables attendus

- 5.2. Architecture logicielle du projet
- 5.3. Maquettes et enchaînement des maquettes
 - 5.3.1. Cartographie
 - 5.3.2. Maquettes
- 5.4. Diagramme du comportement des fonctionnalités de type cas d'utilisations
 - 5.4.1. MCD (MERISE)
 - 5.4.2. MLD (MERISE)
 - 5.4.3. MPD (MERISE)
 - 5.4.4. Diagramme de classes (UML)
- 5.5. Script de création et/ou de modification de la base de données
 - 5.5.1. Script de création
 - 5.5.2. Argumentation
 - 5.5.3. Script de modification
 - 5.5.4. Argumentation
- 5.6. Diagramme du comportement des fonctionnalités de type cas d'utilisations
 - 5.6.1. Diagramme de cas d'utilisation global (UML)
- 5.7. Fonctionnalités détaillées les plus significatives
 - 5.7.1. Fonctionnalité 1
 - 5.7.2. Fonctionnalité 2
 - 5.7.3. Fonctionnalité 3
 - 5.7.4. Fonctionnalité 4
 - 5.7.5. Fonctionnalité 5

6. SPÉCIFICATIONS TECHNIQUES

- 6.1. Référencement

- 6.2. Environnement technique
- 6.3. Navigation et accessibilité
- 6.4. Services tiers
- 6.5. Sécurité

7. RÉALISATIONS

- 7.1. Exemple 1 : xxx (Composants Métier)
 - 7.1.1. Affichage
 - 7.1.2. Extrait(s) de code
 - 7.1.3. Argumentation
- 7.2. Exemple 2 : xxx (Composants Métier)
 - 7.2.1. Affichage
 - 7.2.2. Extrait(s) de code
 - 7.2.3. Argumentation
- 7.3. Exemple 1 : xxx (Composants d'accès aux données)
 - 7.3.1. Affichage
 - 7.3.2. Extrait(s) de code
 - 7.3.3. Argumentation
- 7.4. Exemple 2 : xxx (Composants d'accès aux données)
 - 7.4.1. Affichage
 - 7.4.2. Extrait(s) de code
 - 7.4.3. Argumentation
- 7.5. Exemple 1 : xxx (Autres)
 - 7.5.1. Affichage
 - 7.5.2. Extrait(s) de code

- 7.5.3. Argumentation

8. ÉLÉMENTS DE SÉCURITÉ DE L'APPLICATION

- 8.1. Protection des mots de passe
- 8.2. Authentification sécurisée par JWT
- 8.3. Protection contre les injections et XSS
- 8.4. Gestion des accès CORS
- 8.5. Conformité RGPD
- 8.6. Mesures complémentaires

9. PLAN DE TESTS

- 9.1. Stratégie de tests
- 9.2. Organisation et outillage
- 9.3. Types de tests effectués
- 9.4. Maintenance et évolutions

10. JEU D'ESSAI DE LA FONCTIONNALITÉ LA PLUS REPRÉSENTATIVE

- 10.1. Fonctionnalité testée
- 10.2. Description des scénarios
- 10.3. Résultats des tests
- 10.4. Conclusion

11. VEILLE SUR LES VULNÉRABILITÉS DE SÉCURITÉ

1. Liste des compétences du référentiel couvertes par le projet

Le projet **M&C Society (M&C Billetterie)** mobilise l'ensemble des compétences du titre professionnel **Concepteur Développeur d'Applications (RNCP 37873)**.

Il met en œuvre les trois grands blocs du référentiel : le développement d'une application sécurisée, la conception multicouche, et la préparation du déploiement dans une démarche DevOps.

1.1. Développer une application sécurisée

1.1.1. Installer et configurer son environnement de travail en fonction du projet

L'environnement de développement a été mis en place avec une approche orientée productivité et sécurité :

- **Système d'exploitation** : Windows 11 + WSL2 Ubuntu pour la compatibilité Linux.
- **IDE** : Visual Studio Code, avec extensions ESLint, Prettier, Prisma et Docker pour assurer une cohérence du code et un formatage automatique.
- **Langages et frameworks** : TypeScript, Next.js 15, Prisma ORM, PostgreSQL, Docker.
- **Gestion des dépendances** : npm avec scripts automatisés pour les tests et le déploiement.
- **Contrôle de version** : Git + GitHub, avec gestion par branches (main, develop, feature/*).
- **Outils de gestion de projet** : Trello pour le suivi agile et GitHub Issues pour la traçabilité technique.

La configuration des variables d'environnement a été isolée dans un fichier .env non

versionné, afin de protéger les secrets (clés JWT, credentials Stripe, accès PostgreSQL).

The screenshot displays the Docker Desktop application window. The top navigation bar includes 'CONTAINERS', 'Preview README.md', 'Tests', 'Sécurité', and 'Démarrage Rapide'. The main area is divided into several sections:

- Containers:** A table listing running containers with columns for Name, Image, Container ID, Port(s), and CPU. The containers listed are: 'billetterie-complete' (image: billetterie-complete, port: 3001:3001), 'billetterie-app-dev' (image: billetterie-app-dev, port: 8081:8080), 'adminer' (image: adminer, port: 8084:8081), 'redis-commander' (image: redis-commander, port: 5433:5432), 'postgres-dev' (image: postgres-dev, port: 1025:1025), 'mailhog' (image: mailhog, port: 8025:8025), and 'redis-dev' (image: redis-dev, port: 6379:6379).
- System Resources:** A section showing 'Container CPU usage' and 'Container memory usage' with a 'Show charts' button.
- Images:** A section showing 'Container CPU usage' and 'Container memory usage' with a 'Show charts' button.
- Extensions:** A section showing 'Container CPU usage' and 'Container memory usage' with a 'Show charts' button.
- Terminal:** A terminal window at the bottom showing commands and output for installing and running the application.

Name	Image	Container ID	Port(s)	CPU
billetterie-complete	billetterie-complete	613244ac9c8	3001:3001	...
billetterie-app-dev	billetterie-app-dev	6eb7c54b2b69	8081:8080	...
adminer	adminer	93001ec545a5	8084:8081	...
redis-commander	redis-commander	1551a23e7610	5433:5432	...
postgres-dev	postgres-dev	538e91e9447	1025:1025	...
mailhog	mailhog	dc6acdfcf65	8025:8025	...
redis-dev	redis-dev	dc6acdfcf65	6379:6379	...

1.1.2. Développer des interfaces utilisateur

Le **front-end** a été développé avec **Next.js 15 (App Router)** en TypeScript, pour combiner performance, modularité et sécurité.

L'interface utilisateur a été pensée pour offrir une expérience fluide, moderne et accessible :

- **Design System** : basé sur **Shadcn UI v4** et **Tailwind CSS**, garantissant cohérence visuelle et accessibilité native (composants Radix).
- **Charte graphique** : respect du thème bleu, noir et blanc, assurant une identité visuelle forte.
- **Accessibilité** : navigation clavier, textes alternatifs pour les images, contrastes conformes WCAG 2.1 AA.
- **Responsive Design** : approche mobile-first avec adaptation sur tablette et desktop via CSS Grid et Flexbox.
- **Animations** : intégration de **Framer Motion** pour des transitions fluides.
- **Sécurité front-end** : filtrage des entrées utilisateur, cookies sécurisés (httpOnly, SameSite=Strict, Secure), et communication HTTPS.



[← Retour à l'accueil](#)

Authentification

Connexion

Inscription

 **Particulier**

 **Entreprise**

Informations personnelles

Prénom *

Nom *

Email *

coulet.corentin@gmail.com

Téléphone

+33 6 12 34 56 78

Date de naissance

jj/mm/aaaa 

Sécurité

Mot de passe *

.....

Minimum 8 caractères

Confirmer le mot de passe *

☐ J'accepte les [conditions d'utilisation](#) et la [politique de confidentialité](#)

S'inscrire comme particulier

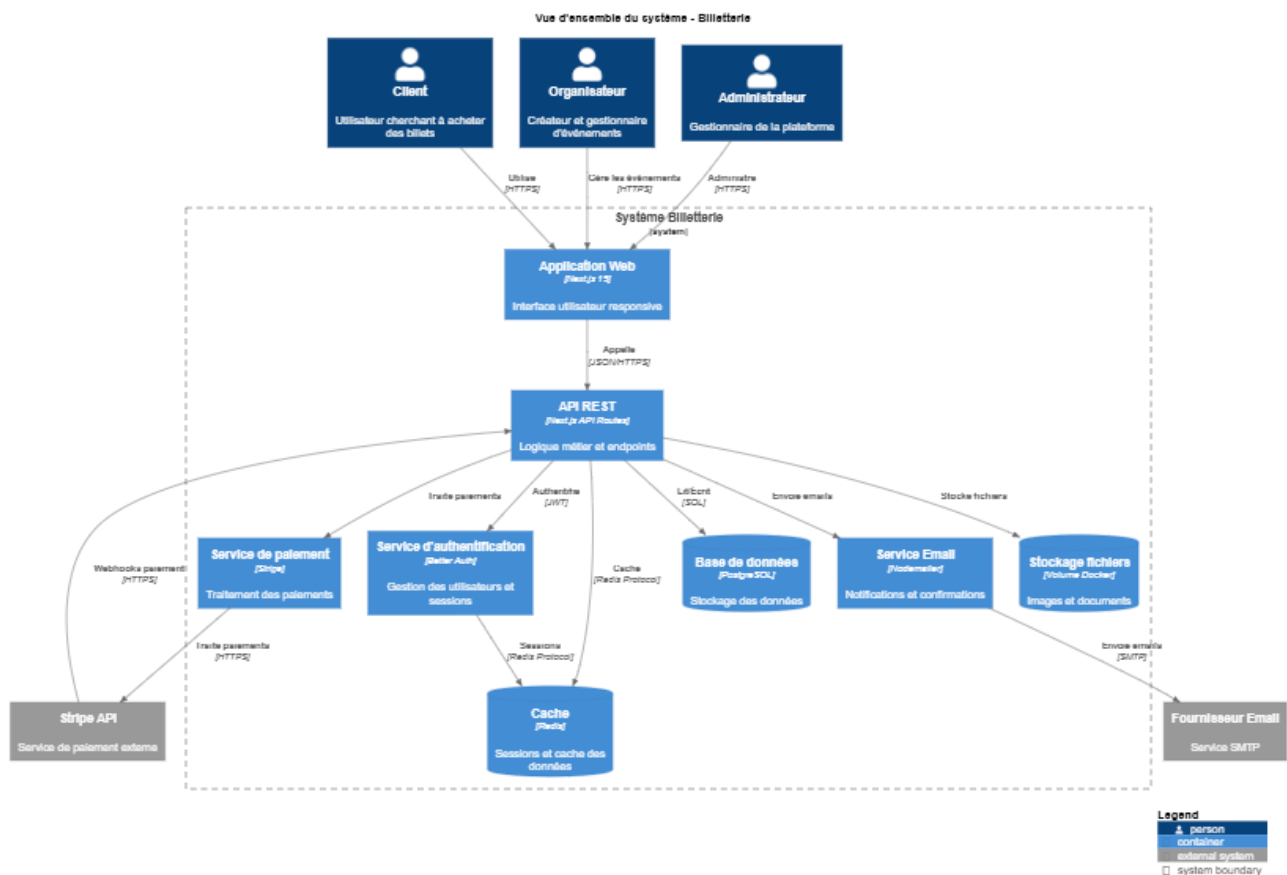
1.1.3. Développer des composants métier

Les composants métier constituent le cœur fonctionnel de M&C Society. Ils sont organisés en **services** spécialisés, assurant une séparation claire entre les responsabilités :

- **UserService** : gestion des utilisateurs, authentification, rôles et permissions.
- **EventService** : création, mise à jour et publication des événements.
- **OrderService** : gestion du panier, création de commandes, intégration Stripe pour les paiements sécurisés.
- **TicketService** : génération et validation des QR codes, rotation automatique pour éviter la fraude.

Chaque service applique les **principes SOLID** et les **bonnes pratiques OWASP**, avec validation des données via **Zod**.

L'architecture garantit la traçabilité et la maintenabilité du code tout en minimisant les risques d'injections ou d'erreurs d'état.



1.1.4. Contribuer à la gestion d'un projet informatique

Le projet a été mené seul, en suivant une **méthodologie agile inspirée de Scrum** :

- **Organisation des tâches** dans Trello : backlog → sprint → en cours → terminé.
- **Suivi GitHub** : commits détaillés, versioning sémantique (SemVer), changelog et releases documentées.
- **Documentation** : rédaction complète dans le dossier /docs (installation, configuration, sécurité, API, tests, déploiement).
- **Tests automatisés** : Jest (unitaires), Playwright (E2E), Stryker (mutation testing).
- **Qualité logicielle** : ESLint, Prettier, Husky (pre-commit hooks) et GitHub Actions pour les checks automatiques.

Cette organisation a permis une visibilité complète sur la progression du projet, une réduction des risques et un contrôle de la qualité continue.

The screenshot shows the GitHub repository page for 'M-C-Billetterie' by user 'CorentinCoulet'. The repository is public and has 19 branches and 0 tags. The main branch is 'main'. The repository has 59 commits, last updated 2 weeks ago. The file explorer on the left shows a directory structure with folders like .backups-console-logs/src/lib/secrets, .github, .secrets, app, config, diagrams, docker, docs, infrastructure, k8s, monitoring, prisma, public, scripts, src, and tests. The right sidebar shows the 'About' section with no description, website, or topics provided. It also includes links to Readme, Contributing, Security policy, Activity, stars, watching, and forks. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published. The 'Languages' section shows a bar chart with the following data:

Language	Percentage
TypeScript	86.8%
JavaScript	3.9%
PowerShell	3.9%
Handlebars	1.3%
Shell	1.0%
Python	0.7%
Other	0.4%

1.2. Concevoir et développer une application sécurisée organisée en couches

1.2.1. Analyser les besoins et maquetter une application

L'analyse fonctionnelle a été menée à partir du besoin principal : permettre la **gestion complète d'événements et de billetterie en ligne** pour trois types d'utilisateurs (visiteur, organisateur, administrateur).

Les besoins ont été traduits en **diagrammes UML et MERISE** (cas d'utilisation, classes, entités-relation) réalisés avec **PlantUML**.

Les maquettes visuelles (wireframes) ont servi à valider les parcours utilisateurs avant le développement.

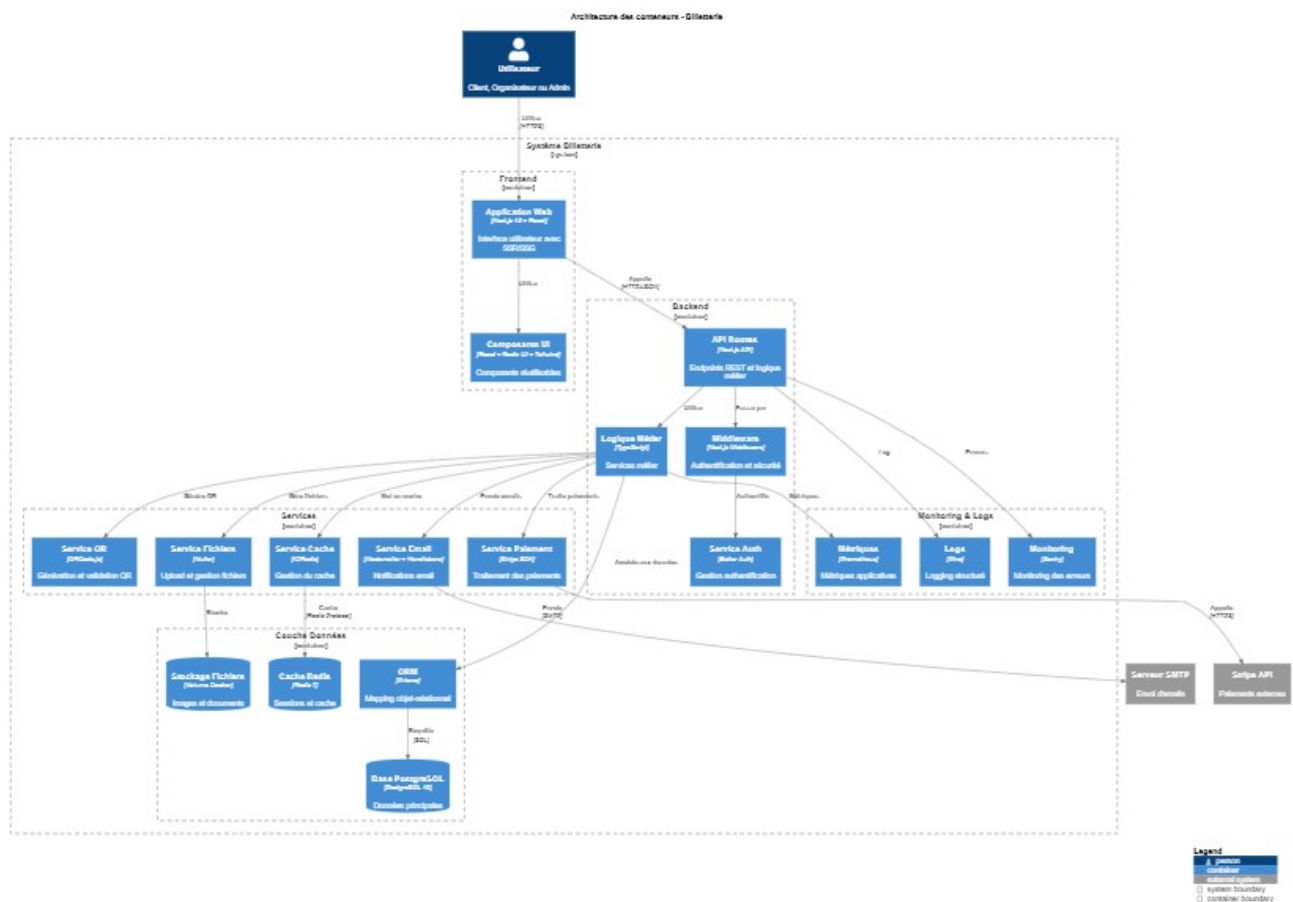
1.2.2. Définir l'architecture logicielle d'une application

L'application adopte une **architecture multicouche hybride** adaptée à Next.js 15 :

- **Présentation (UI)** : pages et composants React.
- **Contrôleurs** : routes API Next.js jouant le rôle d'adaptateurs entre l'interface et la logique métier.
- **Services** : logique métier centralisée et réutilisable.
- **Repositories** : accès aux données via Prisma ORM.

Cette architecture garantit la **séparation des responsabilités**, la **scalabilité**, la **testabilité** et la **sécurité** des échanges entre les couches.

Le système de rôles granulaire (USER / ORGANIZER / ADMIN) repose sur un middleware **RBAC** robuste.



1.2.3. Concevoir et mettre en place une base de données relationnelle

La base de données **PostgreSQL** a été modélisée en trois étapes :

- **MCD / MLD / MPD** avec **MERISE**.
- **Implémentation Prisma** via un fichier `schema.prisma` fortement typé.
- **Migrations automatiques** assurant la cohérence du modèle avec le code.

Les principales entités :

User, Event, Order, Ticket, Organization, Payment, Review.

Des contraintes d'intégrité et d'unicité ont été définies, ainsi que des clés étrangères explicites.

L'utilisation de Prisma assure une forte sécurité contre les injections SQL et simplifie la maintenance du modèle.

```

1  enum OrderStatus {
2      draft
3      pending_payment
4      paid
5      cancelled
6  }
7
8  enum TicketStatus {
9      pending
10     paid
11     cancelled
12     used
13 }
14
15 enum Role {
16     USER
17     ORGANIZER
18     ADMIN
19 }
20
21 model User {
22     id          String          @id @default(uuid())
23     email       String          @unique
24     name        String?
25     password    String
26     isVerified  Boolean          @default(false)
27     emailVerifiedAt DateTime?
28     lastLogin   DateTime?
29     passwordChangedAt DateTime @default(now()) // For password rotation
30     role        Role            @default(USER)
31     tickets     Ticket[]
32     orders      Order[]
33     userSessions UserSession[] // Secure sessions only
34     reviews     Review[]
35     teamRoles    TeamMember[]
36     logs         ActivityLog[]
37     notifications Notification[]
38     eventLogs    EventLog[]
39     blocked      BlockedUser?
40     passwordHistory PasswordHistory[]
41     auditLogs     AuditLog[]
42     loginAttempts LoginAttempt[]
43     metadata     Json?
44     createdAt    DateTime        @default(now())
45     updatedAt    DateTime        @updatedAt
46 }
47
48 model Organizer {
49     id          String          @id @default(uuid())
50     name        String
51     events      Event[]
52     team        TeamMember[]
53     createdAt   DateTime        @default(now())
54     updatedAt   DateTime        @updatedAt
55 }
56
57 model Event {
58     id          String          @id @default(uuid())
59     title        String
60     description   String?
61     date         DateTime
62     location     String
63     maxCapacity  Int?
64     isPublished  Boolean @default(false)
65     isCancelled  Boolean @default(false)
66     allowAnonymousPurchase Boolean @default(false)
67     allowTransfer Boolean @default(false)
68     tickets     Ticket[]
69     reviews     Review[]
70     category    Category? @relation(fields: [categoryId], references: [id])
71     categoryId   String?
72     venue       Venue? @relation(fields: [venueId], references: [id])
73     venueId     String?
74     organizer   Organizer @relation(fields: [organizerId], references: [id])
75     organizerId String
76     theme       Theme? @relation(fields: [themeId], references: [id])
77     themeId     String?
78     metadata     Json?
79     createdAt    DateTime @default(now())
80     updatedAt    DateTime @updatedAt
81 }
82
83 model Order {
84     id          String          @id @default(uuid())
85     user        User            @relation(fields: [userId], references: [id])
86     userId      String
87     tickets     Ticket[]
88     payment     Payment?
89     totalPrice  Float
90     status      OrderStatus
91     promoCode   String?
92     discountAmount Float?
93     currency    String          @default("EUR")
94     metadata     Json?
95     createdAt    DateTime        @default(now())
96     updatedAt    DateTime        @updatedAt
97 }
98
99 model Ticket {
100    id          String          @id @default(uuid())
101    user        User?           @relation(fields: [userId], references: [id])
102    userId      String?
103    event       Event           @relation(fields: [eventId], references: [id])
104    eventId     String
105    order       Order?          @relation(fields: [orderId], references: [id])
106    orderId     String?
107    code        String          @unique
108    status      TicketStatus @default(paid)
109    seatNumber  String?
110
111    currentQRCode String?
112    qrCodeGeneratedAt DateTime?
113    qrRotationInterval Int @default(12)
114    isScanned    Boolean @default(false)
115    scannedAt    DateTime?
116
117    usedAt       DateTime?
118    purchasedAt  DateTime @default(now())
119    metadata     Json?
120 }
121
122 model Payment {
123     id          String          @id @default(uuid())
124     order       Order           @relation(fields: [orderId], references: [id])
125     orderId     String          @unique
126     paymentMethod String
127     paymentStatus String
128     paymentDate  DateTime
129     transactionId String
130     currency     String          @default("EUR")
131 }
132

```

1.2.4. Développer des composants d'accès aux données SQL et NoSQL

Les composants d'accès aux données ont été développés via **Prisma ORM**, garantissant des requêtes typées et sécurisées :

- CRUD complet sur toutes les entités.
- Transactions atomiques (`prisma.$transaction`) pour la création simultanée de commandes et de billets.
- Requêtes optimisées avec sélections partielles (`select`) et relations (`include`).
- Protection contre les injections grâce aux requêtes préparées automatiques.

L'utilisation de Prisma Studio a permis de valider visuellement les données pendant le développement.



```
1  await prisma.$transaction(async (tx) => {
2    const [payment, order] = await Promise.all([
3      tx.payment.findUnique({
4        where: { id: paymentId },
5        include: { order: true }
6      }),
7      tx.order.findUnique({
8        where: { id: orderId },
9        include: { tickets: true }
10     })
11   ]);
12
13   const updatedPayment = await tx.payment.update({
14     where: { id: paymentId },
15     data: {
16       paymentStatus: PaymentStatus.COMPLETED,
17       transactionId: paymentIntentId,
18       paymentDate: new Date()
19     }
20   });
21
22   await tx.order.update({
23     where: { id: orderId },
24     data: { status: 'paid' }
25   });
26
27   await tx.ticket.updateMany({
28     where: { orderId },
29     data: {
30       status: 'paid',
31       purchasedAt: new Date()
32     }
33   });
34
35   return await tx.payment.findUnique({
36     where: { id: paymentId },
37     include: {
38       order: {
39         include: {
40           user: {
41             select: { id: true, name: true, email: true }
42           },
43           tickets: {
44             include: { event: true }
45           }
46         }
47       }
48     }
49   });
50 });
```

1.3. Préparer le déploiement d'une application sécurisée

1.3.1. Préparer et exécuter les plans de tests d'une application

Le projet intègre une **stratégie de tests complète** :

- **Unitaires** (Jest) : logique métier isolée.
- **Intégration** (Jest + Supertest) : interactions entre API et base.
- **End-to-End** (Playwright) : parcours utilisateur complet (achat, validation, paiement).
- **Tests d'accessibilité** : intégration de axe DevTools.
- **Mutation testing** avec **Stryker** pour valider la robustesse des tests.

Un rapport de couverture supérieur à **80 %** a été atteint.

- Environment Variables Tests > should have DATABASE_URL defined

```
expect(received).toBeDefined()
```

Received: **undefined**

```

10 |
11 |   it('should have DATABASE_URL defined', () => {
> 12 |     expect(process.env.DATABASE_URL).toBeDefined();
    |                               ^
13 |     expect(process.env.DATABASE_URL).toContain('postgresql://');
14 |   });
15 |

```

at Object.<anonymous> (tests/unit/env.test.ts:12:38)

FAIL tests/security/basic-security.test.ts

- Basic Security Tests > Environment variables should be properly configured for security

```
expect(received).toBeDefined()
```

Received: **undefined**

```

142 |     // In test environment, these should be set
143 |     expect(process.env.NODE_ENV).toBe('test');
> 144 |     expect(process.env.JWT_SECRET).toBeDefined();
    |                               ^
145 |     expect(process.env.JWT_SECRET!.length).toBeGreaterThan(30);
146 |   });
147 |

```

at Object.<anonymous> (tests/security/basic-security.test.ts:144:36)

Test Suites: **17 failed**, **51 passed**, 68 total
 Tests: **26 failed**, **1210 passed**, 1236 total
 Snapshots: **35 passed**, 35 total
 Time: 68.387 s
 Ran all test suites.

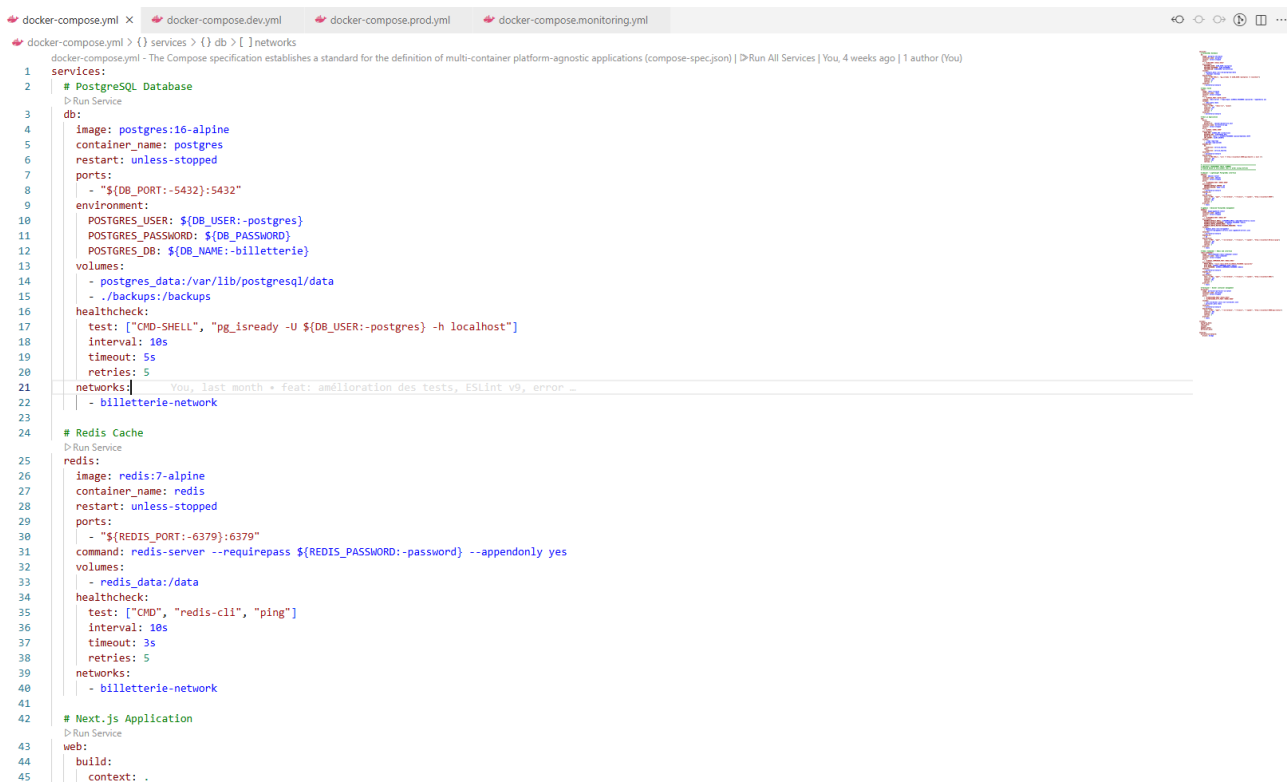
PS F:\projets persos\billetterie project\billetterie-complete> []

1.3.2. Préparer et documenter le déploiement d'une application

Le déploiement repose sur **Docker** et **Docker Compose**, permettant de reproduire fidèlement les environnements de développement, test et production.

La documentation comprend :

- Fichiers Dockerfile et docker-compose.yml.
- Variables d'environnement pour PostgreSQL et les services externes.
- Procédure d'installation détaillée (docs/deployment.md).
- Commandes automatisées (npm run build, npm run start).



```

1 services:
2   # PostgreSQL Database
3   db:
4     image: postgres:16-alpine
5     container_name: postgres
6     restart: unless-stopped
7     ports:
8       - "${DB_PORT:-5432}:5432"
9     environment:
10      POSTGRES_USER: ${DB_USER:-postgres}
11      POSTGRES_PASSWORD: ${DB_PASSWORD}
12      POSTGRES_DB: ${DB_NAME:-billetterie}
13     volumes:
14       - postgres_data:/var/lib/postgresql/data
15       - ./backups:/backups
16     healthcheck:
17       test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-postgres} -h localhost"]
18       interval: 10s
19       timeout: 5s
20       retries: 5
21   networks:
22     - billetterie-network
23
24   # Redis Cache
25   redis:
26     image: redis:7-alpine
27     container_name: redis
28     restart: unless-stopped
29     ports:
30       - "${REDIS_PORT:-6379}:6379"
31     command: redis-server --requirepass ${REDIS_PASSWORD:-password} --appendonly yes
32     volumes:
33       - redis_data:/data
34     healthcheck:
35       test: ["CMD", "redis-cli", "ping"]
36       interval: 10s
37       timeout: 3s
38       retries: 5
39     networks:
40       - billetterie-network
41
42   # Next.js Application
43   web:
44     build:
45     context: .

```

1.3.3. Contribuer à la mise en production dans une démarche DevOps

Une démarche **DevOps** a été adoptée dès la conception :

- **Monitoring** : stack Prometheus + Grafana pour suivre les performances.
- **Logging** : journalisation structurée (format JSON) des événements et erreurs critiques.
- **Alertes automatisées** : détection des anomalies applicatives.

- **Sécurité continue** : audit OWASP, rotation des clés JWT, nettoyage automatique des sessions expirées.

Cette approche permet une supervision proactive et une maintenance facilitée du système en production.

2. Cahier des charges

2.1. Description de l'existant

La gestion d'événements implique souvent l'utilisation de plusieurs outils distincts pour les réservations, la billetterie, le suivi commercial et la relation client. Cette fragmentation peut nuire à la fluidité des opérations, ralentir la prise de décision et dégrader l'expérience des utilisateurs.

Le projet M&C Society répond à ce constat en proposant une solution centralisée et intuitive, pensée pour simplifier l'organisation d'événements tout en offrant aux utilisateurs finaux une expérience fluide et cohérente. L'objectif est de permettre aux organisateurs de gagner en autonomie, en efficacité et en visibilité sur leurs performances.

2.2. Reprise de l'existant

Il n'existe aucune version antérieure du projet. Il s'agit d'une création ex nihilo. Aucun nom de domaine ni hébergement n'est encore attribué. Aucun patrimoine documentaire n'est à reprendre : ni ancien cahier des charges, ni spécification fonctionnelle.

2.3. Principes de référencement

Le projet accorde une importance stratégique au référencement naturel (SEO) afin d'optimiser la visibilité de la plateforme dans les moteurs de recherche et de maximiser la découvrabilité des événements publics. Cette approche repose sur plusieurs piliers techniques complémentaires.

L'architecture des URLs adopte une structure sémantique et hiérarchique, avec des slugs descriptifs intégrant naturellement des mots-clés pertinents. Chaque page événementielle dispose de métadonnées uniques (title, description) et intègre des balises structurées selon le standard Schema.org, notamment via JSON-LD pour les événements et les lieux. Ces données enrichies permettent l'affichage de rich snippets dans les résultats de recherche, incluant dates, prix et disponibilités.

Un plan de site dynamique (sitemap.xml) est généré automatiquement à chaque publication ou modification d'événement, facilitant l'indexation par les moteurs de recherche. Le rendu côté serveur (SSR) avec Next.js garantit une indexation efficace du contenu, tandis que l'optimisation des performances (Core Web Vitals) et la structure HTML sémantique répondent aux critères de qualité des algorithmes modernes.

L'ensemble de ces dispositions vise à générer du trafic qualifié vers les pages événementielles tout en offrant une expérience utilisateur qui satisfait les exigences techniques et éditoriales du référencement naturel.

2.4. Exigences de performances et de volumétrie

La plateforme vise à accueillir jusqu'à 10 000 visiteurs uniques par mois en phase de lancement, avec une montée en charge progressive jusqu'à 50 000 utilisateurs mensuels. Le temps de réponse moyen attendu est inférieur à 300 millisecondes pour les actions courantes (navigation, consultation d'un événement). Une disponibilité de 99,5 % est exigée. Le catalogue pourra regrouper jusqu'à 1 000 événements actifs simultanément, avec des pics de consultation à prévoir lors des lancements ou campagnes de communication.

2.5. Multilinguisme & adaptations pour un public spécifique

Multilinguisme

Le site sera initialement disponible en français, avec une ouverture à l'anglais prévue dans une future phase. Le contenu sera géré de manière à faciliter les traductions ultérieures via un système d'internationalisation (i18n).

Accessibilité Numérique - Conformité Réglementaire

L'application respecte les normes WCAG 2.1 niveau AA (Web Content Accessibility Guidelines) et s'aligne sur les exigences du RGAA 4.1 (Référentiel Général d'Amélioration de l'Accessibilité) français, conformément à la directive européenne 2016/2102 et à la loi du 11 février 2005 pour l'égalité des droits et des chances.

La plateforme garantit une perception optimale du contenu pour tous les utilisateurs, notamment à travers des contrastes de couleurs respectant un ratio minimum de 4.5:1 pour le texte standard et 3:1 pour le texte large, conformément au critère WCAG 1.4.3. La palette de couleurs a été validée avec Tailwind CSS, et toutes les images fonctionnelles disposent d'alternatives textuelles descriptives via des attributs alt, tandis que les éléments purement décoratifs sont masqués aux technologies d'assistance avec `aria-hidden="true"`.

L'utilisabilité est assurée par une navigation clavier complète permettant d'accéder à tous les composants interactifs via les touches Tab, Enter et Space, avec un focus visible personnalisé et un ordre de tabulation logique. Les sessions utilisateur disposent de durées configurables avec des alertes avant expiration, et la navigation reste cohérente sur l'ensemble du site grâce à un menu principal fixe et des boutons de retour explicites sur chaque page.

La compréhensibilité repose sur une structure sémantique HTML5 rigoureuse utilisant les balises `<header>`, `<main>`, `<nav>`, `<section>` et `<footer>`, avec une hiérarchie de titres respectée et des attributs ARIA (`aria-labelledby`, `aria-describedby`) pour les sections critiques. Tous les formulaires associent des labels explicites aux champs de saisie, et les messages d'erreur sont contextuels et constructifs. Les composants dynamiques (modales, notifications, onglets) intègrent les rôles ARIA appropriés (`role="dialog"`, `role="alert"`, `role="tabpanel"`) pour une annonce vocale correcte par les lecteurs d'écran.

La robustesse technique est garantie par l'utilisation de composants Shadcn UI v4 basés sur Radix UI, reconnus pour leur conformité ARIA native. La plateforme a été testée avec les principaux lecteurs d'écran (NVDA, JAWS, VoiceOver) et le code HTML5 est validé W3C pour assurer une compatibilité maximale avec les technologies d'assistance.

Pour les personnes en situation de handicap visuel, le site supporte un zoom navigateur jusqu'à 200% sans perte de fonctionnalité, propose un mode sombre automatique basé sur les préférences système, et utilise des tailles de police relatives permettant la personnalisation. Les utilisateurs aveugles bénéficient d'une navigation 100% clavier avec lecteur d'écran, de textes alternatifs détaillés et de liens d'évitement (skip links). L'information n'est jamais transmise par la couleur seule pour accommoder les personnes daltoniennes, avec une combinaison systématique d'icônes et de textes pour les statuts critiques.

Les handicaps moteurs sont pris en compte avec des cibles tactiles minimales de 44x44 pixels sur mobile (WCAG 2.5.5), des zones cliquables étendues pour les boutons critiques et l'absence de gestes complexes requis. Pour les handicaps cognitifs, le langage employé reste simple et structuré selon les principes du FALC (Facile À Lire et à Comprendre), avec des messages d'erreur constructifs proposant des solutions et des confirmations systématiques pour les actions irréversibles.

La conformité est vérifiée continuellement grâce à des tests automatisés intégrant axe DevTools dans les tests Playwright et Jest avec Testing Library (1277 tests passants couvrant l'accessibilité). Des audits manuels trimestriels sont réalisés avec les grilles RGAA, et des tests utilisateurs avec personnes en situation de handicap sont planifiés pour T4 2025. Une déclaration d'accessibilité publique sera créée conformément au décret n° 2019-768, accompagnée d'un schéma pluriannuel de mise en accessibilité et d'un plan d'actions annuel avec indicateurs de suivi.

L'objectif est d'atteindre 80% de conformité WCAG 2.1 AA au lancement, puis 100% d'ici fin 2026, avec des audits externes certifiés pour valider la conformité RGAA 4.1 niveau Or (équivalent WCAG AAA sur critères sélectionnés).

2.6. Description graphique et ergonomique

M&C Billetterie adopte un design moderne et élégant centré sur l'expérience utilisateur, alliant esthétique professionnelle et accessibilité.

Identité Visuelle

L'interface repose sur un système de couleurs analogues dominé par des nuances de bleu profond évoquant la confiance et le professionnalisme. Les accents bleu-violacé créent une ambiance premium adaptée aux événements culturels.

Composants UI

L'application utilise Shadcn UI v4 avec 44 composants réutilisables (buttons, cards, dialogs, forms, etc.) garantissant une cohérence graphique totale. Le système intègre des effets de glassmorphism (.glass-card avec backdrop-blur-lg et transparence) pour un rendu visuel sophistiqué et immersif.

Typographie & Hiérarchie

La police Inter (Google Fonts) structure le contenu selon une hiérarchie claire :

- Titres principaux (H1) : 3-5xl bold
- Sections (H2) : 2-3xl semibold
- Contenu : base regular
- Métadonnées : small medium

Architecture Responsive

Approche mobile-first avec adaptation desktop fluide grâce à CSS Grid et Flexbox. Navigation mobile optimisée avec menu burger overlay, cartes empilées et boutons tactiles dimensionnés pour l'interaction tactile.

Animations & Interactions

Framer Motion assure des transitions fluides entre pages et des animations d'entrée subtiles. Les interactions bénéficient d'un feedback visuel immédiat (hover, focus, loading states) pour une expérience réactive et intuitive.

Ergonomie Fonctionnelle

Le parcours utilisateur suit un flux linéaire optimisé : Landing → Discovery (filtres/recherche) → Sélection → Panier → Profil. Chaque étape minimise les frictions grâce à une gestion d'état centralisée (Zustand via useKV) pour la persistance des favoris, du panier et des sessions.

L'ensemble crée une plateforme accessible, performante et visuellement cohérente, alliant design raffiné et fonctionnalité efficace pour une expérience de réservation fluide.

2.6.1. Composants de la charte graphique

L'identité visuelle du site repose sur une charte moderne en bleu, noir et blanc. Les éléments graphiques incluent :

- Un logo sobre et distinctif
- Une police lisible sans-serif, avec des tailles adaptées selon les supports
- Une palette principale : bleu clair, noir profond, blanc neutre
- Une cohérence graphique assurée sur toutes les pages

2.6.2. Design, Responsive design et autres exigences liées au design

Le site devra être **entièrement responsive** et offrir une **expérience utilisateur fluide** sur desktop, tablette et mobile. Le style visuel recherchée est un **design moderne**, légèrement minimaliste, avec des animations légères. La navigation sera claire, avec des appels à l'action visibles et une hiérarchie bien définie des contenus.

2.7. Besoins fonctionnels « métier »

2.7.1. Utilisateurs du projet

Les types d'utilisateurs concernés sont :

- Les visiteurs : navigation libre, inscription, achat de billets
- Les utilisateurs authentifiés : gestion de leur profil, historique d'achats
- Les organisateurs : création, édition et gestion d'événements
- Les administrateurs : modération, gestion des utilisateurs, statistiques

2.7.2. Informations relatives aux contenus

Les contenus disponibles incluent : fiches événements, visuels (affiches, photos), descriptions textuelles, vidéos intégrées, documents à télécharger (conditions générales, programmes). Le site respectera le RGPD : les données personnelles seront protégées, une politique de confidentialité sera affichée et le consentement explicite demandé pour la collecte de données.

2.7.3. Inventaire des besoins fonctionnels

Thème	Qui	Quoi	Pourquoi
Authentification	Utilisateur	Créer un compte	Participer à un événement
Authentification	Utilisateur	Se connecter / Se déconnecter	Gérer ses billets
Profil	Utilisateur	Modifier ses informations	Mettre à jour ses données
Gestion événements	Organisateur	Créer un événement	Le rendre visible au public
Billetterie	Visiteur	Acheter un billet	Participer à un événement
Administration	Admin	Modérer un événement	Maintenir la qualité du service
Statistiques	Admin	Consulter les indicateurs clés	Suivre les performances

Fonctionnalités supplémentaires envisagées :

- Espace client avec accès sécurisé
- Système de notification (email)
- Intégration d'un moteur de recherche interne
- Interface d'administration avec tableaux de bord
- Règles d'accès en fonction des rôles

2.8. Budget

Le budget total prévisionnel est estimé à **90 jours/homme** répartis comme suit :

Phase	Durée estimée
Analyse & Conception	15 j/h
Développement back-end	25 j/h
Développement front-end	20 j/h
Intégration & Tests	20 j/h
Documentation & Livraison	5 j/h
Réserves & imprévus	5 j/h

3. Présentation de l'entreprise et du service

3.1. Présentation de l'entreprise et du service

Présentation de l'entreprise

M&C Billetterie est une jeune société française fondée en 2025, spécialisée dans les solutions de billetterie en ligne pour événements culturels, sportifs et professionnels de petite à moyenne envergure. Notre mission est de rendre la vente et le contrôle des billets simples, rapides et fiables pour les organisateurs comme pour le public. Basée en France et opérant en mode SaaS, M&C Billetterie s'adresse aux salles de spectacle, associations, festivals locaux, clubs sportifs, écoles et organisateurs d'événements B2B (salons, conférences). L'équipe, à taille humaine, réunit des profils techniques et opérationnels (produit, développement, support client, commercial) avec une culture de service et de transparence.

Présentation du service

La plateforme M&C Billetterie couvre l'ensemble du cycle de vie d'un événement: création (fiche événement, jauge, catégories de tarifs), mise en vente (page de vente responsive, paniers, codes promo), paiement sécurisé via un prestataire certifié, émission immédiate des billets (PDF/Wallet) et contrôle d'accès sur site (scan de QR codes via application mobile). Côté organisateur, un espace d'administration centralisé permet de suivre les ventes en temps réel, gérer les équipes (rôles simples), envoyer des confirmations et exporter les données essentielles (commandes, listes d'accès). Côté public, le parcours d'achat est pensé pour être clair et rapide: choix des billets, paiement, réception du billet et informations pratiques. La solution s'intègre avec les outils courants (emailing, paiement) et propose une API simple pour les besoins de base.

Positionnement et valeur apportée

M&C Billetterie mise sur la simplicité d'usage, la fiabilité opérationnelle les jours "chauds", et la proximité avec les organisateurs. Par rapport aux plateformes généralistes, notre proposition de valeur tient dans un outil léger, rapide à déployer, au coût maîtrisé et avec un accompagnement direct (support réactif, conseils de configuration). Pour les organisateurs, les bénéfices se mesurent en gain de temps, réduction des erreurs de contrôle à l'entrée et meilleure visibilité sur les ventes. Pour le public, l'expérience est fluide et sécurisée.

Engagements et conformité

Les données sont hébergées en Europe et traitées conformément au RGPD (finalités claires, minimisation, droits des personnes). Les paiements sont délégués à un prestataire certifié (les données de carte ne transitent pas par nos serveurs). Nous suivons des indicateurs simples et concrets: disponibilité de la plateforme, temps de réponse du tunnel d'achat, taux d'échec de paiement, et taux de satisfaction des organisateurs.

Modèle économique

Le modèle est volontairement transparent: une commission par billet émis et, selon les besoins, un petit abonnement mensuel pour activer des fonctionnalités additionnelles (contrôle d'accès avancé, exports étendus, intégration API). Aucun frais caché, et des tarifs adaptés au volume des événements.

3.2. Objectifs du projet

M&C Billetterie est une plateforme de réservation d'événements culturels pensée pour répondre aux besoins de trois acteurs clés. Pour l'utilisateur final, l'objectif est d'offrir une expérience d'achat de billets simple et rassurante, où il peut découvrir des événements via une recherche intuitive, sauvegarder ses favoris, gérer son panier en toute fluidité et recevoir instantanément ses billets avec codes QR sécurisés par email, le tout dans une interface moderne accessible depuis son smartphone ou son ordinateur.

Du côté des organisateurs d'événements, la plateforme devient un outil de gestion puissant permettant de créer et administrer leurs événements, de définir les tarifs et catégories de billets, de suivre les ventes en temps réel et de valider les entrées grâce au système de codes QR qui se régénèrent automatiquement toutes les 12 heures pour éviter la fraude. Les paiements Stripe intégrés garantissent des transactions sécurisées tandis que le système d'emails automatisés maintient une communication professionnelle avec les participants sans effort supplémentaire.

3.3. Cible adressée par le projet

M&C Billetterie s'adresse principalement à deux publics distincts aux besoins complémentaires. La première cible regroupe les amateurs d'événements culturels de tous âges qui recherchent une solution moderne et fiable pour réserver leurs billets de spectacles, concerts, festivals ou expositions. Ces utilisateurs privilégient la simplicité d'utilisation et attendent une expérience fluide depuis la découverte d'un événement jusqu'à la réception de leur billet électronique, avec la possibilité de gérer facilement leurs réservations depuis n'importe quel appareil. Ils valorisent particulièrement la sécurité des paiements, la clarté des informations sur les événements et la rapidité du processus d'achat, tout en appréciant des fonctionnalités pratiques comme la sauvegarde de favoris ou l'historique de leurs achats.

La seconde cible englobe les organisateurs et producteurs d'événements culturels, qu'il s'agisse de petites associations locales, de salles de spectacle indépendantes ou de producteurs professionnels gérant plusieurs événements simultanément. Ces acteurs ont besoin d'une plateforme complète leur permettant de promouvoir leurs événements, de gérer plusieurs catégories de tarifs, de suivre leurs ventes en direct et de contrôler efficacement les accès lors des manifestations grâce à un système de validation sécurisé. Ils recherchent également une solution qui minimise les coûts de gestion administrative tout en maximisant la visibilité de leurs événements auprès d'un public qualifié, avec l'assurance d'encaisser leurs revenus de manière sécurisée et transparente via un système de paiement éprouvé.

3.4. Processus utilisateur compacté

Le parcours utilisateur sur M&C Billetterie se veut aussi naturel qu'efficace. Tout commence par l'arrivée sur la plateforme où le visiteur découvre immédiatement une sélection d'événements culturels présentés de manière attractive avec visuels, dates et catégories clairement identifiés. Il peut affiner sa recherche grâce à des filtres intuitifs par type d'événement, date ou lieu, puis consulter les pages détaillées qui regroupent toutes les informations essentielles comme la description, les horaires, le lieu exact et les différentes catégories de places disponibles avec leurs tarifs respectifs. Une fois son choix arrêté, l'utilisateur sélectionne ses billets, les ajoute à son panier où il peut encore ajuster les quantités, puis procède au paiement sécurisé via Stripe en quelques clics seulement. Immédiatement après la transaction, il reçoit par email une confirmation accompagnée de ses billets électroniques munis de codes QR uniques et sécurisés, qu'il pourra présenter directement depuis son smartphone le jour de

l'événement. Son espace personnel lui permet ensuite de retrouver facilement l'historique de ses achats, de gérer ses favoris pour ne manquer aucun événement qui l'intéresse, et de recevoir des notifications sur les nouveautés correspondant à ses préférences, créant ainsi une relation continue et personnalisée avec la plateforme.

Pour l'organisateur d'événements, M&C Billetterie propose un parcours dédié optimisé pour la gestion professionnelle. Après s'être authentifié avec son compte organisateur, il accède à son tableau de bord personnalisé où il peut créer un nouvel événement en renseignant l'ensemble des informations nécessaires comme le titre, la description détaillée, les visuels attractifs, le lieu, les dates et horaires de représentation. Il configure ensuite la billetterie en définissant les différentes catégories de places avec leurs tarifs spécifiques, les quotas disponibles par catégorie et les éventuelles conditions d'accès ou réductions applicables. Une fois l'événement publié sur la plateforme, l'organisateur peut suivre en temps réel l'évolution des ventes grâce à des statistiques détaillées affichant le nombre de billets vendus par catégorie, le chiffre d'affaires généré et les tendances de réservation. Le jour de l'événement, il utilise l'interface de validation pour scanner les codes QR des participants à leur arrivée, le système vérifiant instantanément l'authenticité de chaque billet et empêchant toute tentative de fraude grâce au mécanisme de rotation automatique des codes. Après la manifestation, l'organisateur retrouve dans son espace les rapports complets de fréquentation, les données financières consolidées et peut récupérer ses revenus via le système de paiement Stripe, tandis que les emails automatiques ont maintenu la communication avec les participants sans intervention manuelle de sa part, lui permettant de se concentrer pleinement sur la qualité de son événement.

4. Gestion de projet

Le projet a été mené de manière itérative en m'appuyant sur des outils de gestion modernes. L'objectif était de garder une vision claire de l'avancement, de suivre les priorités (MVP, documentation, tests etc), et d'assurer la traçabilité du code et des décisions techniques.

4.1. Intervenants sur le projet

M&C Billetterie est un projet développé intégralement en solo, reflétant une maîtrise complète du cycle de développement d'une application web professionnelle. Le développeur unique a assumé l'ensemble des responsabilités techniques, de l'architecture logicielle à l'implémentation frontend et backend, en passant par la conteneurisation Docker, la sécurisation avec WAF et authentification JWT, l'intégration des paiements Stripe, la mise en place du monitoring Prometheus-Grafana et la rédaction d'une documentation technique exhaustive. Cette approche solitaire a nécessité l'acquisition d'un large spectre de compétences couvrant aussi bien React et Next.js que PostgreSQL et Prisma, les tests automatisés avec Jest et Playwright, et les bonnes pratiques DevOps pour orchestrer les environnements multi-conteneurs. Le résultat témoigne d'une capacité à concevoir et livrer un projet cohérent de bout en bout, avec une couverture de tests de 80%, plus de quinze documents techniques détaillés et une infrastructure prête pour la production, démontrant rigueur méthodologique et souci constant de qualité professionnelle.

4.2. Méthodologie

M&C Billetterie a été développé selon une approche méthodologique rigoureuse combinant les meilleures pratiques du développement logiciel moderne. Le projet suit une architecture en couches clairement séparées avec une distinction nette entre la présentation (composants React), la logique métier (services et contrôleurs) et l'accès aux données (Prisma ORM), garantissant ainsi maintenabilité et évolutivité du code. L'adoption du principe "infrastructure as code" se manifeste par l'utilisation intensive de Docker et Docker Compose pour orchestrer les différents environnements (développement, test, production), permettant une reproductibilité parfaite des configurations et facilitant les déploiements. La qualité logicielle repose sur une

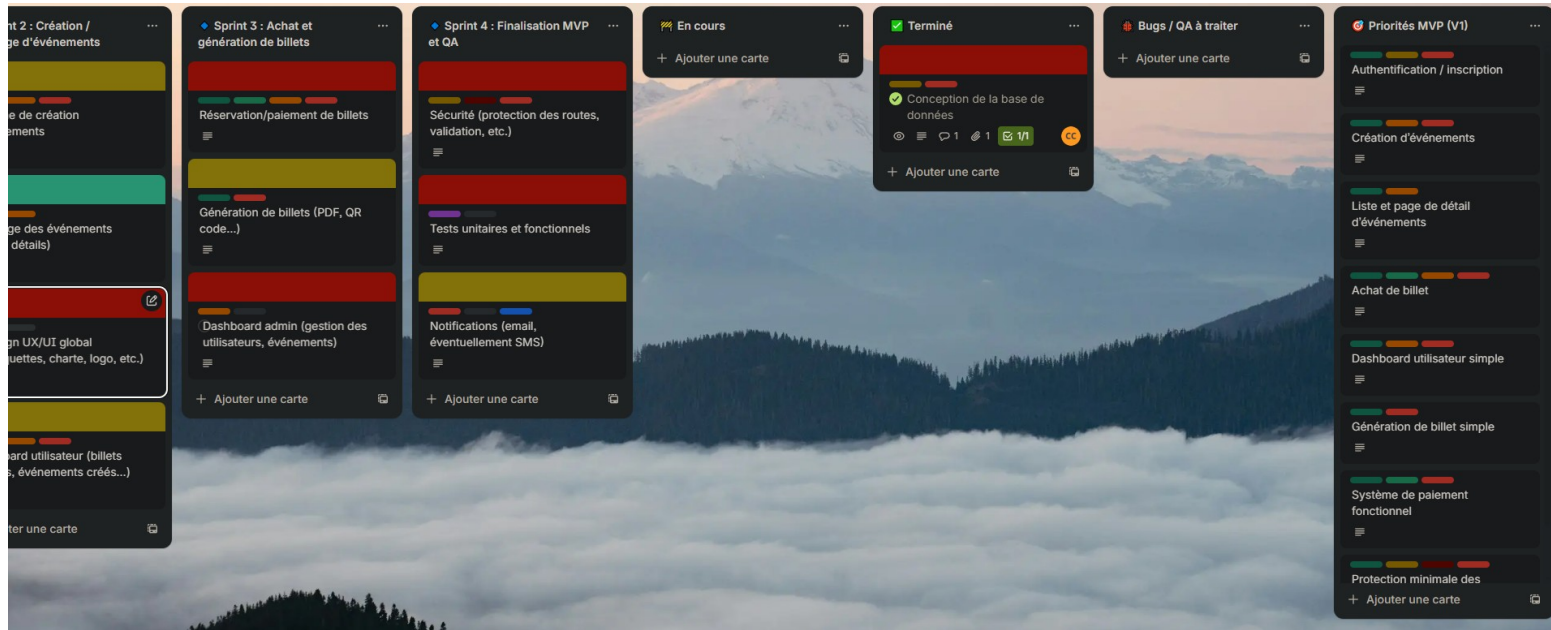
stratégie de tests exhaustive à plusieurs niveaux incluant tests unitaires avec Jest pour valider la logique métier isolée, tests d'intégration pour vérifier les interactions entre composants, tests end-to-end avec Playwright simulant des parcours utilisateurs complets, et même des tests de mutation avec Stryker pour évaluer la robustesse de la couverture de tests.

La sécurité a été intégrée dès la conception plutôt qu'ajoutée après coup, avec l'implémentation d'un Web Application Firewall utilisant ModSecurity, un système d'authentification JWT sécurisé avec rotation des tokens, un rate limiting intelligent protégeant contre les attaques par force brute, et un mécanisme innovant de codes QR rotatifs empêchant la réutilisation frauduleuse des billets. L'observabilité de la plateforme est assurée par une stack de monitoring complète Prometheus-Grafana collectant métriques système et métriques métier, des logs structurés facilitant le diagnostic des incidents, et des alertes automatiques notifiant l'équipe technique en cas d'anomalie. La documentation technique exhaustive organisée dans le dossier doc avec plus de quinze documents couvrant installation, configuration, API, sécurité, tests et déploiement, témoigne d'une volonté de faciliter la collaboration et la transmission de connaissance. Enfin, la gestion du projet s'appuie sur Git avec une stratégie de branches claire, un système de versioning sémantique suivant les conventions SemVer, et un changelog détaillé traçant l'évolution des fonctionnalités et corrections depuis la première version, assurant transparence et traçabilité tout au long du cycle de vie du projet.

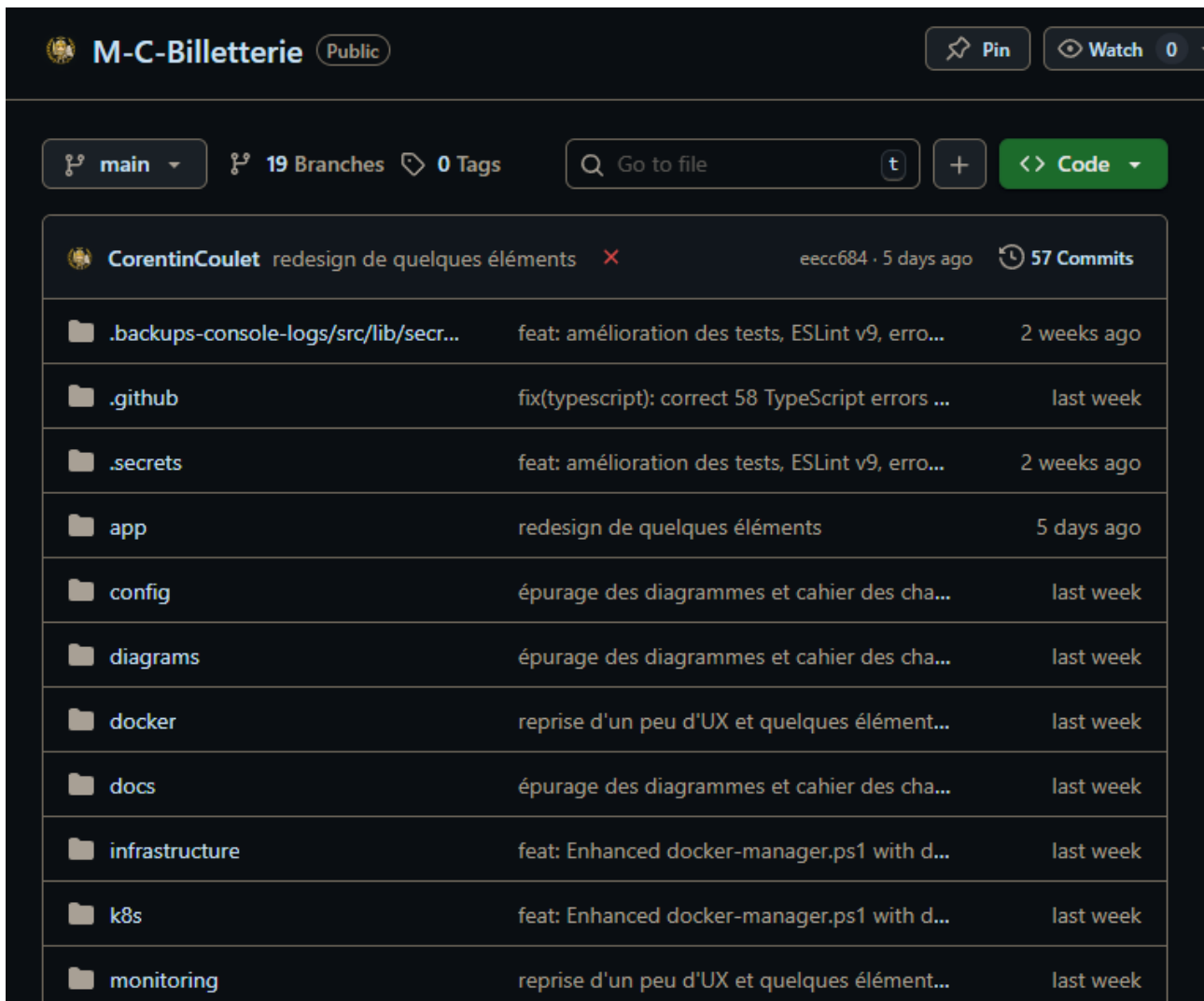
Le projet a suivi une approche **itérative et incrémentale** : les fonctionnalités ont été développées par lots, testées, puis intégrées progressivement. Cette méthode a permis d'ajuster certaines décisions en cours de route (organisation des DTOs, choix techniques, etc.).

4.3. Outils utilisés, plannig et suivi

- **Trello** : organisation des tâches en colonnes, permettant un suivi visuel et simple de l'avancement. (Backlog général, Sprint 1 Mise en place et fondations, Sprint 2 Création / Affichage d'évènements, Sprint 3 Achat et génération de billets, Sprint 4 Finalisation MVP et QA, En cours, Terminé, Bugs/QA à traiter, Priorités MVP V1)



- **GitHub** : gestion du code source, suivi des versions (Git), centralisation des commits et du travail effectué.



- **PlantUML (.puml)** : création des diagrammes (cas d'utilisation, activités, entités-relation, classes, architecture) afin d'assurer une modélisation claire et maintenable.
- Les fonctionnalités majeures ont été planifiées selon les phases définies dans le cahier des charges (analyse, conception, développement, tests, livraison).
- Chaque tâche correspondait à une carte Trello, avec passage dans les colonnes au fur et à mesure.
- Les versions du code ont été balisées par des commits réguliers sur GitHub.

4.4. Objectifs de qualité

- Une réserve de jours a été prévue pour gérer les imprévus (retards, corrections).
- Les diagrammes UML ont servi de référence pour valider les choix fonctionnels avant le codage.
- Les tests ont permis de confirmer la qualité des livrables avant la mise à disposition finale.

5. Spécifications fonctionnelles

L'application **M&C Society (M&C Billetterie)** repose sur une **architecture en couches**, séparant clairement :

- la logique métier (services et composants applicatifs),
- la couche d'accès aux données (PostgreSQL via Prisma ORM),
- et la couche de présentation (interface Next.js).

Les spécifications fonctionnelles qui suivent traduisent le cahier des charges établi précédemment.

5.1 Contraintes du projet et livrables attendus

5.1.1 Criticité de l'application

L'application est considérée comme **critique de niveau modéré**, car elle gère des données personnelles et des transactions financières.

Toute indisponibilité impacte directement la vente de billets et la satisfaction client.

Les exigences de disponibilité sont fixées à **99,5 %**, avec un temps de réponse moyen inférieur à **300 ms** sur les opérations courantes.

5.1.2 Applications connexes

L'application est autonome mais interagit avec plusieurs services externes :

- **Stripe** : gestion des paiements sécurisés.
- **SMTP / Mailer** : envoi des emails transactionnels (confirmation d'achat, réinitialisation de mot de passe, notifications).
- **API de géolocalisation** (future intégration) : aide à la recherche d'événements par lieu.

5.1.3 Services tiers

- **Stripe API** pour les paiements.

- **SendGrid / Nodemailer** pour l'emailing.
- **Cloudinary (ou équivalent)** pour le stockage des visuels d'événements.

5.1.4 Livrables attendus

- Application web responsive complète.
- Base de données PostgreSQL (migrations Prisma).
- Scripts de déploiement Docker.
- Documentation technique et utilisateur.
- Jeux de tests automatisés.

5.2 Architecture logicielle du projet

L'architecture suit un modèle multicouche clair :

1. **Front-end (Next.js)** : affichage et interaction utilisateur.
2. **API Routes (Next.js)** : gestion des contrôleurs REST.
3. **Services (TypeScript)** : logique métier centralisée.
4. **Repositories (Prisma)** : accès aux données PostgreSQL.

Cette séparation garantit une meilleure maintenabilité et une forte sécurité applicative.

Diagramme d'architecture :

Le système de billetterie s'articule autour d'une architecture web moderne qui distingue trois types d'utilisateurs principaux interagissant avec la plateforme. Le client final recherche et achète des billets, l'organisateur crée et gère ses événements, tandis que l'administrateur supervise l'ensemble de la plateforme. Ces trois profils accèdent au système via une interface web unique en HTTPS.

Au cœur du dispositif se trouve une application web développée avec Next.js 15 qui offre une interface utilisateur responsive. Cette application communique avec une API REST également construite sur Next.js via des API Routes, assurant ainsi une cohérence technologique entre le front-end et le back-end. L'API constitue le cerveau du système en orchestrant l'ensemble de la logique métier et en exposant les endpoints nécessaires aux différentes opérations.

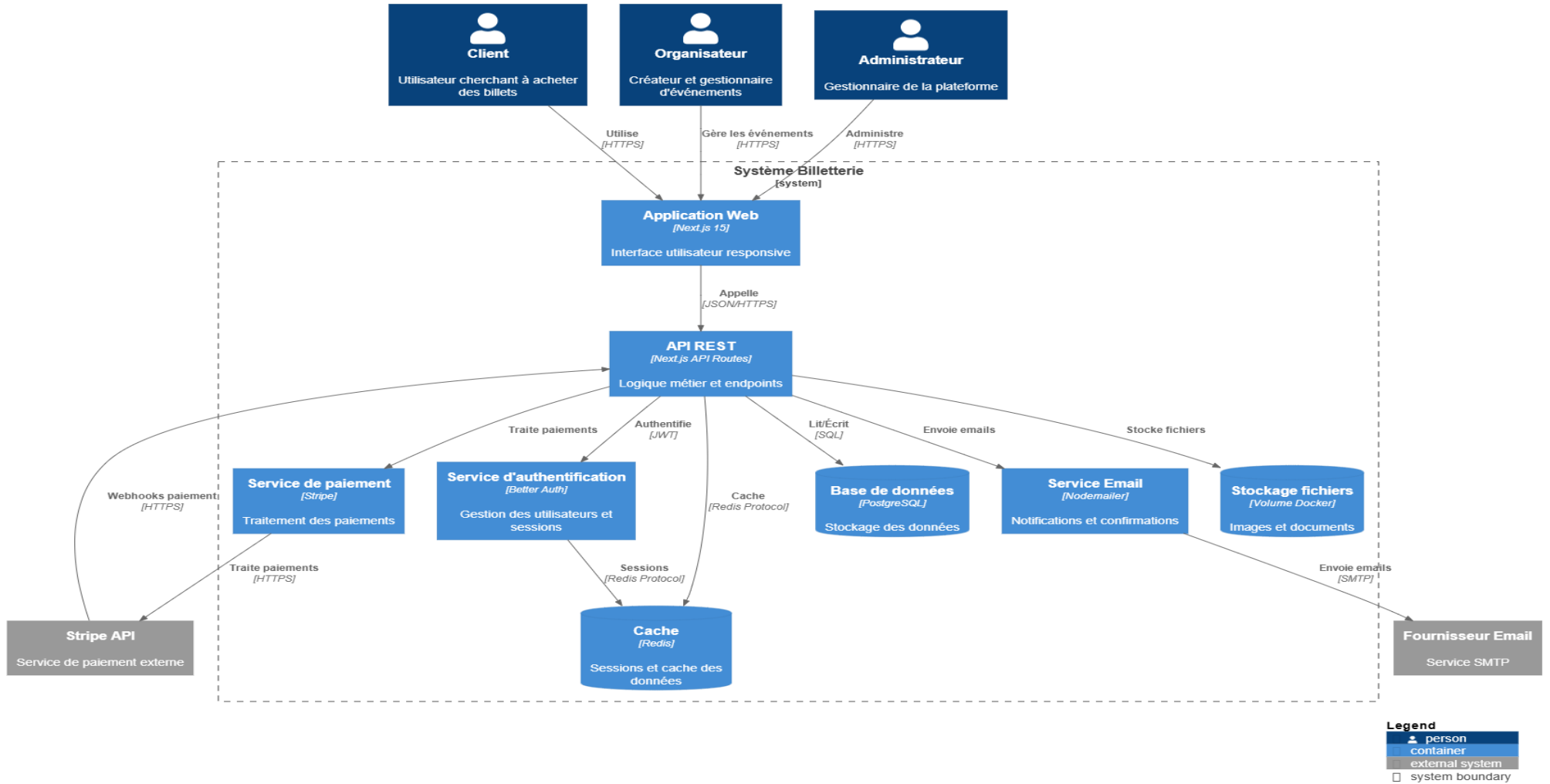
La sécurité et l'authentification sont prises en charge par Better Auth, un service dédié qui gère les utilisateurs, leurs sessions et leurs droits d'accès. Ce service s'appuie sur Redis pour maintenir les sessions actives de manière performante, permettant une expérience utilisateur fluide sans compromettre la sécurité.

Le système de stockage repose sur trois piliers complémentaires. PostgreSQL assure la persistance des données relationnelles avec robustesse, Redis optimise les performances en cachant les données fréquemment consultées et en maintenant les sessions utilisateurs, tandis qu'un volume Docker dédié héberge les fichiers statiques comme les images d'événements et les documents générés.

Les services externes jouent un rôle crucial dans l'écosystème. Stripe gère l'intégralité du cycle de paiement via son API et communique les confirmations de transaction au système par webhooks, garantissant la synchronisation en temps réel des états de commande. Le service email, implémenté avec Nodemailer, se connecte à un fournisseur SMTP pour délivrer les notifications, confirmations de commande et billets électroniques aux utilisateurs.

Cette architecture assure une séparation claire des responsabilités où chaque conteneur remplit une fonction spécifique, facilitant la maintenance, la scalabilité et l'évolution du système. La génération de QR codes est intégrée directement dans l'API comme module interne, évitant la complexité d'un service séparé tout en maintenant la flexibilité nécessaire.

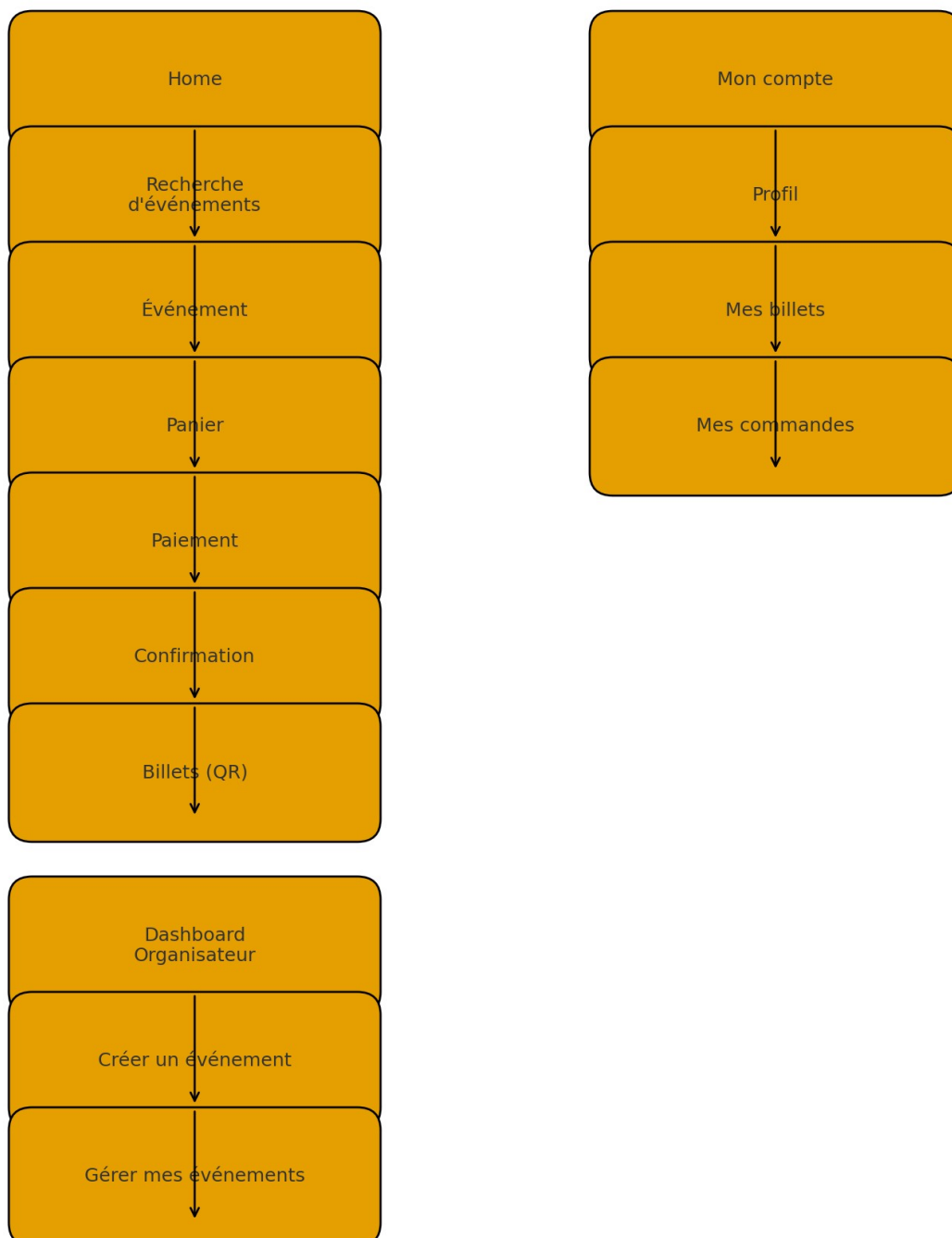
Vue d'ensemble du système - Billetterie



5.3 Maquettes et enchaînement des maquettes

5.3.1 Cartographie

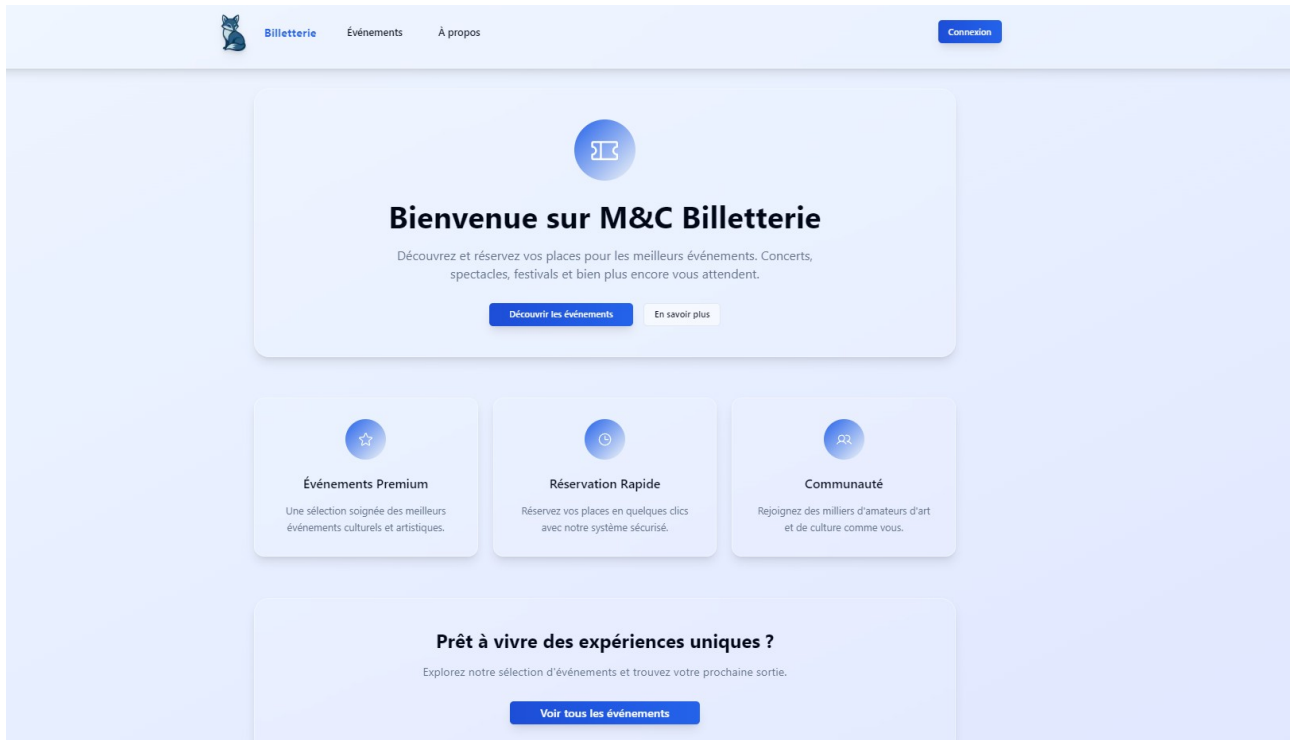
La cartographie fonctionnelle relie les différents rôles (visiteur, utilisateur, organisateur, administrateur) aux écrans et fonctionnalités qui leur sont accessibles.



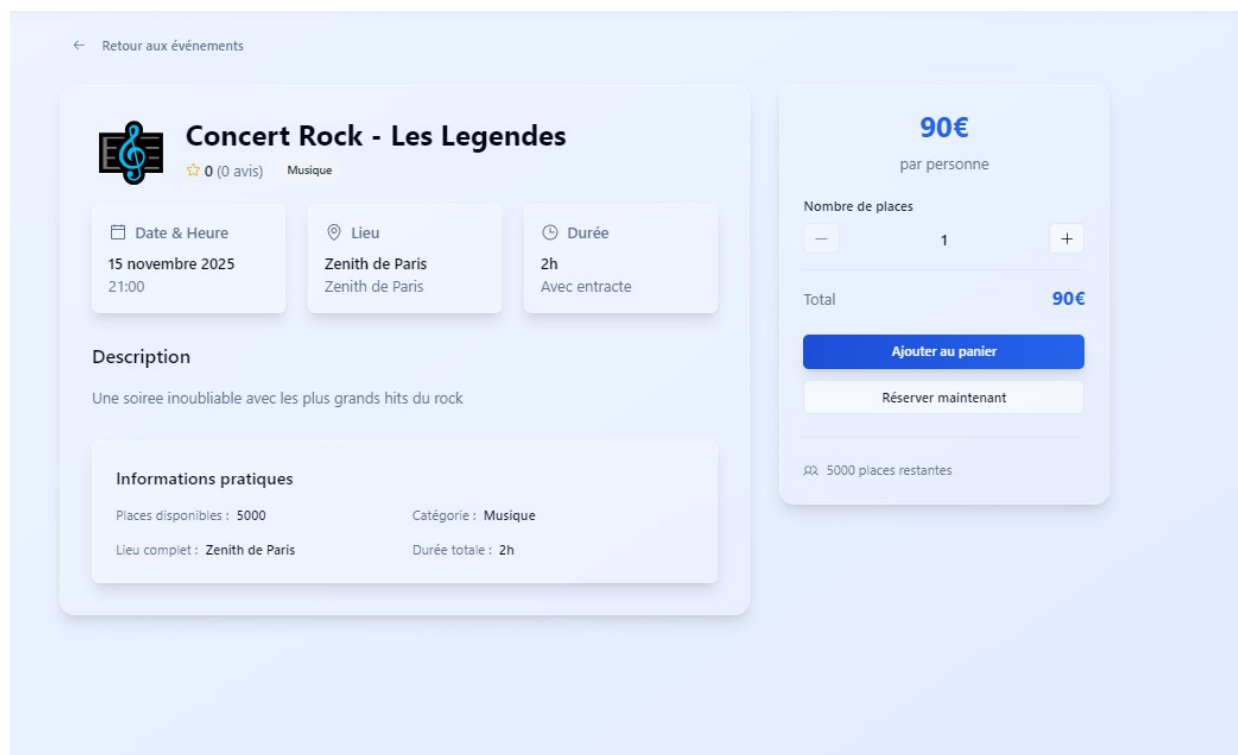
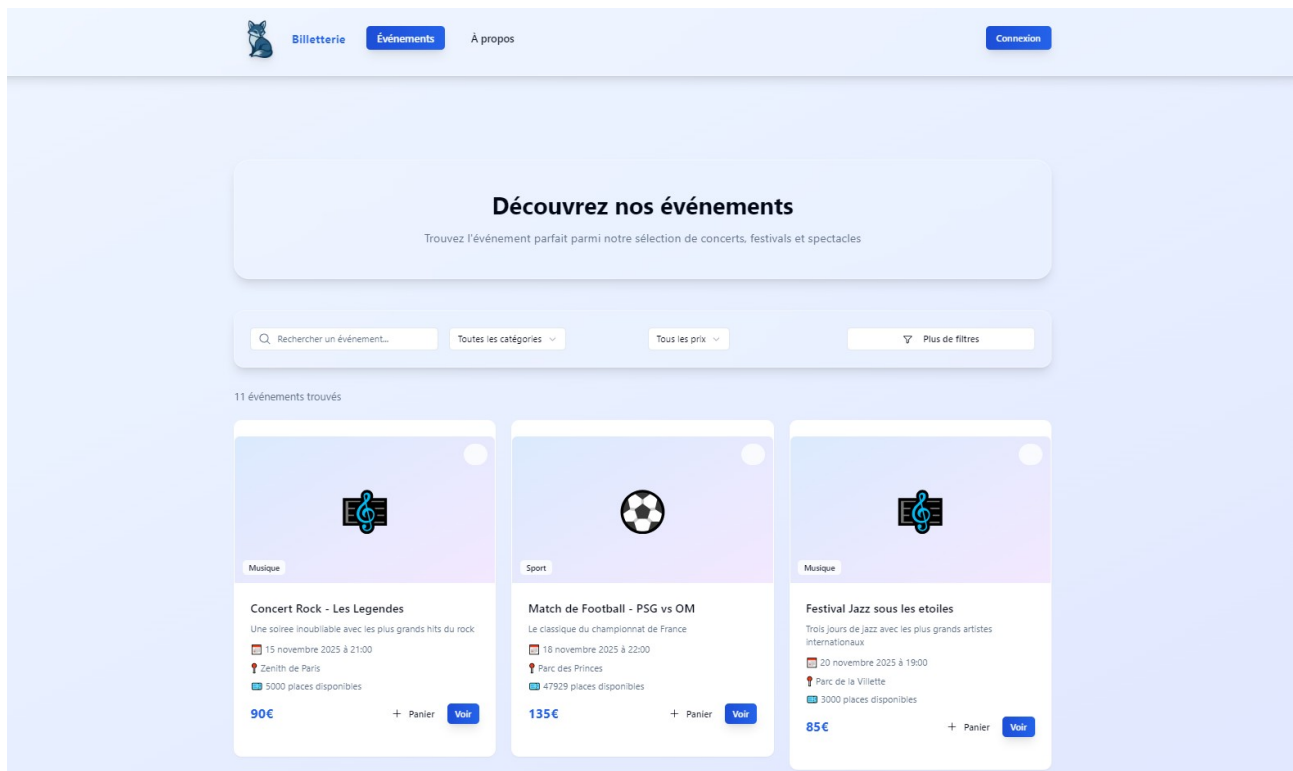
5.3.2 Maquettes

Les maquettes ont été conçues sur Figma selon une logique mobile-first et une hiérarchie visuelle cohérente :

- **Page d'accueil** : recherche d'événements et catégories.



- **Page événement** : détails, tarifs, bouton “Acheter”.

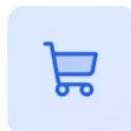


- **Panier** : récapitulatif des billets avant paiement.



Mon Panier

0 articles dans votre panier



Votre panier est vide

Parcourez les événements et ajoutez des billets pour commencer

Découvrir les événements



Mon Panier

2 articles dans votre panier

[Vider le panier](#)

Match de Football - PSG vs OM



135€ par billet

- 1 +

Sous-total

135.00€

Concert Rock - Les Legendes



90€ par billet

- 1 +

Sous-total

90.00€

Récapitulatif

Vérifiez vos articles

Match de Football - PSG vs OM × 1	135.00€
Concert Rock - Les Legendes × 1	90.00€
Sous-total	225.00€
Frais de service	11.25€
TVA (20%)	45.00€

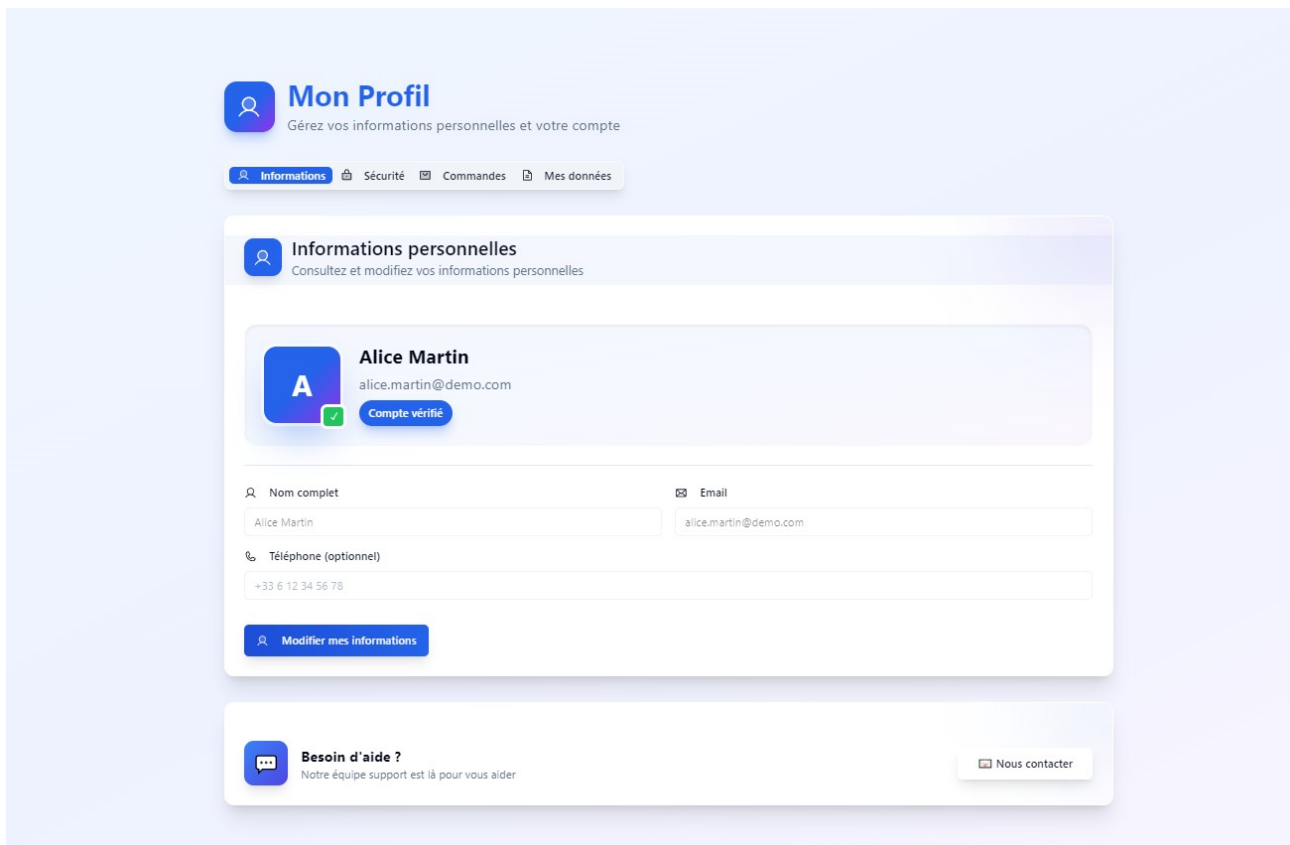
Total 281.25€

[Passer la commande](#)

Paiement sécurisé par Stripe

- ✓ Annulation gratuite jusqu'à 24h avant l'événement
- ✓ Livraison instantanée des billets par email
- ✓ Support client 24/7

- **Profil utilisateur** : historique des commandes et favoris.



- **Dashboard organisateur** : suivi des ventes, création/modification d'événements.

5.4 Diagrammes du comportement des fonctionnalités (cas d'utilisation)

5.4.1 Modèle Conceptuel de Données (MCD, MERISE)

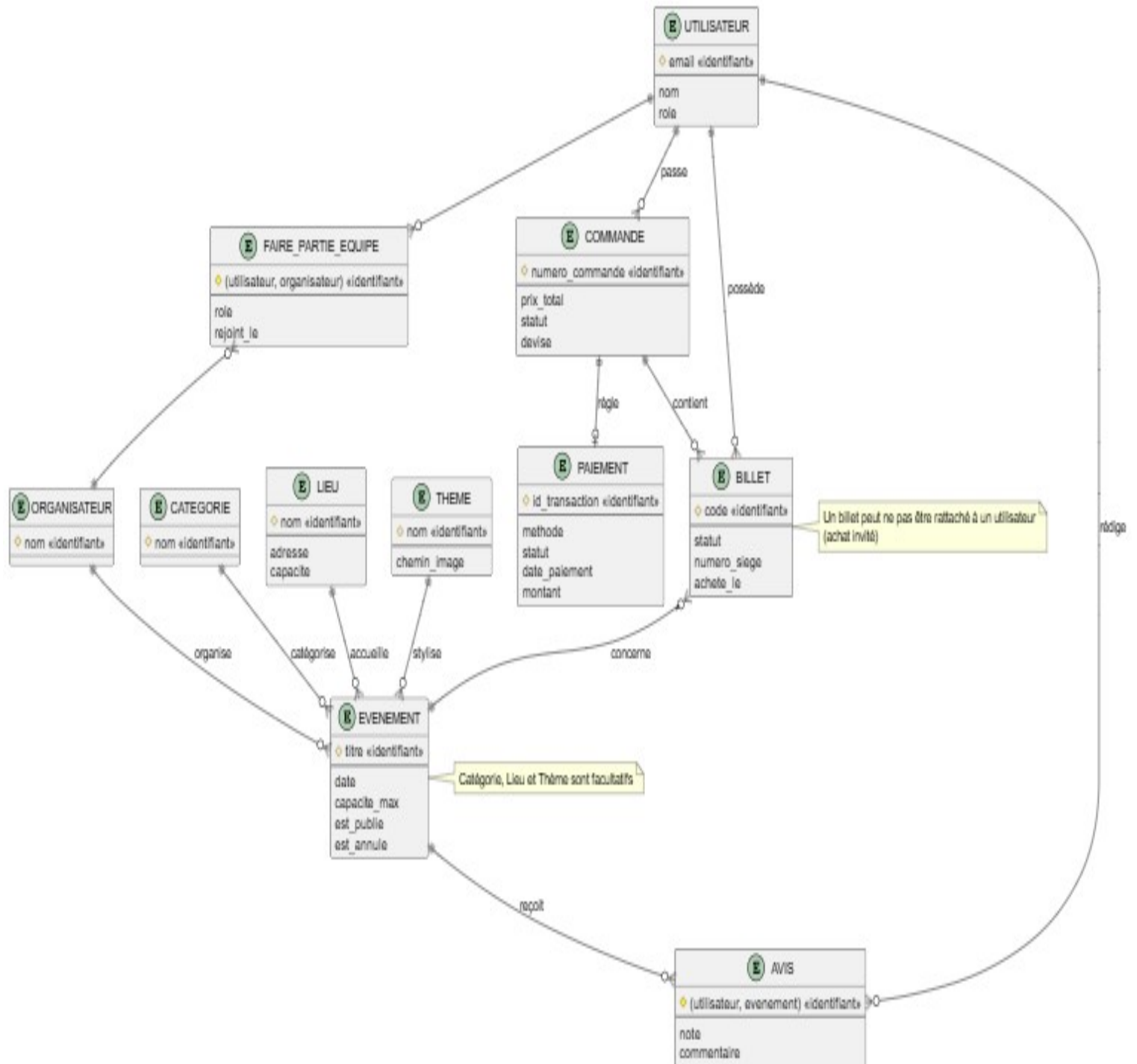
Au cœur du système se trouvent les **UTILISATEUR** et les **EVENEMENT**, qui constituent les deux piliers principaux. Les utilisateurs, identifiés par leur email, nom et rôle, peuvent interagir avec le système de plusieurs manières. Ils passent des commandes pour acquérir des billets, rédigent des avis sur les événements auxquels ils participent, et possèdent directement les billets qu'ils ont achetés. Toutefois, le système prévoit également la possibilité d'achats invités, où un billet peut exister sans être rattaché à un utilisateur spécifique.

Les EVENEMENT sont décrits par leur titre, date et capacité maximale, avec des indicateurs permettant de savoir s'ils sont publiés ou annulés. Chaque événement est organisé par un ORGANISATEUR, entité distincte possédant son propre nom. Cette séparation permet à un même organisateur de gérer plusieurs événements. Pour enrichir la description des événements, trois attributs optionnels sont disponibles : une CATEGORIE pour classifier l'événement, un LIEU définissant l'endroit où se déroule la manifestation avec son adresse et sa capacité, et un THEME qui permet d'appliquer une stylisation visuelle personnalisée.

Le processus d'achat est modélisé à travers les COMMANDE, qui agrègent un ensemble de BILLET pour un montant total dans une devise spécifique. Chaque commande possède un statut permettant de suivre son cycle de vie. Les billets générés contiennent un code unique, un statut, un numéro de siège et une date d'achat, et sont rattachés à un événement particulier. Le règlement financier s'effectue via un PAIEMENT, qui enregistre la méthode utilisée, le statut du paiement, sa date et son montant.

Une relation particulière mérite attention : l'association FAIRE_PARTIE_EQUIPE qui lie les utilisateurs aux organisateurs. Cette relation porteuse d'attributs permet de modéliser l'appartenance d'un utilisateur à l'équipe d'un organisateur, en précisant son rôle et la date à laquelle il a rejoint cette équipe. Cette structure autorise des configurations flexibles où un utilisateur peut collaborer avec plusieurs organisateurs.

Enfin, les AVIS créent un lien de retour entre les utilisateurs et les événements. Un utilisateur rédige des avis comprenant une note et un commentaire, tandis que les événements reçoivent ces évaluations, permettant ainsi de constituer une base de feedback pour améliorer l'expérience utilisateur et la qualité des événements proposés.



5.4.2 Modèle Logique de Données (MLD, MERISE)

Le Modèle Logique de Données transforme la vision conceptuelle en une structure technique prête à être implémentée dans une base de données relationnelle, en ajoutant tous les détails nécessaires à son fonctionnement opérationnel.

Chaque entité est maintenant dotée d'un identifiant unique sous forme d'UUID servant de clé primaire, garantissant l'unicité et la traçabilité de chaque enregistrement. Les dates de création et de modification sont systématiquement ajoutées pour assurer un audit temporel complet des données. L'entité UTILISATEUR s'enrichit considérablement avec un mot de passe hashé, un système de vérification d'email avec horodatage, un suivi de la dernière connexion et de la dernière modification du mot de passe. Un champ métadonnées au format JSON permet de stocker des informations additionnelles sans rigidifier le schéma, offrant ainsi une grande flexibilité évolutive.

Les EVENEMENT intègrent désormais plusieurs clés étrangères pointant vers leurs entités associées : l'organisateur qui le gère, la catégorie optionnelle qui le classe, le lieu qui l'accueille et le thème qui définit son apparence visuelle. Deux nouveaux attributs font leur apparition : l'autorisation d'achat anonyme qui permet de vendre des billets sans compte utilisateur, et l'autorisation de transfert qui détermine si un billet peut être cédé à une autre personne. La description passe d'un simple texte à un champ de type Text permettant des contenus plus étoffés.

La gestion des BILLET devient particulièrement sophistiquée avec l'ajout d'un système de QR code dynamique. Chaque billet possède un QR code actuel qui peut être régénéré selon un intervalle de rotation configurable, renforçant ainsi la sécurité contre la fraude et la duplication. Le système trace précisément quand le QR code a été généré, si le billet a été scanné et utilisé, avec les horodatages correspondants. La relation avec l'utilisateur peut être nulle pour gérer les achats anonymes, tandis que les clés étrangères vers l'événement et la commande restent obligatoires pour maintenir l'intégrité référentielle.

Les COMMANDE intègrent un système de promotion avec un code promo et un montant de remise associé, permettant de gérer des stratégies commerciales variées. Le champ métadonnées peut contenir des informations complémentaires comme l'adresse IP de l'acheteur ou des détails sur le parcours d'achat. La devise est explicitement stockée pour gérer des transactions internationales.

L'entité PAIEMENT établit une relation un-à-un avec la commande via une clé étrangère unique, garantissant qu'une commande ne peut avoir qu'un seul paiement associé. Elle conserve l'identifiant de transaction fourni par le processeur de paiement, la méthode utilisée et le statut du paiement, créant ainsi une piste d'audit complète. Une commande peut exister sans paiement si elle est en statut brouillon, permettant de gérer le panier avant validation.

La relation MEMBRE_EQUIPE matérialise l'association porteuse d'attributs du MCD en une table de liaison complète avec son propre identifiant. Elle relie utilisateurs et organisateurs tout en capturant le rôle spécifique de chaque membre et sa date d'entrée dans l'équipe. Cette structure permet des requêtes efficaces pour retrouver tous les membres d'un organisateur ou tous les organisateurs auxquels un utilisateur contribue.

Les entités CATEGORIE, LIEU et THEME deviennent des tables de référence complètes. Le lieu détaille maintenant précisément son adresse et sa capacité physique. Le thème s'enrichit d'une couleur principale et de métadonnées JSON pour stocker des paramètres de stylisation supplémentaires. La catégorie maintient une contrainte d'unicité sur son nom pour éviter les doublons.

L'entité AVIS impose une contrainte d'unicité composite sur le couple utilisateur-événement, empêchant ainsi un utilisateur de laisser plusieurs avis pour le même événement. La note est typée comme entier, suggérant probablement un système d'évaluation sur une échelle fixe.

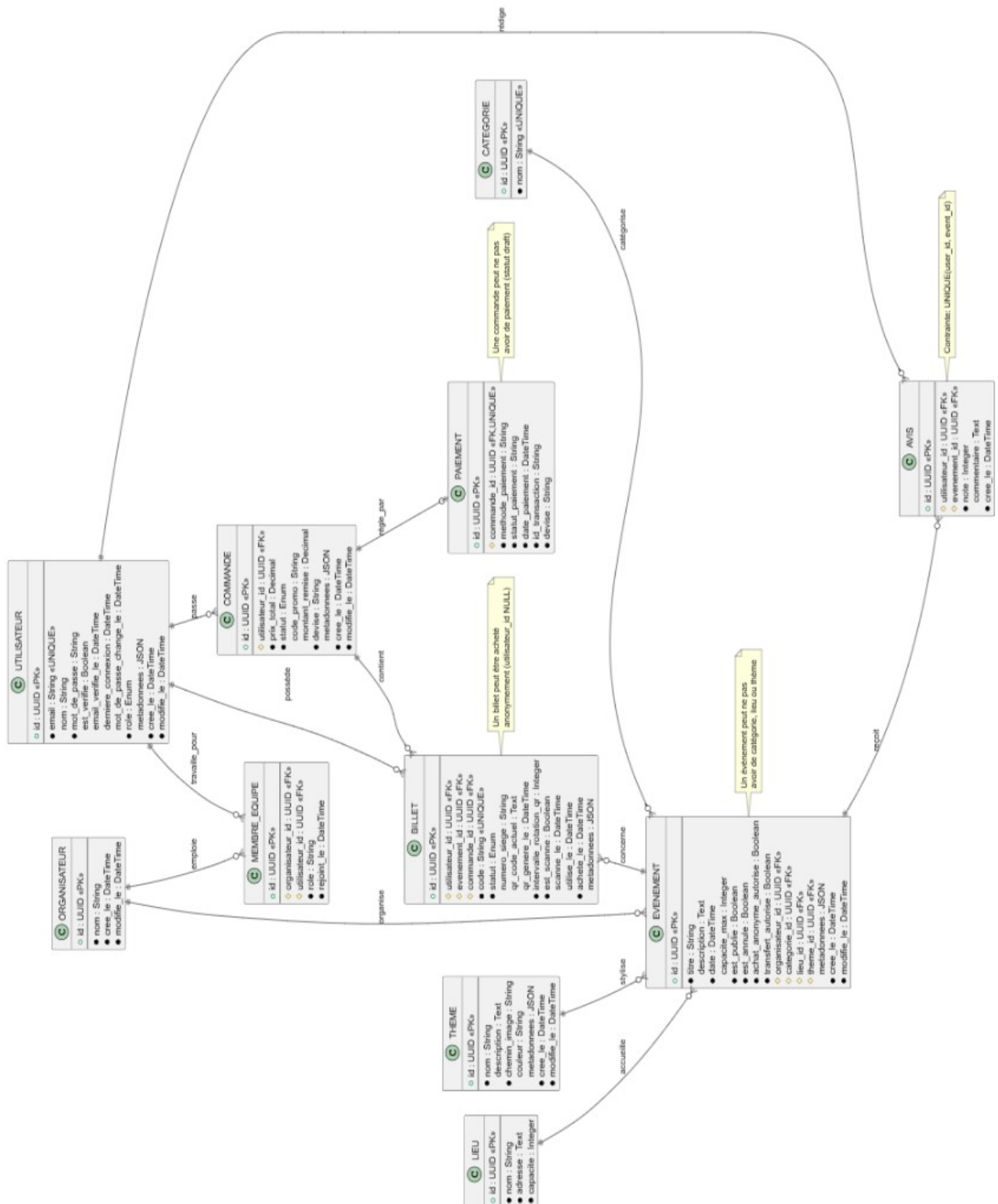
Ce modèle logique traduit donc la conception abstraite en un schéma de base de données robuste, avec des contraintes d'intégrité, des types de données précis, et des mécanismes de sécurité et d'audit intégrés, tout en maintenant la flexibilité nécessaire pour l'évolution future du système grâce aux champs métadonnées JSON.

Rôle Visiteur :

Un visiteur non connecté peut explorer librement le catalogue d'événements disponibles sur la plateforme. Il consulte les événements, effectue des recherches et affine ses résultats en filtrant par catégorie, date ou lieu. Pour chaque événement, il accède aux détails complets incluant les tarifs et la disponibilité des billets.

Le visiteur navigue également dans les différentes catégories d'événements proposées et accède aux informations légales et pratiques du site : FAQ, conditions générales de vente et politique de confidentialité. En cas de besoin, il peut contacter le support client.

Pour aller plus loin et effectuer des achats, le visiteur doit créer un compte en s'inscrivant. Il peut également se connecter s'il possède déjà un compte, ou utiliser la fonction de récupération de mot de passe en cas d'oubli.



5.4.3 Modèle Physique de Données (MPD, MERISE)

Le Modèle Physique de Données représente l'implémentation concrète dans PostgreSQL, spécifiant tous les détails techniques nécessaires au déploiement effectif de la base de données, incluant les types de données précis, les contraintes, les index de performance et les mécanismes de sécurité.

La table users adopte des types PostgreSQL spécifiques avec des VARCHAR limitées à 255 caractères pour l'email et le nom, garantissant une taille raisonnable tout en conservant la flexibilité. Le mot de passe est stocké dans un VARCHAR(255) dimensionné pour accueillir les hash bcrypt ou argon2. Les champs temporels utilisent systématiquement TIMESTAMPTZ pour gérer les fuseaux horaires, évitant ainsi les problèmes de décalage horaire dans un contexte international. Un type ENUM personnalisé nommé ROLE définit les rôles possibles avec une valeur par défaut 'USER'. Plusieurs index sont créés sur l'email et le rôle pour accélérer les requêtes de connexion et de filtrage par profil. Un trigger optionnel peut automatiser la mise à jour du champ updated_at à chaque modification d'enregistrement.

Les events implémentent une indexation composite sophistiquée combinant date et statut de publication, optimisant les requêtes fréquentes qui cherchent les événements publiés dans une période donnée. Le champ location reste en VARCHAR(255) malgré l'existence de la relation avec venues, probablement pour gérer des cas où l'événement n'a pas de lieu fixe ou pour des informations complémentaires comme une salle spécifique. Des contraintes de vérification peuvent être ajoutées via SQL pour s'assurer que la date est future et qu'un événement annulé ne peut être publié. Le champ metadata en JSONB permet des requêtes performantes sur le contenu JSON contrairement au type JSON standard.

La table orders définit le prix avec DECIMAL(10,2) offrant une précision exacte essentielle pour les transactions financières, évitant les erreurs d'arrondi des nombres flottants. Une contrainte vérifie que le prix total est positif ou nul, et une autre peut contrôler que le montant de remise ne dépasse pas le prix total. Le type ENUM ORDER_STATUS encode les différents états possibles : draft pour un panier non validé, pending_payment en attente de règlement, paid pour les commandes payées, et cancelled pour les annulations. Un index composite sur user_id et status optimise les requêtes listant les commandes d'un utilisateur par statut.

Les tickets présentent une gestion avancée avec plusieurs horodatages traçant le cycle de vie complet : `purchased_at` pour l'achat, `qr_code_generated_at` pour la génération du QR code, `scanned_at` pour le scan à l'entrée, et `used_at` pour la validation finale. L'intervalle de rotation du QR code est stocké en entier représentant probablement des heures, avec une valeur par défaut de 12 heures. Un index partiel sur les tickets actifs permet d'optimiser les requêtes ne ciblant que les billets valides en excluant ceux utilisés ou annulés. Un trigger peut générer automatiquement le code unique du billet lors de l'insertion.

La table `payments` établit une contrainte `UNIQUE` sur `order_id` matérialisant physiquement la relation un-à-un avec les commandes. Cette contrainte empêche au niveau base de données qu'une commande puisse avoir plusieurs paiements, garantissant l'intégrité même en cas d'erreur applicative. L'identifiant de transaction conserve la référence fournie par le processeur de paiement externe comme Stripe ou PayPal.

Les venues imposent une contrainte de capacité strictement positive, empêchant la création de lieux avec une capacité nulle ou négative. Cette validation au niveau base de données constitue un dernier rempart contre les données incohérentes.

Les themes incluent une validation potentielle du format couleur via une expression régulière PostgreSQL vérifiant que la valeur correspond au format hexadécimal standard `#RRGGBB`. Le chemin d'image utilise un `VARCHAR(500)` permettant de stocker des URL complètes ou des chemins relatifs assez longs.

La table `reviews` implémente une contrainte `UNIQUE` composite sur le couple `user_id` et `event_id`, traduisant physiquement la règle métier qui empêche un utilisateur de laisser plusieurs avis sur le même événement. Une contrainte peut valider que la note est comprise entre 1 et 5, cohérente avec un système d'évaluation par étoiles standard.

La table `team_members` utilise également une contrainte `UNIQUE` composite garantissant qu'un utilisateur ne peut apparaître qu'une seule fois dans l'équipe d'un organisateur donné, bien qu'il puisse faire partie de plusieurs équipes différentes.

Le modèle intègre un système de sécurité et d'audit exhaustif avec `user_sessions` qui trace chaque session utilisateur avec son token, adresse IP, user agent et informations sur l'appareil. Le suivi de `last_activity_at` permet d'implémenter des mécanismes de timeout, tandis que `expires_at` gère l'expiration automatique des sessions. Une session peut être détruite avec une raison enregistrée dans `destroy_reason`.

Les `activity_logs` et `event_logs` fournissent deux niveaux de traçabilité différents, le premier pour les actions utilisateur sur des ressources spécifiques identifiées par leur type et ID, le second pour des événements système plus généraux avec un contexte textuel.

La table `login_attempts` capture toutes les tentatives de connexion réussies ou échouées avec leur adresse IP et user agent, permettant de détecter des patterns d'attaque par force brute. Un champ `risk_score` peut stocker une évaluation du risque calculée par l'application, tandis que `session_id` lie éventuellement la tentative à une session créée.

Les `audit_logs` constituent un journal d'audit complet avec un hash d'événement garantissant l'intégrité et la non-répudiation des actions. Le niveau de risque et le marqueur de sensibilité permettent de classifier les événements pour une analyse ultérieure ou des obligations de conformité réglementaire.

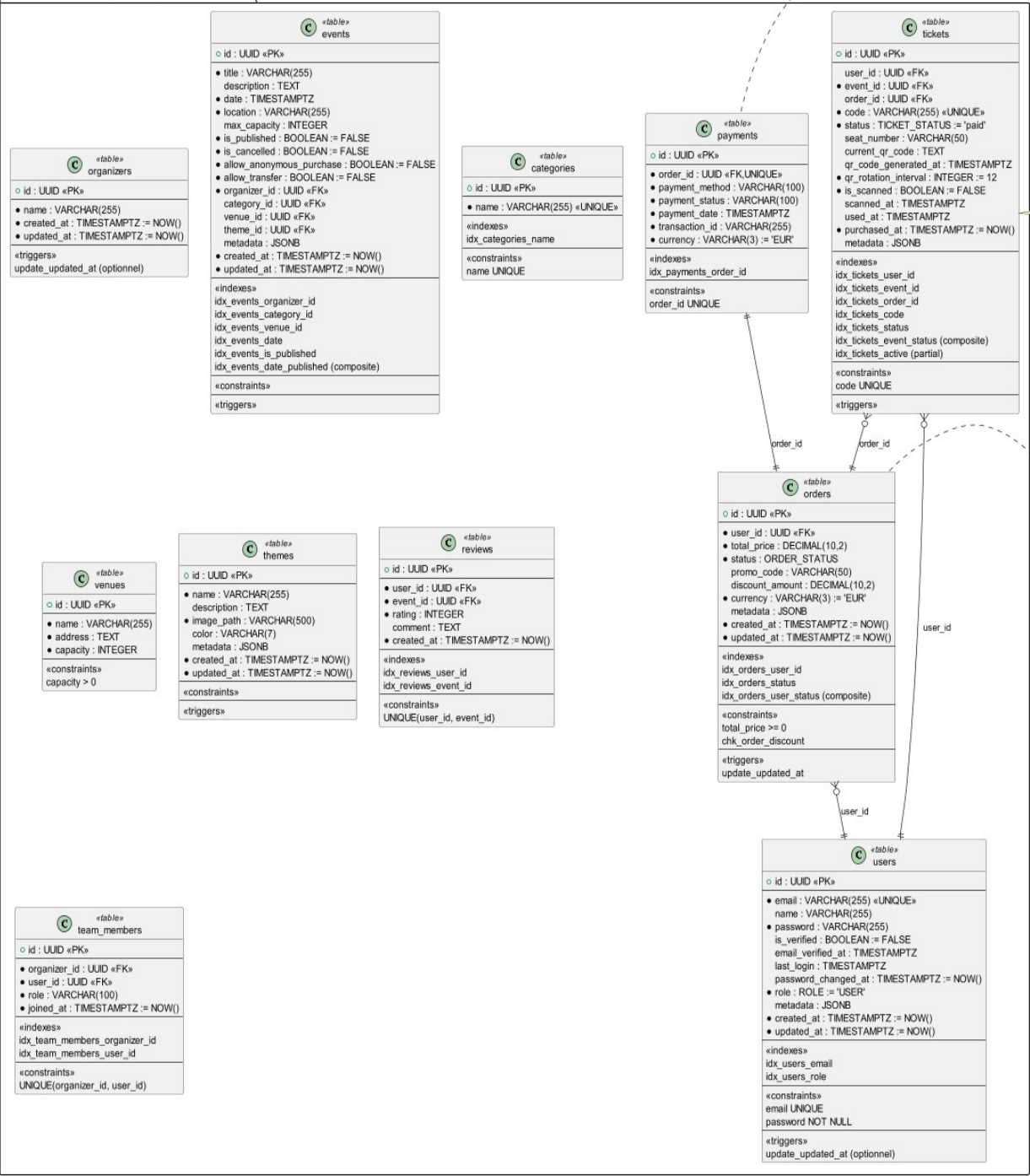
La table `blocked_ip` gère le blocage d'adresses IP suspectes avec possibilité d'expiration automatique, un compteur de blocages successifs, et la traçabilité de qui a débloqué l'IP et quand.

Les `system_backups` enregistrent toutes les sauvegardes système avec leur type, statut, chemin de fichier chiffré, clé de chiffrement, taille et checksum SHA-256 pour vérifier l'intégrité. La durée de rétention configurable permet une gestion automatisée du cycle de vie des sauvegardes.

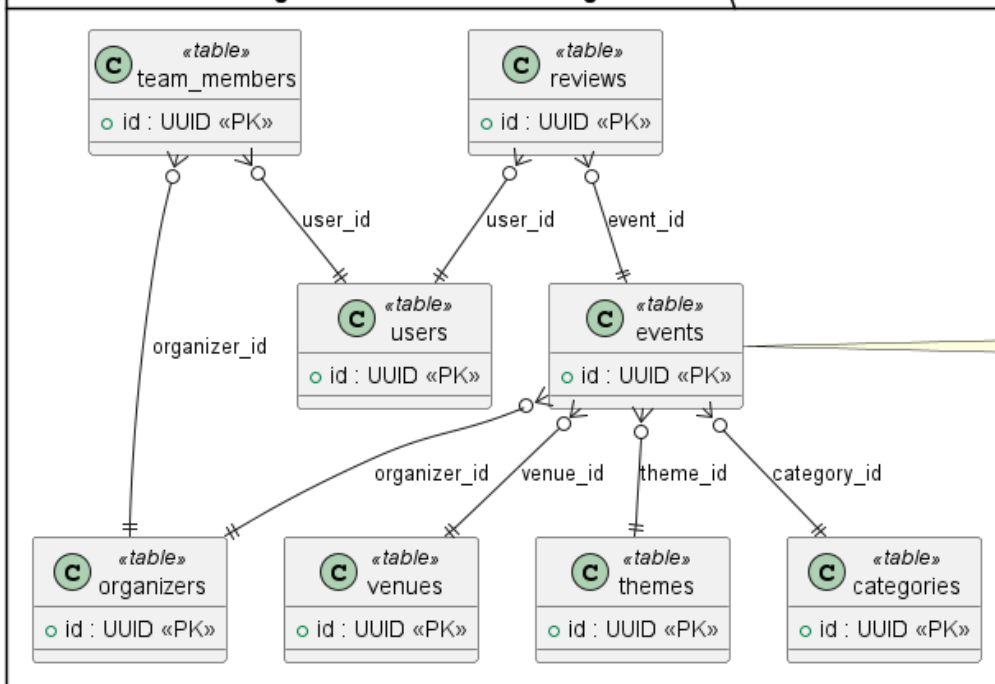
La `password_history` conserve l'historique des mots de passe hashés d'un utilisateur, permettant d'implémenter une politique empêchant la réutilisation de mots de passe récents, renforçant ainsi la sécurité.

Ce modèle physique traduit donc non seulement la structure logique en instructions SQL concrètes avec types de données PostgreSQL spécifiques, mais y intègre également une couche complète de sécurité, d'audit et de performance via indexes stratégiques, contraintes d'intégrité robustes et tables de traçabilité exhaustives, créant ainsi un système prêt pour un environnement de production exigeant.

Base de Données PostgreSQL - Système de Billetterie



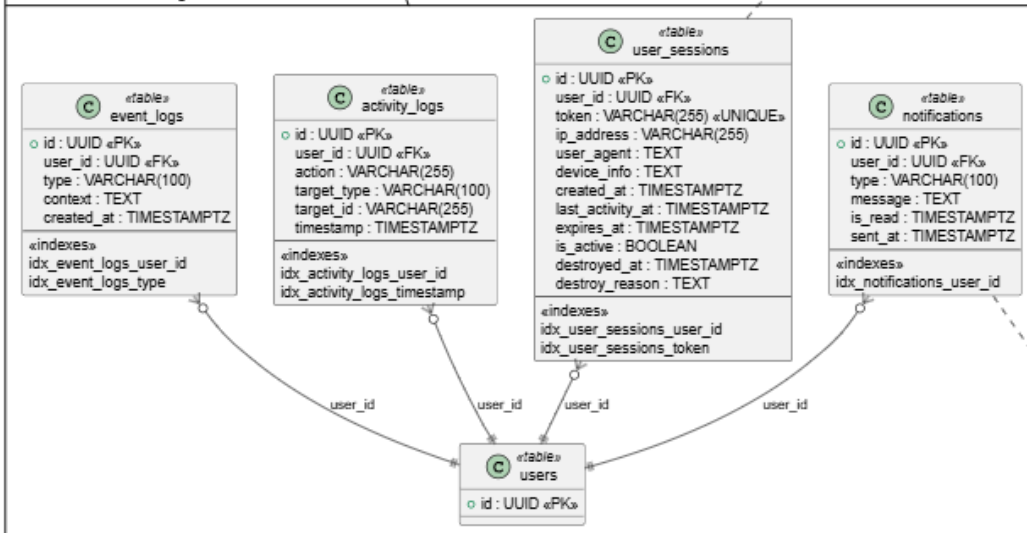
Base de Données PostgreSQL - Événements & Organisateurs



Contraintes métier :

- Date future
- Pas publié si annulé
- Capacité > 0

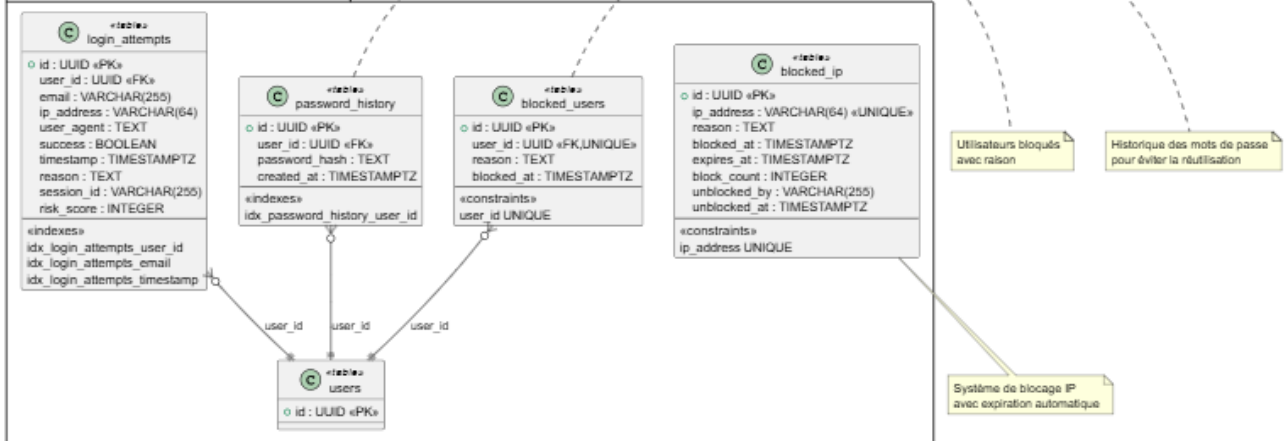
Base de Données PostgreSQL - Sessions & Activités



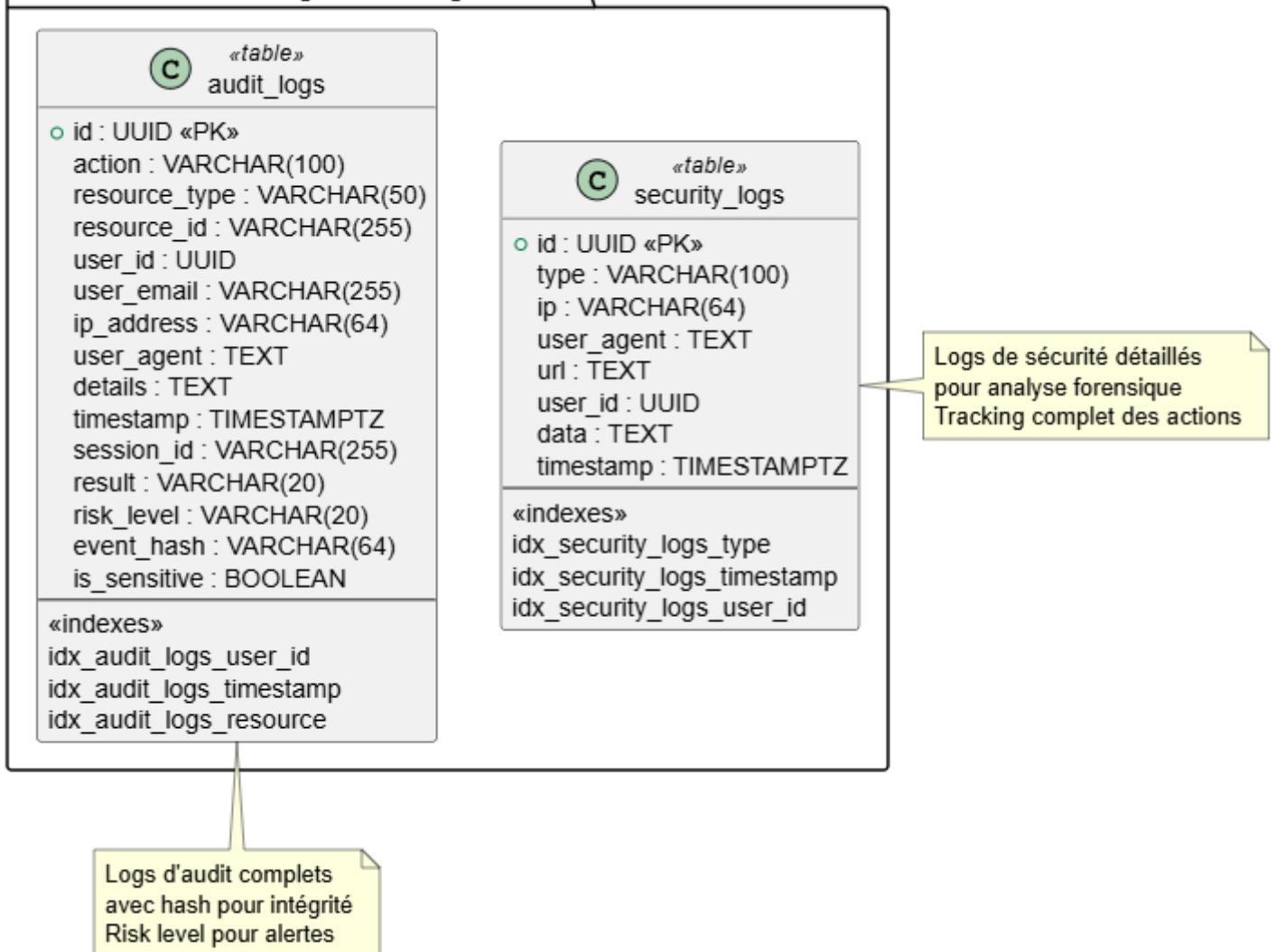
Gestion des sessions utilisateur
avec tracking IP et device
Expiration automatique

Système de notifications
avec suivi de lecture

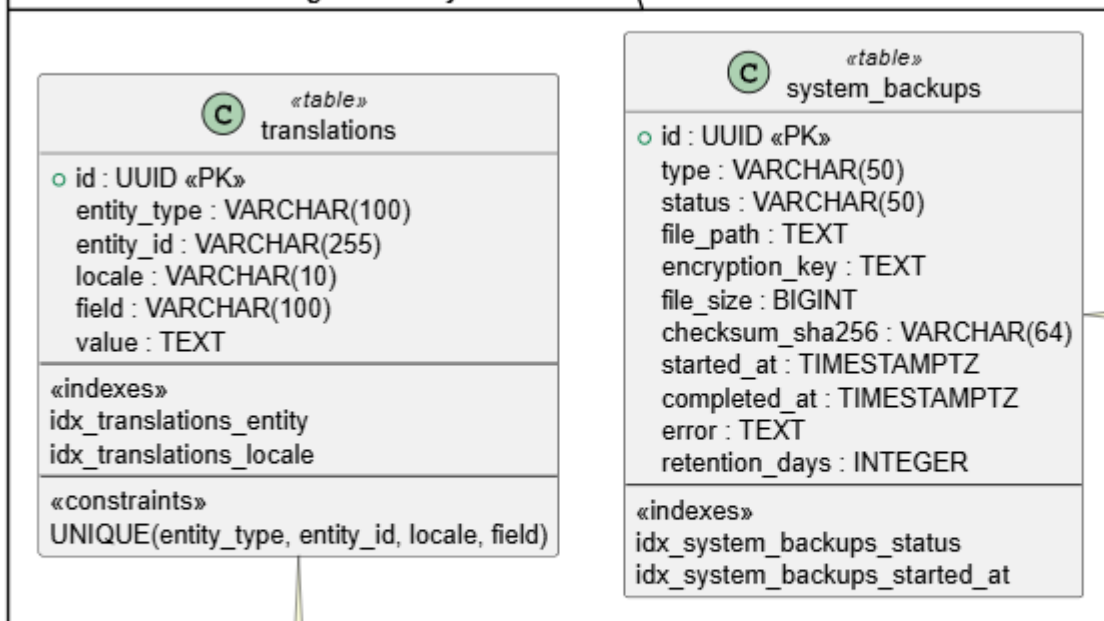
Base de Données PostgreSQL - Sécurité Utilisateurs



Base de Données PostgreSQL - Logs & Audit



Base de Données PostgreSQL - Système & i18n



Gestion des sauvegardes avec chiffrement et checksum
Rétention configurable

Système i18n
Traductions par entité et par locale

5.4.4 Diagramme de classes (UML)

Le système de billetterie est structuré autour d'une architecture modulaire en sept blocs métier interconnectés, avec le Core Business comme élément central du dispositif.

Le bloc Core Business constitue le noyau du système et maintient des relations bidirectionnelles avec l'ensemble des autres composants. Il communique directement avec l'Event Management pour la gestion des événements, s'appuie sur l'Organization pour structurer l'appartenance des entités, collabore avec le Payment System pour traiter les transactions financières, est sécurisé par le module Security & Audit, et utilise les System Services pour la journalisation et les notifications.

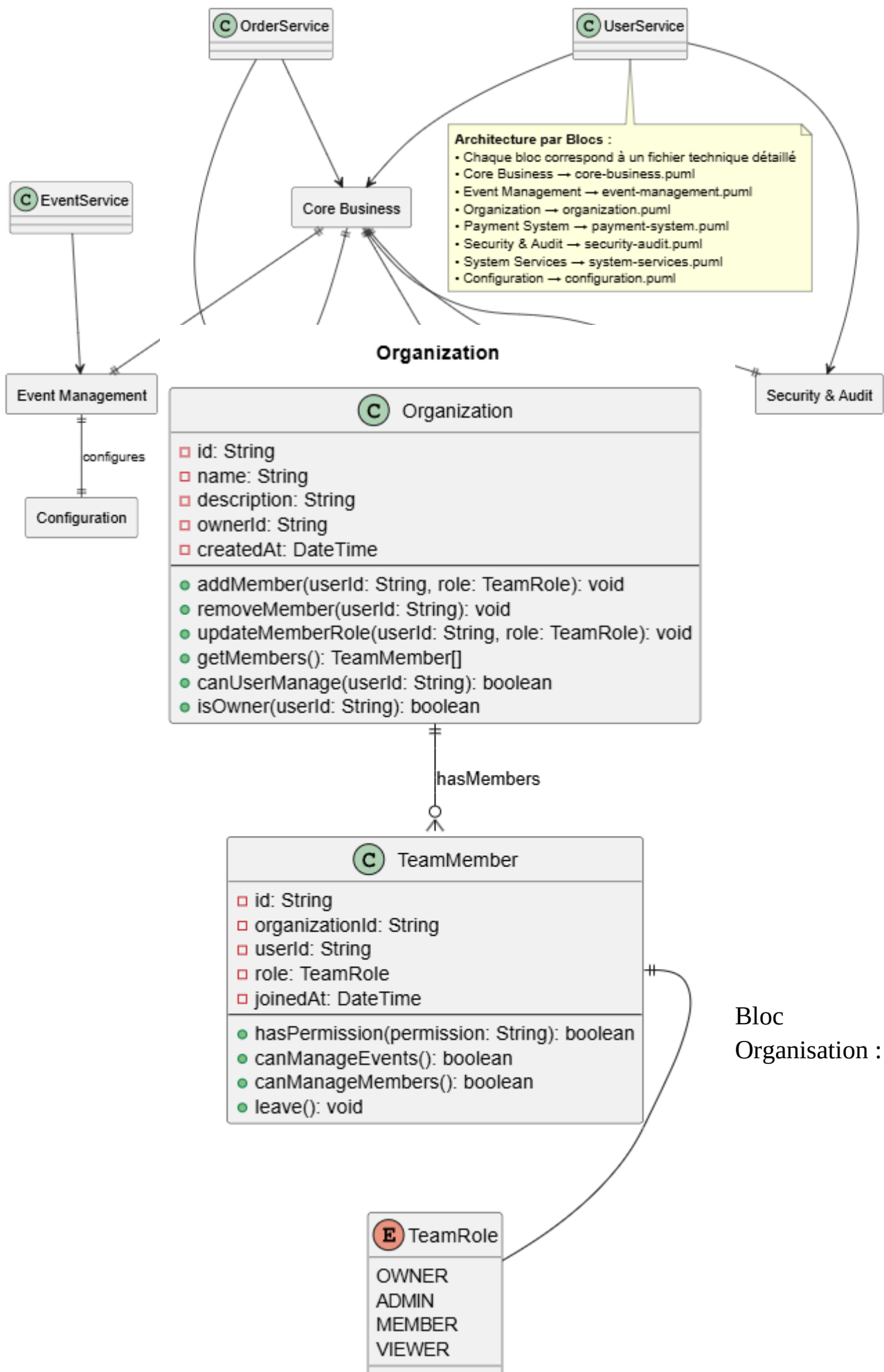
La configuration du système repose sur un lien entre l'Event Management et le bloc Configuration, permettant de paramétrer finement les événements proposés. Cette séparation assure une flexibilité dans l'adaptation du système aux besoins évolutifs.

L'orchestration des opérations métier est assurée par trois services clés qui agissent comme des chefs d'orchestre. L'EventService coordonne tout ce qui concerne la gestion événementielle, l'OrderService fait le pont entre le cœur métier et le système de paiement pour gérer l'intégralité du cycle de commande, tandis que l'UserService harmonise les interactions entre les utilisateurs, le Core Business et les mécanismes de sécurité et d'audit.

Cette conception favorise une séparation claire des responsabilités où chaque bloc technique correspond à un domaine fonctionnel spécifique, rendant le système à la fois maintenable et évolutif. Les fichiers de diagrammes détaillés permettent de plonger dans la structure interne de chaque module sans compromettre la lisibilité de la vue d'ensemble.

Diagramme global avec tous les blocs :

Vue Globale - Architecture Classes du Système de Billetterie



Bloc
Organisation :

Diagramme d'entité-relation :

Vue d'Ensemble

Le système de billetterie est conçu autour d'une architecture modulaire où chaque module assume des responsabilités spécifiques tout en communiquant efficacement avec les autres composants. Au cœur de cette architecture se trouve le concept d'utilisateur multi-rôles, qui permet à une même entité User de porter différentes casquettes selon les besoins : simple utilisateur, organisateur d'événements, ou administrateur système.

Le Module Core : Le Pilier Central

Le module Core constitue la fondation du système en gérant les entités fondamentales que sont les utilisateurs (User), les commandes (Order) et les billets (Ticket). C'est ici que s'établit la gestion de base des utilisateurs, incluant leurs profils et leurs rôles. Ce module agit comme un hub central puisque presque tous les autres modules interagissent avec lui d'une manière ou d'une autre, que ce soit pour créer des commandes, attribuer des rôles ou gérer l'authentification des utilisateurs.

Le Module Events : La Vitrine du Système

Le module Events prend en charge tout ce qui concerne la gestion des événements, depuis leur création jusqu'à leur publication. Il englobe les événements eux-mêmes (Event), leurs catégories (Category) et les lieux où ils se déroulent (Venue). Ce module travaille en étroite collaboration avec le module Core pour générer les tickets associés aux événements, et s'appuie sur le module Configuration pour personnaliser l'apparence et recueillir les avis des participants.

Le Module Organizer : La Dimension Professionnelle

Le module Organizer ajoute une couche d'organisation professionnelle au système en permettant la création de structures organisationnelles (Organization) et la gestion d'équipes (TeamMember). Ce qui est particulièrement intéressant ici, c'est que les membres d'équipe ne sont autres que des utilisateurs du système ayant endossé le rôle d'organisateur. Ces organisations sont ensuite responsables de la création et de la gestion des événements qu'elles proposent au public.

Le Module Payment : Le Cœur Financier

Le module Payment gère l'intégralité des transactions financières du système, du traitement des paiements à l'émission des factures. Il est directement lié au module Core puisque chaque paiement correspond à une commande passée par un utilisateur. Ce module assure la traçabilité complète des transactions et garantit l'intégrité financière de la plateforme.

Le Module Configuration : La Personnalisation

Le module Configuration offre les outils nécessaires pour personnaliser l'expérience utilisateur à travers les thèmes visuels (Theme), la gestion des avis et commentaires (Reviews), ainsi que l'internationalisation via les traductions. Il intervient principalement au niveau des événements pour permettre aux organisateurs de personnaliser la présentation de leurs manifestations et aux utilisateurs de partager leurs retours d'expérience.

Le Module Security : Le Gardien du Système

Le module Security assure la protection de l'ensemble de la plateforme en gérant les sessions utilisateurs (UserSession), en traçant toutes les actions sensibles (AuditLog), en surveillant les tentatives de connexion suspectes (LoginAttempt) et en bloquant les adresses IP malveillantes (BlockedIP). Ce module travaille main dans la main avec le module Core pour sécuriser les sessions des utilisateurs et alimente le module System en informations de sécurité pour une surveillance globale.

Le Module System : L'Orchestrateur Technique

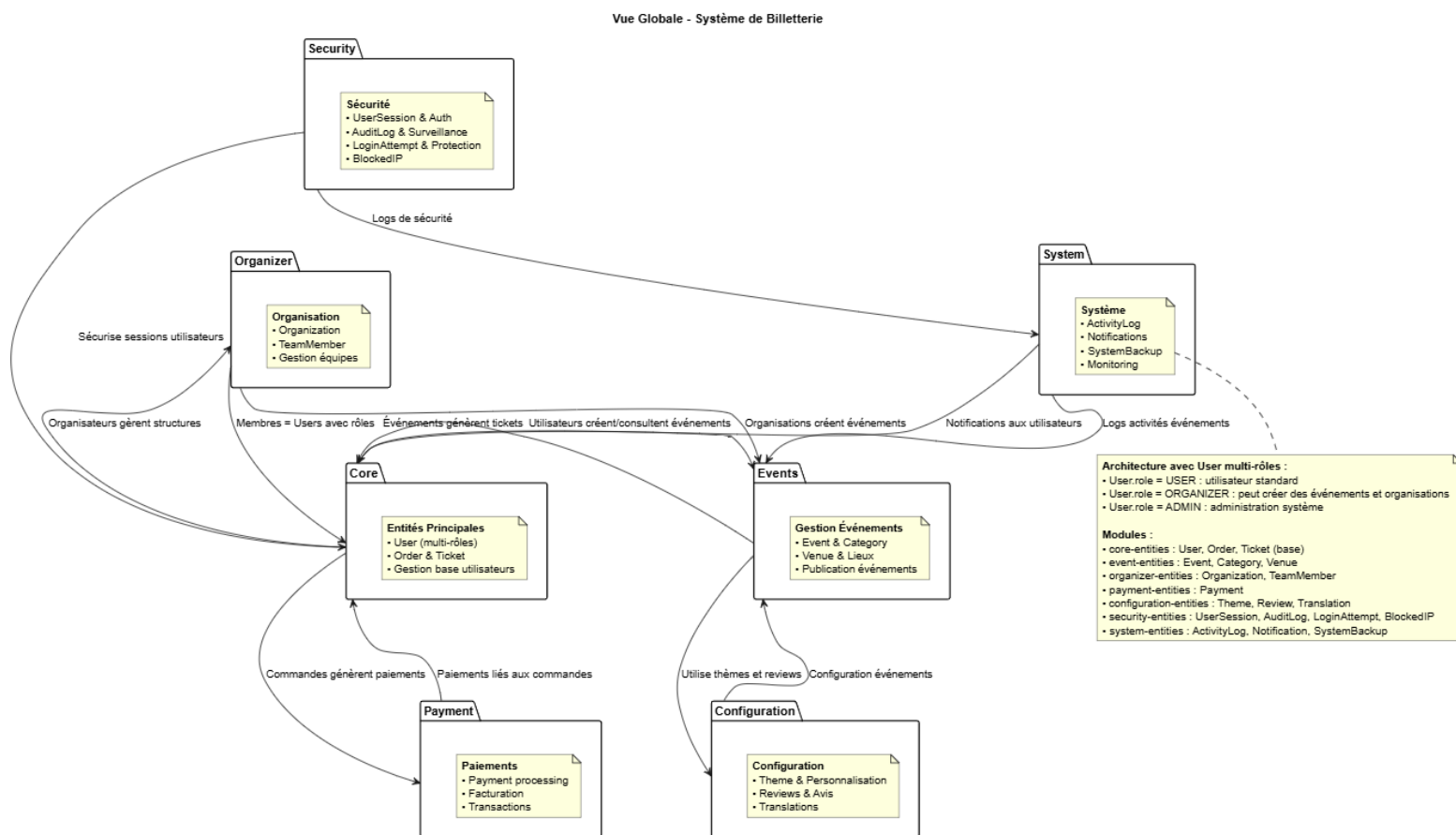
Le module System prend en charge les aspects techniques et opérationnels de la plateforme, notamment la journalisation des activités (ActivityLog), l'envoi de notifications aux utilisateurs, la gestion des sauvegardes système (SystemBackup) et le monitoring général. Il collecte les informations en provenance de tous les autres modules, en particulier celles relatives à la sécurité, et assure que les utilisateurs restent informés des événements qui les concernent.

Les Interactions Inter-Modules

L'architecture se distingue par la richesse de ses interactions. Les utilisateurs du module Core créent et consultent les événements du module Events, génèrent des commandes qui déclenchent des paiements dans le module Payment, et peuvent devenir organisateurs en créant des structures dans le module Organizer. Les

organisations ainsi créées prennent ensuite la main sur la création d'événements, qui eux-mêmes génèrent des tickets pour les utilisateurs. Le module Security veille sur toutes ces opérations en sécurisant les sessions et en journalisant les actions critiques, pendant que le module System orchestre les notifications et maintient l'ensemble du système opérationnel.

Cette architecture modulaire permet une séparation claire des responsabilités tout en maintenant une cohérence globale, facilitant ainsi la maintenance, l'évolution et la scalabilité de la plateforme de billetterie.



5.5 Script de création et/ou de modification de la base de données

5.5.1 Script de création

Les scripts de création sont générés automatiquement par Prisma à partir du fichier schema.prisma.

5.5.2 Argumentation

L'utilisation de Prisma ORM assure :

- la conformité au modèle de données défini (entités, relations, contraintes) ;
- la sécurité contre les injections SQL ;
- et une maintenance facilitée grâce aux migrations versionnées.

5.5.3 Script de modification

Les évolutions du modèle se font via des migrations Prisma :

```
npx prisma migrate dev --name add_ticket_status
```

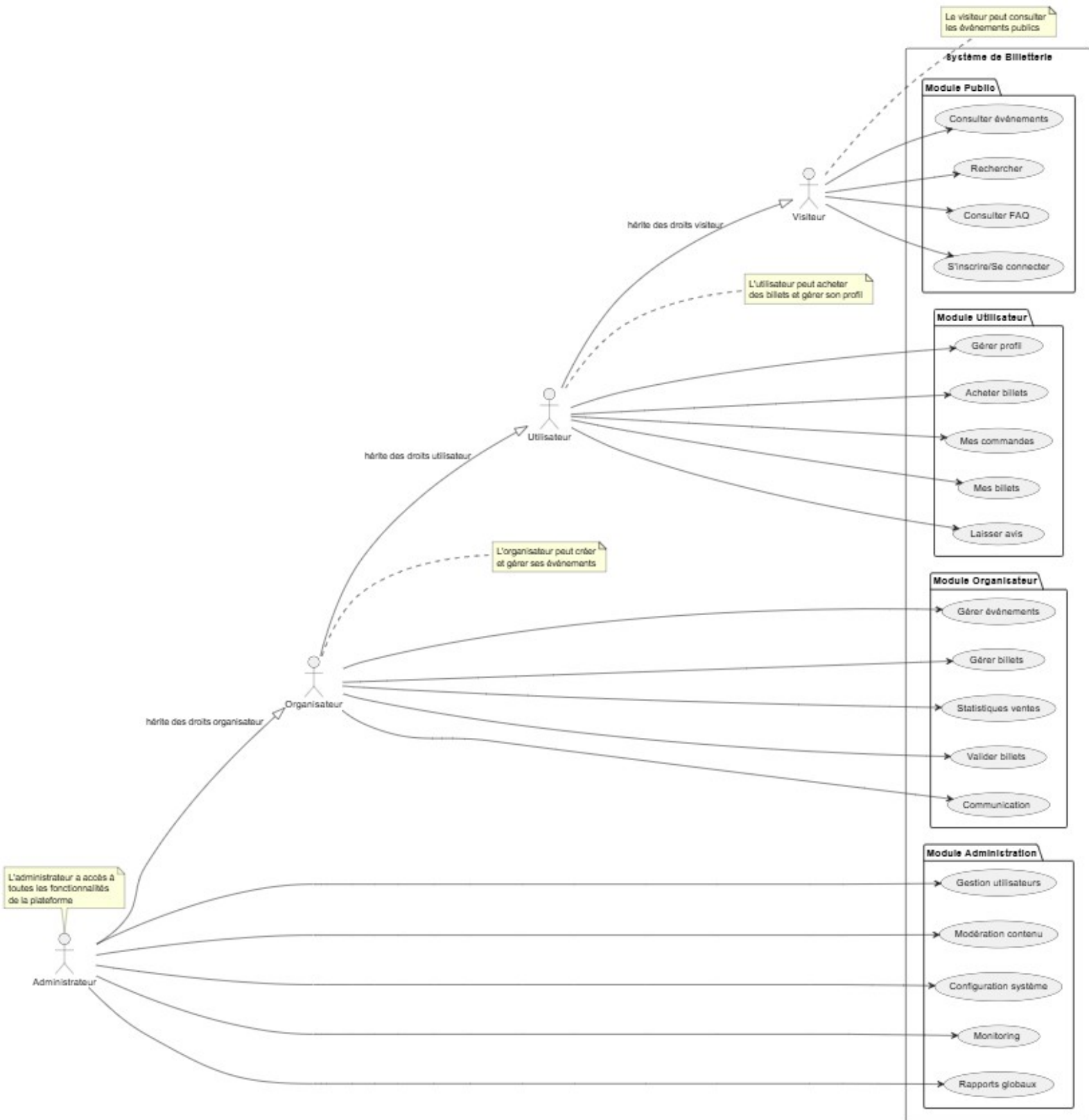
5.5.4 Argumentation

Chaque migration est traçable, versionnée et réversible, garantissant la cohérence entre le code et la base.

5.6. Diagramme du comportement des fonctionnalités de type cas d'utilisations

5.6.1 Diagramme de cas d'utilisation global (UML)

Vue d'ensemble des cas d'utilisation - Système de Billetterie

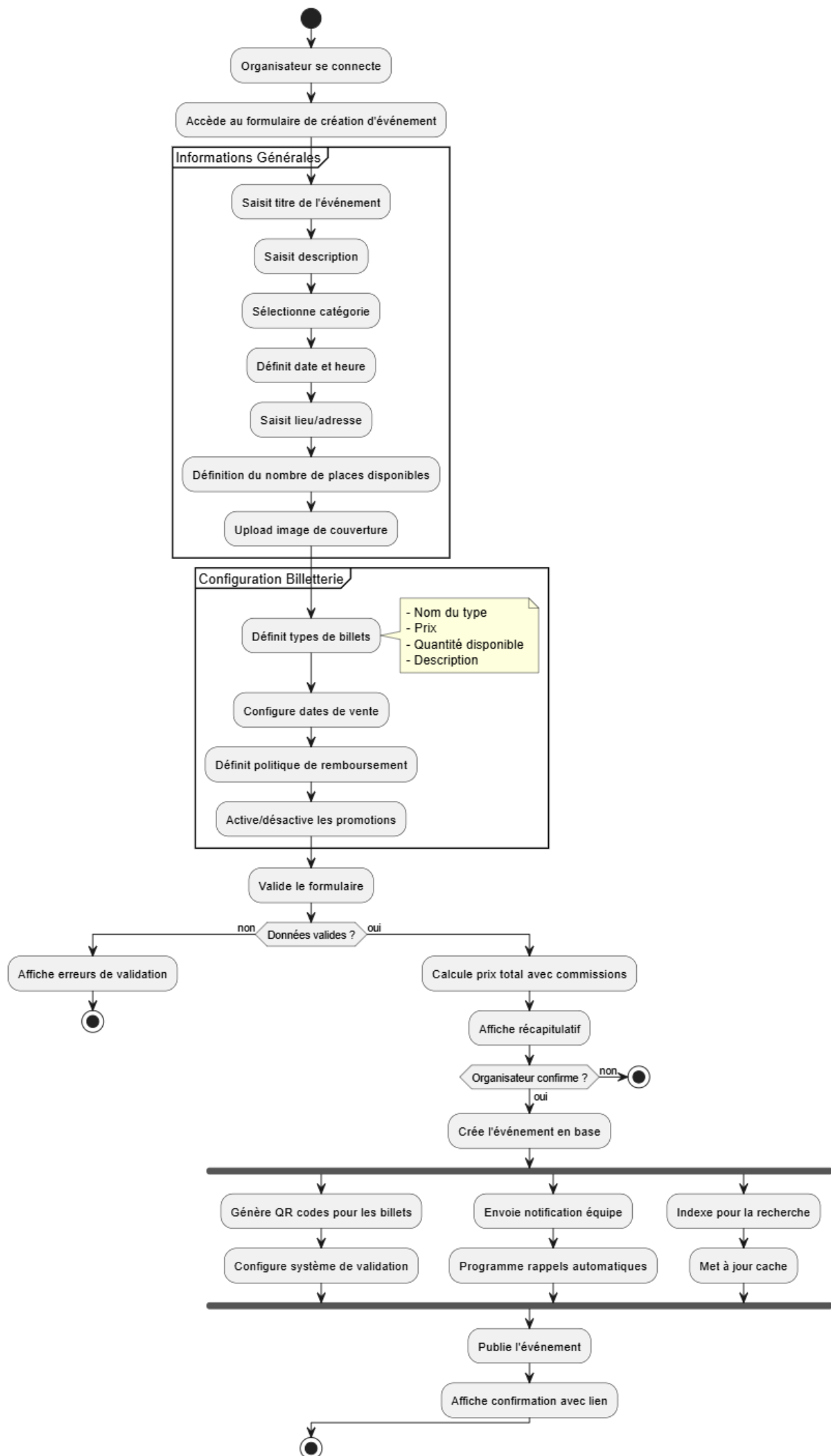


Diagrammes d'activités :

La création d'événement commence par la connexion de l'organisateur qui accède ensuite au formulaire dédié. Il y saisit d'abord les informations générales comprenant le titre, la description, la catégorie, la date, l'heure, le lieu, le nombre de places et peut ajouter une image de couverture. Il configure ensuite la billetterie en définissant les différents types de billets avec leurs prix et quantités, les dates de vente, la politique de remboursement et les options de promotion.

Après validation du formulaire, si des erreurs sont détectées, elles sont affichées et le processus s'arrête. Si tout est correct, le système calcule le prix total avec les commissions et présente un récapitulatif. Une fois confirmé par l'organisateur, l'événement est créé en base de données. Le système lance alors plusieurs tâches en parallèle : génération des QR codes et configuration de la validation, envoi des notifications et programmation des rappels, ainsi que l'indexation pour la recherche et la mise à jour du cache. L'événement est finalement publié et un message de confirmation avec le lien est affiché à l'organisateur.

Création d'Événement



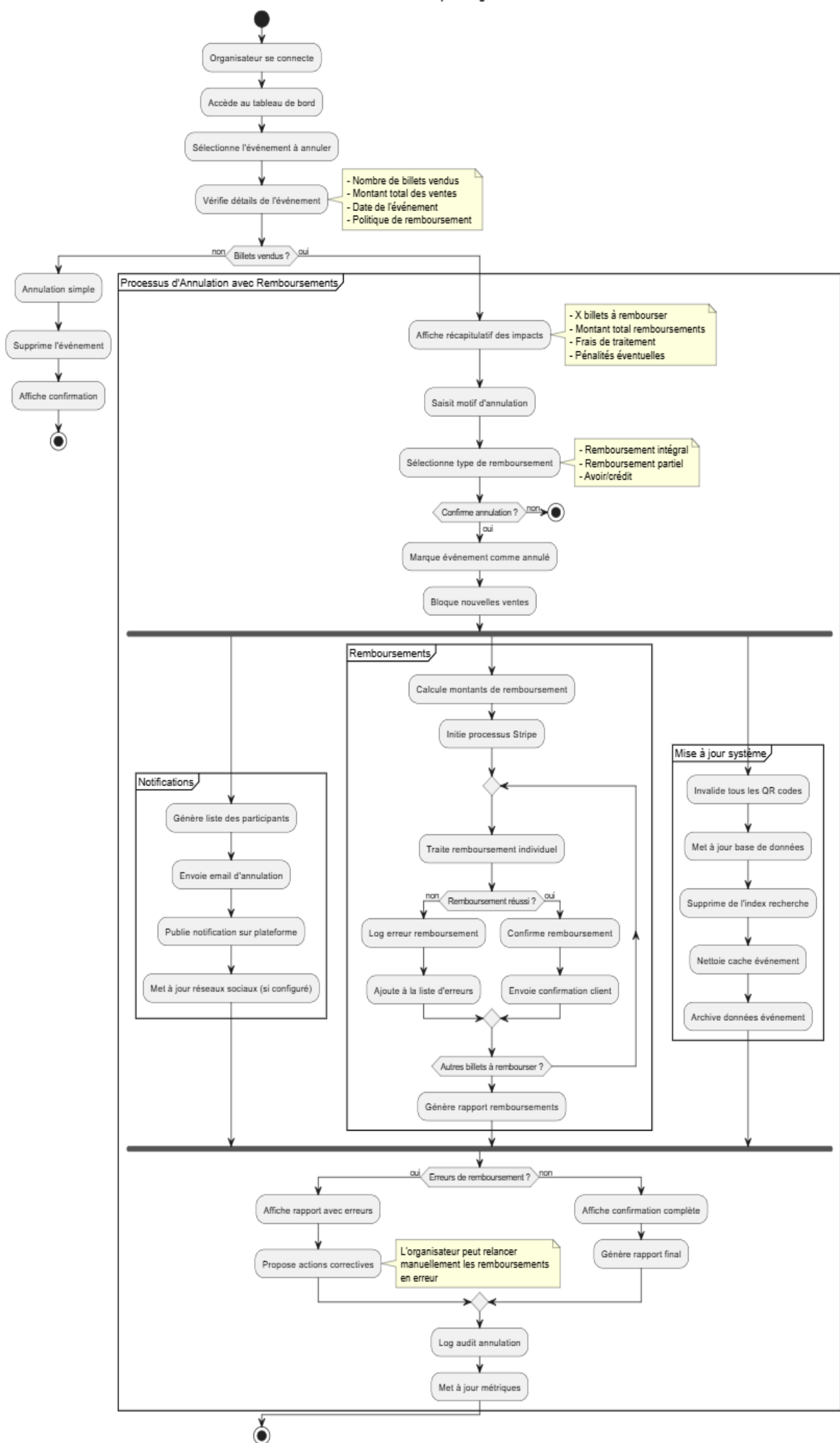
L'annulation d'événement commence lorsque l'organisateur se connecte, accède à son tableau de bord et sélectionne l'événement concerné. Le système vérifie les détails incluant le nombre de billets vendus, le montant des ventes et la politique de remboursement.

Si aucun billet n'a été vendu, l'événement est simplement supprimé. Si des billets ont été vendus, l'organisateur visualise un récapitulatif des impacts, saisit un motif d'annulation et choisit le type de remboursement souhaité. Après confirmation, l'événement est marqué comme annulé et les nouvelles ventes sont bloquées.

Le système lance alors trois processus parallèles : l'envoi des notifications aux participants par email et sur la plateforme, le traitement des remboursements via Stripe avec journalisation des erreurs éventuelles, et la mise à jour du système qui invalide les QR codes, actualise la base de données, supprime l'événement de la recherche et archive les données.

En cas d'erreurs de remboursement, un rapport avec actions correctives est affiché permettant de relancer manuellement les transactions échouées. Sinon, une confirmation complète avec rapport final est présentée. L'opération est enregistrée dans les logs d'audit et les métriques sont mises à jour.

Annulation d'Événement par l'Organisateur



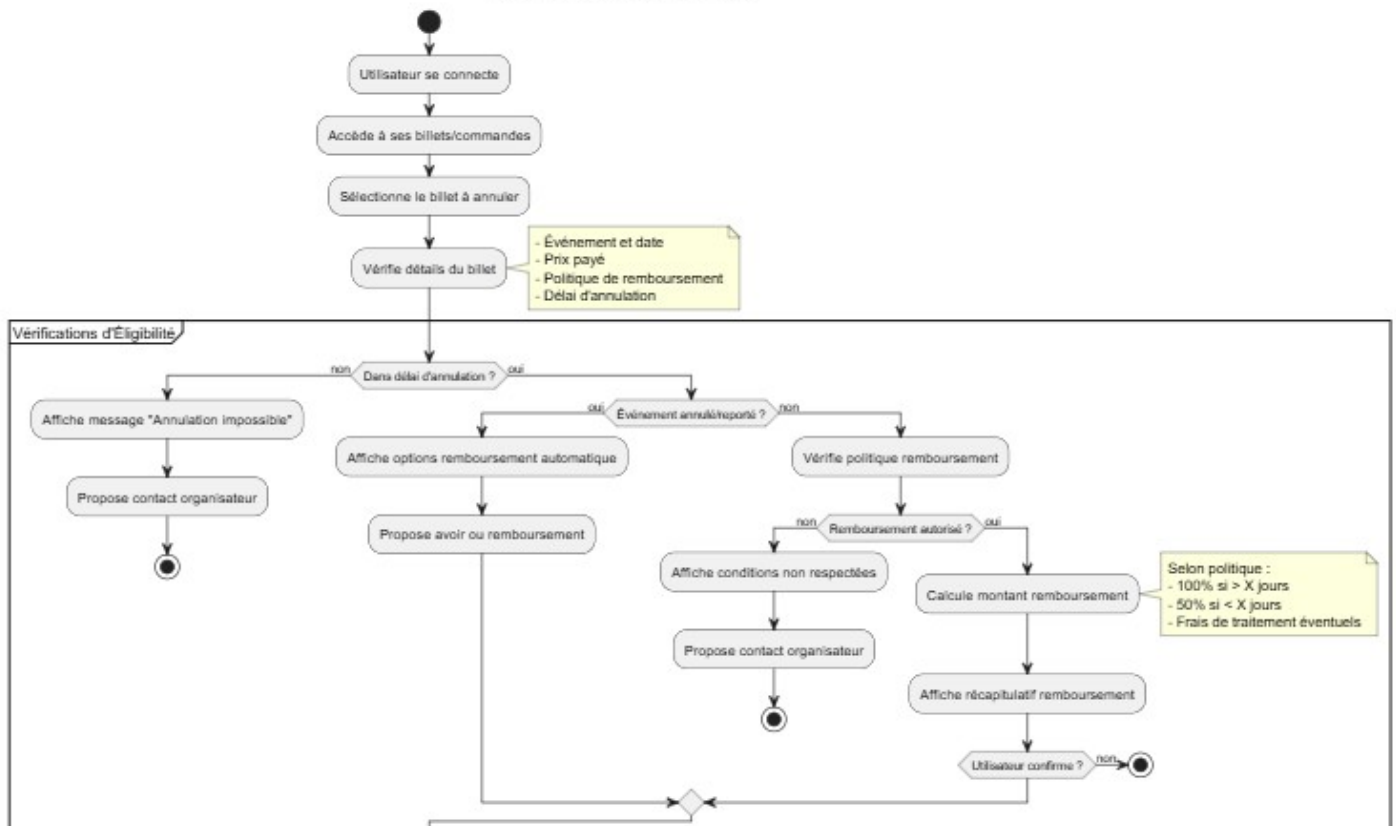
L'annulation de billet commence lorsque l'utilisateur se connecte, accède à ses billets et sélectionne celui à annuler. Le système vérifie les détails incluant l'événement, la date, le prix et la politique de remboursement.

Si le délai d'annulation est dépassé, un message indique l'impossibilité d'annuler et propose de contacter l'organisateur. Si l'événement est annulé ou reporté, des options de remboursement automatique ou d'avoir sont proposées. Pour un événement normal, le système vérifie l'éligibilité au remboursement, calcule le montant selon la politique définie et affiche un récapitulatif. L'utilisateur doit confirmer pour poursuivre.

Une fois confirmé, le billet est marqué comme annulé, le QR code est invalidé et la place est libérée. Le système lance quatre processus parallèles : le remboursement via Stripe avec gestion des erreurs éventuelles, l'envoi d'un email de confirmation avec mise à jour du statut de commande, la notification de l'organisateur avec actualisation des statistiques, et l'enregistrement dans les logs d'audit avec mise à jour des métriques.

Si une liste d'attente existe, le premier inscrit est notifié et peut acheter le billet libéré. L'utilisateur reçoit la confirmation d'annulation avec le délai de remboursement, peut donner un feedback et voit son interface mise à jour.

Annulation de Billet par l'Utilisateur



5.7 Fonctionnalités détaillées les plus significatives

5.7.1 Fonctionnalité 1 – Gestion des utilisateurs

- **Description** : création, authentification, modification et suppression de compte.
- **Sécurité** : hachage bcrypt, token JWT, validation Zod.
- **Valeur ajoutée** : expérience utilisateur fluide et sécurisée.

5.7.2 Fonctionnalité 2 – Gestion des rôles et permissions

- **Rôles** : Visiteur, Utilisateur, Organisateur, Administrateur.
- **Contrôle d'accès** : middleware RBAC sur les routes protégées.
- **Valeur ajoutée** : granularité et gestion fine des droits.

5.7.3 Fonctionnalité 3 – Gestion des événements

- **Organisateurs** : création, édition, publication, suppression.
- **Options avancées** : catégories, tarifs, quotas, codes promo.
- **Valeur ajoutée** : flexibilité et autonomie des organisateurs.

5.7.4 Fonctionnalité 4 – Système de billetterie

- **Achat** : interface intuitive avec panier et paiement Stripe.
- **Billet** : génération PDF + QR Code rotatif toutes les 12 h.
- **Valeur ajoutée** : expérience fluide et anti-fraude renforcée.

5.7.5 Fonctionnalité 5 – Tableau de bord administrateur

- **Vue globale** : événements, ventes, utilisateurs.
- **Statistiques** : taux de conversion, revenus, activité.
- **Valeur ajoutée** : supervision complète du système.

Annexe : Diagrammes de cas d'utilisation par rôle

Rôle Visiteur

Un visiteur non connecté peut explorer librement le catalogue d'événements disponibles sur la plateforme.

Il consulte les événements, effectue des recherches et affine ses résultats en filtrant par catégorie, date ou lieu.

Pour chaque événement, il accède aux détails complets incluant les tarifs et la disponibilité des billets.

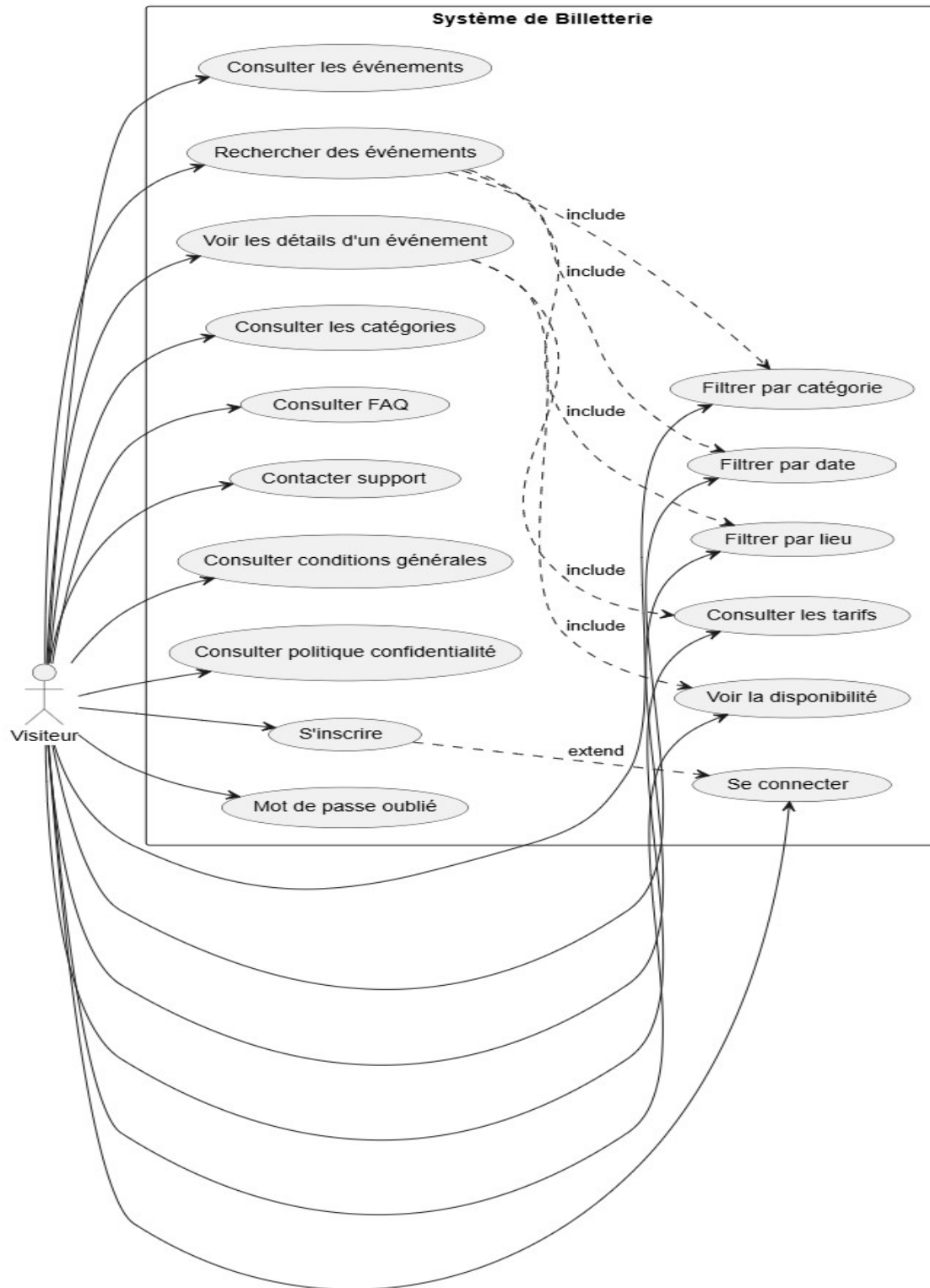
Le visiteur navigue également dans les différentes catégories d'événements proposées et accède aux informations légales et pratiques du site : FAQ, conditions générales de vente et politique de confidentialité.

En cas de besoin, il peut contacter le support client.

Pour aller plus loin et effectuer des achats, le visiteur doit créer un compte en s'inscrivant.

Il peut également se connecter s'il possède déjà un compte, ou utiliser la fonction de récupération de mot de passe en cas d'oubli.

Diagramme de cas d'utilisation - Rôle Visiteur



Rôle Utilisateur :

Un utilisateur connecté hérite de toutes les fonctionnalités d'un visiteur et bénéficie d'un accès complet au système de billetterie. Il gère sa session en se déconnectant, réinitialisant son mot de passe ou vérifiant son adresse email.

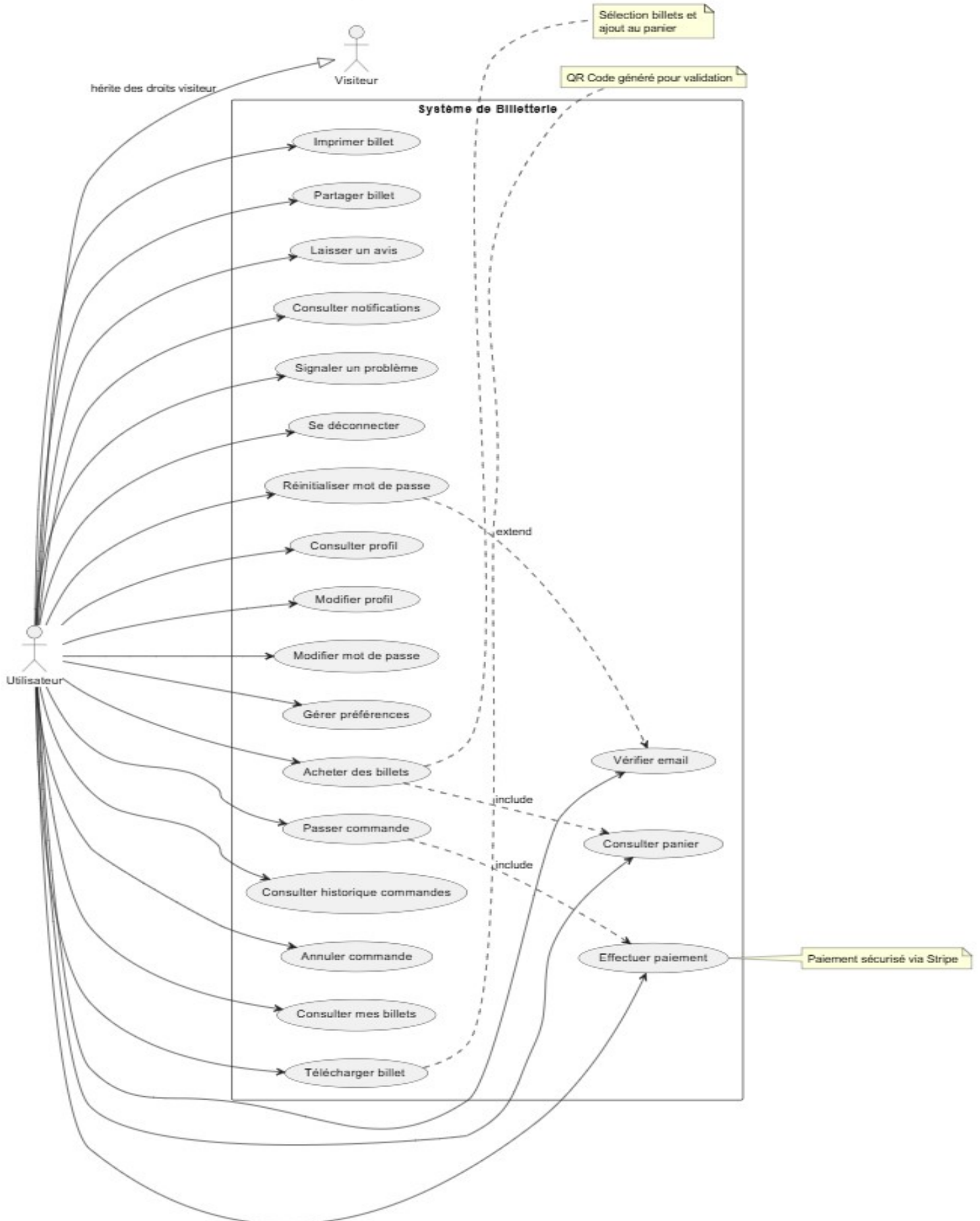
Son espace personnel lui permet de consulter et modifier son profil, changer son mot de passe et personnaliser ses préférences. Il accède à toutes ses notifications pour rester informé des actualités et mises à jour.

L'utilisateur peut acheter des billets en consultant son panier avant de passer commande. Le paiement s'effectue de manière sécurisée via Stripe. Il dispose d'un historique complet de ses commandes et peut les annuler si nécessaire avant l'événement.

Tous ses billets sont accessibles depuis son compte. Il peut les consulter, les télécharger au format électronique avec QR Code pour validation, les imprimer ou les partager avec d'autres personnes.

L'utilisateur interagit également avec la communauté en laissant des avis sur les événements auxquels il a participé. En cas de difficulté, il peut signaler un problème au support technique.

Diagramme de cas d'utilisation - Rôle Utilisateur



Rôle Organisateur :

Un organisateur hérite de tous les droits d'un utilisateur standard et dispose de fonctionnalités étendues pour gérer ses événements et leur commercialisation.

Il crée, modifie, publie ou annule ses événements, avec la possibilité de les dupliquer pour gagner du temps. Il gère également les catégories pour mieux organiser son catalogue. Pour chaque événement, il configure les différents types de billets, définit les tarifs, ajuste les disponibilités et peut créer des billets VIP. Il génère aussi des codes promotionnels pour stimuler les ventes.

Son tableau de bord lui offre une vue complète sur ses activités avec des statistiques détaillées : ventes par période, taux de conversion et suivi des revenus. Il consulte toutes les commandes, gère les remboursements si nécessaire et exporte des rapports pour ses analyses. Il accède à la liste complète des participants de ses événements et valide les billets en scannant les QR codes en temps réel lors de l'accès à l'événement.

L'organisateur communique directement avec ses participants en envoyant des notifications par email, push ou SMS. Il modère également les avis clients laissés sur ses événements.

Enfin, il personnalise son profil organisateur, gère son équipe de collaborateurs, configure les paramètres de son compte et consulte l'historique de toutes ses activités sur la plateforme.

Diagramme de cas d'utilisation - Rôle Organisateur

hérite des droits utilisateur



Utilisateur

Validation en temps réel
via QR code

Notifications :
- Email
- Push
- SMS

Système de Billetterie



Organisateur

Exporter rapports

Voir liste participants

Valider billets

Gérer accès événement

Envoyer notifications

Gérer avis clients

Gérer équipe

Configurer paramètres

Consulter historique activités

Modifier événement

Créer événement

Publier événement

Annuler événement

Dupliquer événement

Gérer catégories événements

Configurer billets VIP

Générer codes promotionnels

Consulter tableau de bord

Gérer remboursements

Communiquer avec participants

Scanner QR codes

Modifier profil organisateur

Modifier tarifs

Créer types de billets

Gérer disponibilités

Voir statistiques ventes

Suivre revenus

Consulter commandes

Analytics avancées :
- Ventes par période
- Conversion
- Revenus

include

include

extend

include

extend

include

include

include

include

extend

6. Spécifications techniques

6.1 Référencement

L'application utilise plusieurs mécanismes intégrés à Next.js afin d'optimiser son référencement naturel (SEO) :

- **URLs propres et sémantiques** grâce au routing basé sur les fichiers, permettant une meilleure indexation par les moteurs de recherche.
- **Titres et métadonnées dynamiques**, générés côté serveur, afin d'offrir des pages cohérentes et contextuelles.
- **Rendu côté serveur (SSR) et génération statique (SSG/ISR)** permettant aux robots d'indexation d'accéder directement au contenu HTML optimisé, améliorant la visibilité dans les moteurs de recherche.
- **Performance optimisée** via le code splitting automatique et l'optimisation native des images, contribuant au SEO technique.
- **Sitemap automatisé (prévu dans une version ultérieure)** pour faciliter l'exploration complète du site par les moteurs.

6.2 Environnement technique

L'environnement technique repose sur une architecture moderne centrée sur Next.js et Prisma.

Backend : Next.js & API Routes

Le backend repose sur Next.js et ses API Routes, qui permettent d'exposer une API REST directement au sein de l'application sans serveur dédié.

Cette approche garantit une cohérence technologique entre le front-end et le back-end et facilite le déploiement.

ORM : Prisma (PostgreSQL)

Prisma est un ORM moderne offrant :

- un client entièrement typé généré à partir du schéma Prisma,
- un modèle de données clair et déclaratif,
- des migrations gérées automatiquement,
- d'excellentes performances grâce à son moteur Rust,
- un accès simplifié à PostgreSQL via un client pensé pour TypeScript.

Prisma Studio permet également d'inspecter graphiquement les données en base.

Base de données : PostgreSQL

PostgreSQL est choisi pour :

- sa robustesse et sa conformité aux normes SQL,
- son respect strict des transactions ACID,
- ses performances sur les requêtes complexes,
- ses types avancés et index spécialisés,
- son extensibilité et sa gratuité.

C'est la solution standard pour les applications modernes nécessitant fiabilité et scalabilité.

Authentication : bcrypt

Bcrypt assure :

- un hachage sécurisé et lent (résistance brute-force),
- un salt intégré à chaque hash,
- un cost factor ajustable,
- une validation sécurisée des mots de passe sans stocker la version originale.

Il s'agit d'une méthode recommandée par l'OWASP pour la gestion sécurisée des mots de passe.

Outils et gestion du projet

- **Git / GitHub** : versionnement du code, gestion des branches, pull requests.
- **Postman** : test des endpoints et documentation interactive de l'API.
- **Trello** : gestion du backlog, suivi des tâches et organisation Kanban.

Gestion des environnements

L'application est structurée autour de trois environnements distincts :

- **Développement (DEV)**
Exécution locale via Next.js (mode dev), base PostgreSQL locale, et migrations Prisma automatiques.
Le hot reload permet des itérations rapides et la génération du client Prisma est déclenchée automatiquement.
- **Test (TEST)**
Utilisation d'une base PostgreSQL dédiée aux tests, isolée du reste du système.
Les jeux d'essai, les tests API, les tests de sécurité et les tests de charge sont exécutés dans cet environnement.
- **Production (PROD)**
Application construite en mode optimisé via next build, servie derrière un reverse proxy en HTTPS.
PostgreSQL, Redis, Nodemailer, Stripe et le système de fichiers sont exécutés dans des conteneurs séparés via Docker Compose.

Chaque environnement dispose de son propre fichier de configuration (.env.local, .env.test, .env.production) pour assurer une isolation stricte et éviter les fuites de secrets.

Conteneurisation & orchestration

L'application repose sur une architecture conteneurisée à l'aide de **Docker** :

- **Next.js** dans un conteneur dédié (front + API Routes)
- **PostgreSQL** dans un conteneur sécurisé avec stockage persistant

- **Redis** pour le cache et les sessions (si activé)
- **SMTP / Mail service**
- **Stripe Webhook Listener**
- **Stockage des fichiers** (volume Docker pour les images & billets PDF)

L'ensemble est orchestré via **Docker Compose**, facilitant :

- la reproductibilité des environnements
- la montée en charge
- le déploiement sur serveur dédié ou VPS
- la modularité des services

Les volumes Docker assurent la persistance des données (base, images, logs, backups).

Workflow Git

Le projet utilise **GitHub** pour le versionnement et le suivi du code, avec un workflow simple :

- **branche main** : version stable
- **branche feature/*** : nouvelles fonctionnalités
- **pull requests** systématiques pour valider les modifications
- **versionnage sémantique** pour les releases (1.0.0, 1.1.0, etc.)

Ce workflow permet d'assurer la stabilité du code et de faciliter la revue, même en projet individuel.

Configuration & gestion des secrets

Les paramètres sensibles sont stockés dans des fichiers `.env` non versionnés, contenant notamment :

- `STRIPE_SECRET_KEY`

- SMTP_USER / SMTP_PASSWORD
- DATABASE_URL
- JWT_SECRET
- REDIS_URL

Pour renforcer la sécurité :

- Les fichiers .env* sont exclus via .gitignore.
- Les valeurs sont chargées au runtime via process.env.
- Aucune clé sensible n'est exposée côté client.

Ces bonnes pratiques garantissent la sécurité des secrets et la conformité RGPD/PCI DSS.

6.3 Navigation et accessibilité

Navigation

L'application propose une navigation claire et intuitive, organisée autour d'une structure cohérente permettant aux utilisateurs d'accéder rapidement aux fonctionnalités essentielles :

- **Header global** : accès à la page d'accueil, à la recherche d'événements, aux catégories et à l'espace utilisateur (connexion, profil, billets, paramètres).
- **Navigation organisateur** : création d'événement, gestion des événements publiés, suivi des ventes et des statistiques.
- **Pied de page (footer)** : mentions légales, politique de confidentialité, CGU/CGV, contact et support.
- **Parcours utilisateurs structurés** : achat de billets, création d'événements, consultation des commandes, gestion du compte.
- **Interface responsive** permettant une navigation fluide sur desktop, tablette et mobile.

Cette organisation garantit une expérience utilisateur simple, logique et adaptée à

chaque profil (utilisateur, organisateur, administrateur).

Accessibilité

L'application respecte les principes des normes **WCAG 2.1** et **RGAA 4.1**, afin d'assurer l'accès à tous les utilisateurs, y compris en situation de handicap :

- **Navigation au clavier** complète, avec gestion précise du focus.
- **Contrastes élevés** conformes aux recommandations WCAG.
- **Texte alternatif** sur toutes les images importantes pour les lecteurs d'écran.
- **Balises sémantiques et rôles ARIA** garantissant une interprétation correcte par les technologies d'assistance.
- **Responsive design** permettant une lisibilité optimale sur tous les écrans.

Ces mesures assurent une utilisation fluide, accessible et conforme aux standards modernes.

6.4 Services tiers

Plusieurs services externes sont intégrés dans l'application pour renforcer ses fonctionnalités :

Paielements – Stripe

Stripe gère l'intégralité du processus de paiement :

- création et validation des paiements,
- gestion des statuts de transaction,
- réception des notifications via **webhooks**,
- synchronisation en temps réel avec les commandes internes.

Email – SMTP via Nodemailer

L'application utilise Nodemailer relié à un serveur SMTP pour :

- envoyer les confirmations de commandes,

- transmettre les billets électroniques,
- notifier les organisateurs et utilisateurs.

Stockage des fichiers

Un volume de stockage est utilisé pour héberger :

- les images d'événements,
- les fichiers générés (tickets PDF, QR codes).

Cette séparation favorise la modularité et la scalabilité.

Cache – Redis

Redis peut être utilisé pour :

- accélérer la gestion des sessions utilisateurs,
- stocker les données fréquemment consultées,
- réduire la charge sur PostgreSQL.

6.5 Sécurité

La sécurité de l'application repose sur une combinaison de mesures techniques, organisationnelles et structurelles.

Authentification & gestion des sessions

- Hash des mots de passe via **bcrypt**.
- Gestion des droits et rôles utilisateurs (admin, organisateur, utilisateur).
- Sessions sécurisées ou tokens selon la configuration (Better Auth possible).

Communications sécurisées

- Communication **100 % HTTPS**.
- Protection via en-têtes de sécurité :

- **CSP** (Content Security Policy),
 - **HSTS**,
 - **X-Frame-Options**,
 - **X-Content-Type-Options**.
- Cookies protégés (Secure, HttpOnly, SameSite).

Sécurité applicative

- Rate limiting pour limiter les attaques par force brute.
- Journalisation des tentatives de connexion (LoginAttempt).
- Blocage automatique d'IP malveillantes (BlockedIP).
- Audit des actions sensibles via **AuditLog**.

Sécurité des données

- Respect des permissions selon les rôles.
- Isolation des conteneurs via Docker.
- Sauvegardes régulières de PostgreSQL (SystemBackup).
- Politique stricte de gestion et rotation des secrets.

Surveillance et monitoring

- Journalisation centralisée (ActivityLog).
- Suivi des événements critiques.
- Alerte en cas d'échec de paiement ou activité inhabituelle.

Ces mécanismes combinés garantissent une protection robuste contre les attaques courantes et assurent l'intégrité du système.

7. Réalisations

Fonctionnalités clés développées :

- **Inscription et connexion sécurisée**

[← Retour à l'accueil](#)

Authentification

Connexion

Inscription

 Particulier

 Entreprise

Informations personnelles

Prénom *

Nom *

Email *

coulet.corentin@gmail.com

Téléphone

+33 6 12 34 56 78

Date de naissance

jj/mm/aaaa



Sécurité

Mot de passe *

.....

Minimum 8 caractères

Confirmer le mot de passe *

☐ J'accepte les [conditions d'utilisation](#) et la [politique de confidentialité](#)

S'inscrire comme particulier

The image shows a web form for authentication. At the top left, there is a link '← Retour à l'accueil'. The form is titled 'Authentification' in blue. It has two tabs: 'Connexion' (active) and 'Inscription'. Below the tabs, there are two input fields: 'Email' with the value 'coulet.corentin@gmail.com' and 'Mot de passe' with masked characters '*****'. At the bottom, there is a blue button labeled 'Se connecter'.

- **Système de création et de gestion d'événements**
- **Achat de billets avec vérification des places disponibles**
- **Filtrage d'événements par catégorie, date ou lieu**
- **Tableau de bord administrateur avec statistiques clés**

Exemple de code intégré :

- DTO et validation lors de l'inscription.

Cette validation côté client offre un feedback immédiat à l'utilisateur sans attendre la réponse du serveur. Elle améliore l'expérience utilisateur (UX) en évitant des allers-retours réseau inutiles. Cependant, elle ne remplace jamais la validation serveur qui reste obligatoire pour la sécurité.



```
1  const handleRegister = async (e: React.FormEvent) => {
2    e.preventDefault()
3    setLoading(true)
4
5    if (registerData.password !== registerData.confirmPassword) {
6      alert('Les mots de passe ne correspondent pas')
7      setLoading(false)
8      return
9    }
10
11    if (!registerData.acceptTerms) {
12      alert('Vous devez accepter les conditions d\'utilisation')
13      setLoading(false)
14      return
15    }
16
17    if (userType === 'organizer' && (!registerData.companyName || !registerData.siret)) {
18      alert('Veuillez remplir les informations de votre entreprise')
19      setLoading(false)
20      return
21    }
22
23    // Appel API...
24  }
```

Zod permet de définir un schéma déclaratif qui valide ET type les données simultanément. La méthode `.refine()` ajoute des règles métier personnalisées. Toute donnée non conforme est rejetée avant même d'atteindre la base de données, évitant ainsi les injections et données corrompues.



```

1  import { z } from 'zod';
2
3  const registerSchema = z.object({
4    email: z.string().email('Email invalide'),
5    password: z.string().min(8, 'Le mot de passe doit contenir au moins 8 caractères'),
6    name: z.string().min(2, 'Le nom doit contenir au moins 2 caractères').optional(),
7    confirmPassword: z.string().optional(),
8  }).refine((data) => {
9    if (data.confirmPassword && data.password !== data.confirmPassword) {
10     return false;
11    }
12    return true;
13  }, {
14    message: "Les mots de passe ne correspondent pas",
15    path: ["confirmPassword"],
16  });
17
18  async function handleRegister(request: NextRequest) {
19    const { data, error } = await validateBody(request, registerSchema);
20    if (error) return error;
21
22    const result = await authService.register(data.email, data.password, data.name);
23  }

```

Les cookies httpOnly ne sont pas accessibles par JavaScript côté client, ce qui empêche les attaques XSS (Cross-Site Scripting) de voler le token. L'attribut sameSite: 'strict' protège contre les attaques CSRF (Cross-Site Request Forgery). En production, secure: true force HTTPS.



```

1  if (result.token) {
2    response.cookies.set('auth-token', result.token, {
3      httpOnly: true,
4      secure: process.env.NODE_ENV === 'production',
5      sameSite: 'strict',
6      maxAge: 60 * 60 * 24 * 7,
7    });
8  }

```

Le service implémente la logique métier : vérification d'unicité, hash du mot de passe avec bcrypt (algorithme à coût adaptatif), et création sécurisée. Le select Prisma garantit que le mot de passe n'est jamais retourné dans la réponse. Cette couche constitue la défense finale contre les données invalides.

```
1  async register(email: string, password: string, name?: string): Promise<LoginResult | null> {
2    try {
3      if (!email || !password) {
4        throw new Error('Email and password are required');
5      }
6
7      if (password.length < 8) {
8        throw new Error('Password must be at least 8 characters long');
9      }
10
11     const existingUser = await prisma.user.findUnique({
12       where: { email }
13     });
14     if (existingUser) {
15       throw new Error('User already exists');
16     }
17
18     const hashedPassword = await bcrypt.hash(password, 10);
19
20     const user = await prisma.user.create({
21       data: {
22         email,
23         password: hashedPassword,
24         name,
25         role: 'USER'
26       },
27       select: {
28         id: true,
29         email: true,
30         name: true,
31         role: true,
32       }
33     });
34
35     return { user, token };
36   } catch (error) {
37     console.error('Registration error:', error);
38     return null;
39   }
40 }
```

Les DTOs (Data Transfer Objects) définissent des contrats explicites entre les couches. RegisterDto pour l'entrée, UserResponseDto pour la sortie. TypeScript garantit qu'aucun développeur ne peut accidentellement exposer le mot de passe hashé. C'est du type safety appliqué à la sécurité.



```
1  export interface RegisterDto {
2    email: string;
3    password: string;
4    name?: string;
5  }
6
7  export interface UserResponseDto {
8    id: string;
9    email: string;
10   name: string;
11   role: UserRole;
12   isVerified: boolean;
13   createdAt: Date;
14 }
```

- Middleware d'authentification.

Ce middleware assure la sécurité des routes API en vérifiant l'authentification des utilisateurs via des tokens JWT et en validant leurs sessions en base de données. Il

propose également un système de contrôle d'accès basé sur les rôles (RBAC) pour restreindre l'accès à certaines fonctionnalités selon les privilèges utilisateur.



```
1  export interface AuthenticatedRequest extends NextApiRequest {  
2    user?: {  
3      id: string;  
4      email: string;  
5      role?: string;  
6      sessionId?: string;  
7    };  
8  }
```



```
1 export async function isAuthenticated(  
2   req: NextApiRequest,  
3   res: NextApiResponse,  
4   next: NextHandler  
5 ) {  
6   try {  
7     const token = req.cookies.token || req.headers.authorization?.replace('Bearer ', '');  
8  
9     if (!token) {  
10      return res.status(401).json({ message: 'Authentication required' });  
11    }  
12  
13    const user = await authService.validateToken(token);  
14  
15    if (!user || !user.sessionId) {  
16      return res.status(401).json({ message: 'Invalid or expired token' });  
17    }  
18  
19    const session = await prisma.userSession.findUnique({ where: { id: user.sessionId } });  
20    if (!session || session.expiresAt < new Date()) {  
21      return res.status(401).json({ message: 'Session expirée' });  
22    }  
23  
24    (req as AuthenticatedRequest).user = user;  
25  
26    return next();  
27  } catch (error) {  
28    console.error('Authentication error:', error);  
29    return res.status(401).json({ message: 'Authentication failed' });  
30  }  
31 }
```



```
1 export function hasRoles(roles: string[]) {  
2   return async (req: NextApiRequest, res: NextApiResponse, next: NextHandler) => {  
3     try {  
4       await isAuthenticated(req, res, async () => {  
5         const user = (req as AuthenticatedRequest).user;  
6         if (!user || !user.role || !roles.includes(user.role)) {  
7           return res.status(403).json({ message: 'Insufficient permissions' });  
8         }  
9         return next();  
10      });  
11    } catch (error) {  
12      console.error('Authorization error:', error);  
13      return res.status(403).json({ message: 'Insufficient permissions' });  
14    }  
15  };  
16 }
```

- Requête Prisma pour créer un événement lié à un utilisateur.

Exemple avec la création d'une commande de billets : Cette fonction crée une commande de billets de manière atomique grâce à une transaction. Elle commence par vérifier la disponibilité de chaque ticket demandé en consultant la base de données et en s'assurant qu'aucun ticket n'est déjà réservé et que l'événement associé n'est pas passé. Une fois ces vérifications effectuées, elle calcule le montant total de la commande en multipliant le nombre de tickets par leur prix unitaire. La fonction crée ensuite un enregistrement de commande dans la base avec le statut "en attente de paiement" et des métadonnées incluant les informations du client et un identifiant de transaction unique. Elle lie ensuite tous les tickets validés à cette nouvelle commande en mettant à jour leur statut et leur référence de commande. Enfin, elle retourne l'objet complet de la commande avec toutes les informations relatives à l'utilisateur, aux tickets et au paiement éventuel. Si une erreur survient à n'importe quelle étape du processus, la transaction est automatiquement annulée pour garantir qu'aucune donnée partielle ne soit enregistrée, assurant ainsi la cohérence et l'intégrité des données dans le système.

Version prisma :

```

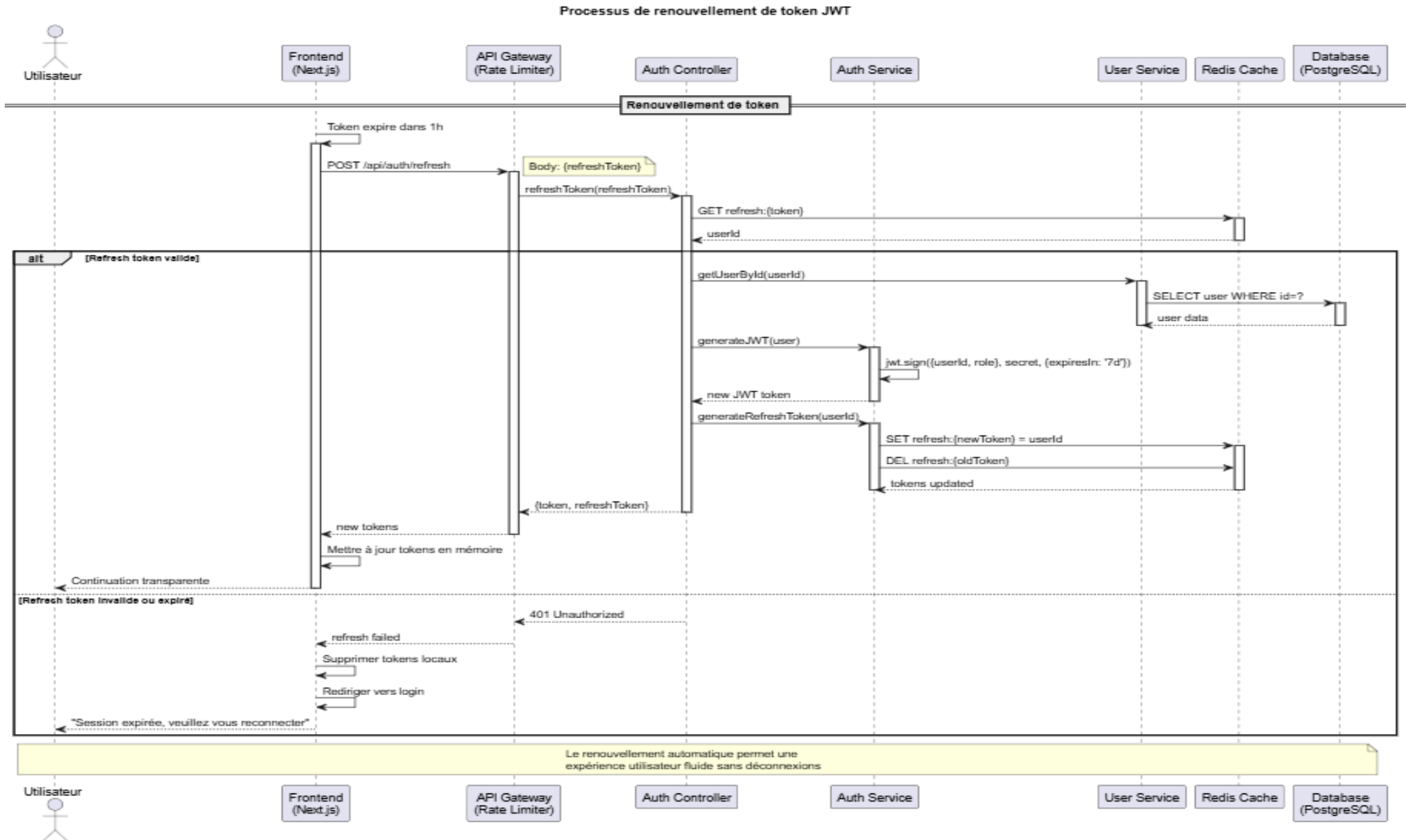
1  async createOrder(data: OrderCreateInput): Promise<OrderWithRelations> {
2    const { userId, tickets, customerInfo } = data;
3    return await prisma.$transaction(async (tx) => {
4      let totalAmount = 0;
5      const ticketItems: Array<{
6        ticketId: string;
7        quantity: number;
8        unitPrice: number;
9      }> = [];
10     const reservedTickets: string[] = [];
11
12     try {
13       for (const item of tickets) {
14         const ticket = await tx.ticket.findUnique({
15           where: { id: item.ticketId },
16           include: {
17             event: true,
18             order: true
19           }
20         });
21
22         if (!ticket) {
23           throw new Error(`Ticket with ID ${item.ticketId} not found`);
24         }
25
26         if (ticket.orderId) {
27           throw new Error(`Ticket ${item.ticketId} is already reserved`);
28         }
29
30         if (ticket.event && new Date(ticket.event.date) < new Date()) {
31           throw new Error(`Event ${ticket.event.title} has already passed`);
32         }
33
34         const unitPrice = 50;
35
36         totalAmount += unitPrice * item.quantity;
37         ticketItems.push({
38           ticketId: item.ticketId,
39           quantity: item.quantity,
40           unitPrice
41         });
42
43         reservedTickets.push(item.ticketId);
44       }
45
46       const order = await tx.order.create({
47         data: {
48           userId,
49           status: OrderStatus.pending_payment,
50           totalPrice: totalAmount,
51           currency: 'EUR',
52           metadata: {
53             customerInfo,
54             ticketItems,
55             transactionId: `TXN_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`
56           }
57         }
58       });
59
60       for (const ticketId of reservedTickets) {
61         await tx.ticket.update({
62           where: { id: ticketId },
63           data: {
64             orderId: order.id,
65             status: 'pending'
66           }
67         });
68       }
69
70       return await tx.order.findUnique({
71         where: { id: order.id },
72         include: {
73           user: {
74             select: {
75               id: true,
76               name: true,
77               email: true
78             }
79           },
80           tickets: {
81             include: {
82               event: true
83             }
84           },
85           payment: true
86         }
87       }) as OrderWithRelations;
88
89     } catch (error) {
90       throw new Error(`Order creation failed: ${error instanceof Error ? error.message : 'Unknown error'}`);
91     }
92   });
93 }

```

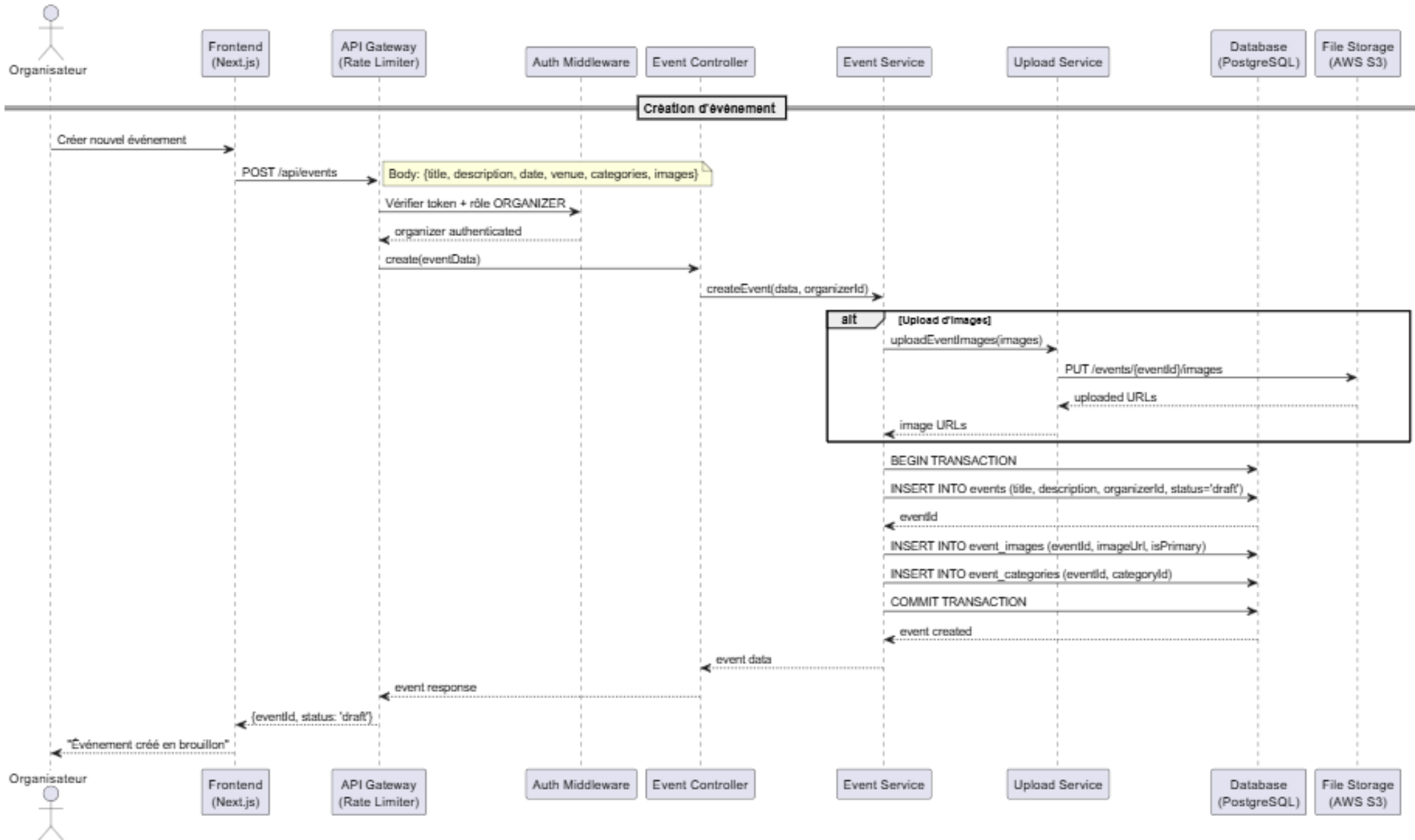
Version SQL :

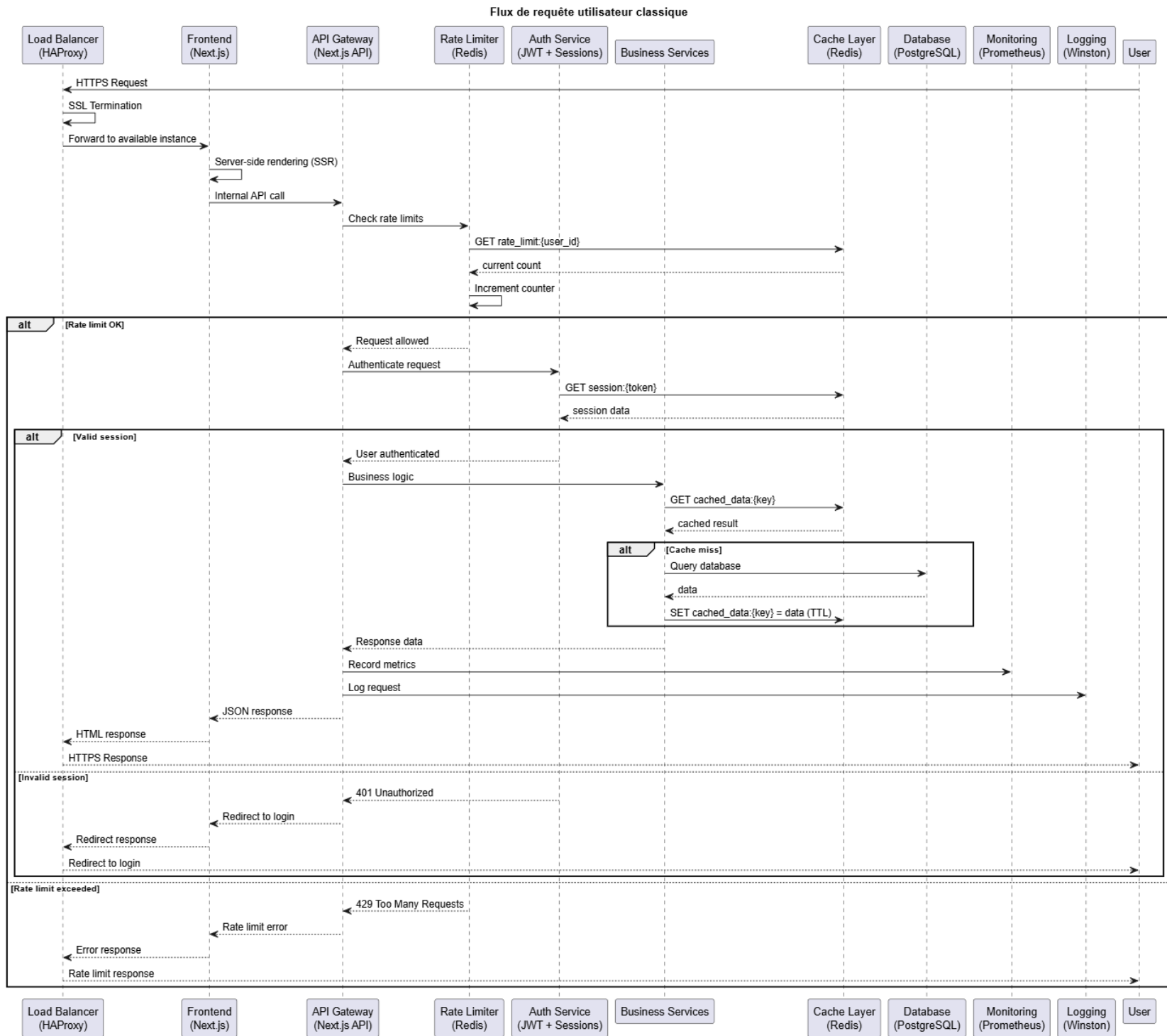
```
1 BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
2
3 SELECT t.id, t."eventId", t."orderId", t.status,
4        e.title AS event_title, e.date AS event_date
5 FROM "Ticket" t
6 INNER JOIN "Event" e ON t."eventId" = e.id
7 WHERE t.id = ANY($1)
8 FOR UPDATE NOWAIT;
9
10 DO $$
11 DECLARE
12     v_order_id UUID;
13     v_total_amount NUMERIC := 0;
14     v_ticket_count INT;
15     v_unit_price NUMERIC := 50;
16 BEGIN
17     SELECT COUNT(*) INTO v_ticket_count
18     FROM "Ticket" t
19     INNER JOIN "Event" e ON t."eventId" = e.id
20     WHERE t.id = ANY($1)
21           AND t."orderId" IS NULL
22           AND e.date >= NOW();
23
24     IF v_ticket_count < ARRAY_LENGTH($1, 1) THEN
25         RAISE EXCEPTION 'One or more tickets are not available';
26     END IF;
27
28     v_total_amount := v_ticket_count * v_unit_price;
29
30     INSERT INTO "Order" (
31         id, "userId", status, "totalPrice",
32         currency, metadata, "createdAt", "updatedAt"
33     ) VALUES (
34         gen_random_uuid(), $2, 'pending_payment', v_total_amount,
35         'EUR',
36         jsonb_build_object(
37             'customerInfo', $3,
38             'ticketItems', '[]'::JSONB,
39             'transactionId',
40             CONCAT('TXN_', EXTRACT(EPOCH FROM NOW())::TEXT, '_',
41                 substr(md5(random())::text, 1, 9))
42         ),
43         NOW(), NOW()
44     ) RETURNING id INTO v_order_id;
45
46     UPDATE "Ticket"
47     SET "orderId" = v_order_id, status = 'pending', "updatedAt" = NOW()
48     WHERE id = ANY($1) AND "orderId" IS NULL;
49
50     IF NOT FOUND THEN
51         RAISE EXCEPTION 'Failed to reserve tickets';
52     END IF;
53 END $$;
54
55 SELECT o.id, o.status, o."totalPrice", o.currency, o."createdAt",
56        json_build_object('id', u.id, 'name', u.name,
57                          'email', u.email) AS user,
58        (SELECT json_agg(json_build_object(
59            'id', t.id, 'code', t.code, 'status', t.status,
60            'event', json_build_object('id', e.id, 'title', e.title,
61                                       'date', e.date, 'location', e.location)
62        ))
63     FROM "Ticket" t
64     INNER JOIN "Event" e ON t."eventId" = e.id
65     WHERE t."orderId" = o.id
66     ) AS tickets
67 FROM "Order" o
68 INNER JOIN "User" u ON o."userId" = u.id
69 WHERE o.id = (SELECT id FROM "Order" ORDER BY "createdAt" DESC LIMIT 1);
70
71 COMMIT;
```

Diagrammes de séquence :

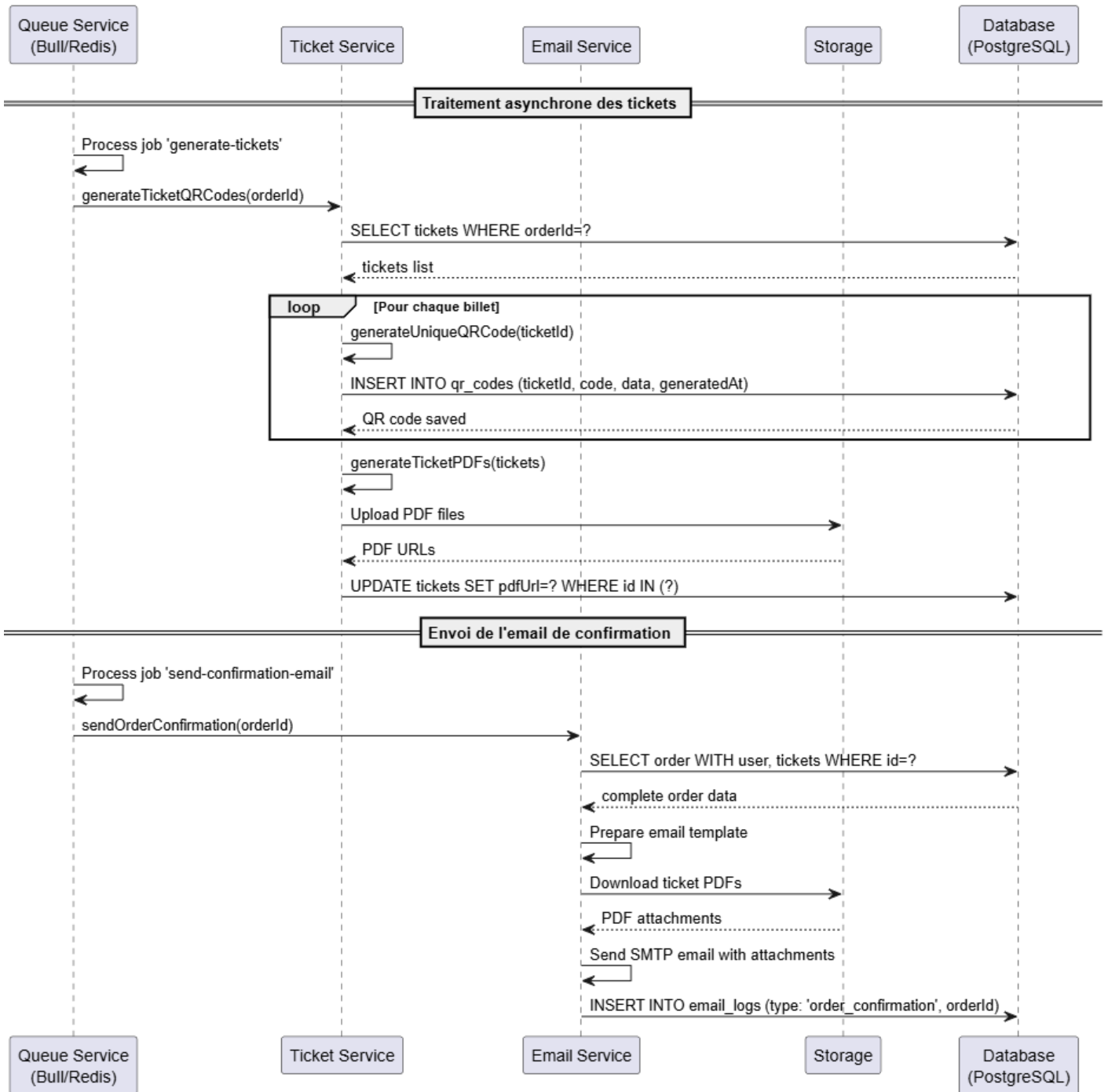


Creation d'évènement par l'organisateur





Génération des billets et envoi d'emails de confirmation



8. Éléments de sécurité de l'application

La sécurité de l'application M&C Society a été intégrée dès la phase de conception et repose sur une approche globale couvrant la protection des données, l'authentification, la prévention des attaques et le contrôle des accès. L'objectif est d'assurer la confidentialité, l'intégrité et la disponibilité des informations tout en respectant les normes et bonnes pratiques en matière de cybersécurité.

8.1 Protection des mots de passe

La protection des mots de passe constitue un élément fondamental de la sécurité de l'application :

- **Hachage robuste** : tous les mots de passe sont hachés avec l'algorithme bcrypt, configuré pour 10 à 12 rounds selon les besoins de performance et de sécurité.
- **Politique de mot de passe fort** : les mots de passe doivent comporter au minimum 8 caractères et inclure une majuscule, une minuscule, un chiffre et un caractère spécial, afin de limiter les risques de compromission par force brute ou dictionnaire.
- **Gestion de l'historique** : chaque modification de mot de passe est suivie via le champ `passwordChangedAt`, ce qui permet de détecter et de prévenir la réutilisation de mots de passe anciens ou compromis.
- **Aucune conservation en clair** : les mots de passe ne sont jamais stockés en clair dans la base de données, assurant une protection maximale en cas de fuite de données.
- **Suivi et audit** : toutes les opérations relatives aux mots de passe (création, modification, réinitialisation) sont enregistrées dans les journaux de sécurité, permettant une traçabilité complète.

8.2 Authentification sécurisée par JWT

L'authentification repose sur l'usage de **JSON Web Tokens (JWT)**, avec des mécanismes robustes pour la gestion des sessions et le contrôle d'accès :

- **Séparation des tokens** : l'application utilise des tokens d'accès de courte durée (15 minutes) et des tokens de rafraîchissement d'une durée plus longue (7 jours), afin de limiter l'exposition en cas de compromission.
- **Stockage sécurisé des sessions** : toutes les sessions utilisateur sont conservées dans la base PostgreSQL, permettant le suivi et la révocation centralisée en cas de besoin.
- **Invalidation automatique** : les sessions expirées sont supprimées automatiquement, et un administrateur peut révoquer manuellement toute session suspecte.
- **Contrôle d'accès basé sur les rôles (RBAC)** : l'application distingue différents niveaux d'accès (VISITOR < USER < ORGANIZER < ADMIN), garantissant que chaque utilisateur ne puisse exécuter que les actions autorisées selon son rôle.
- **Protection contre le détournement** : les tokens sont signés et vérifiés côté serveur, et les mécanismes de renouvellement sont sécurisés pour éviter l'usurpation d'identité.

8.3 Protection contre les injections et XSS

M&C Society met en œuvre des mécanismes stricts pour prévenir les attaques classiques telles que les injections SQL ou les scripts intersites (XSS) :

- **Validation stricte des entrées** : toutes les données fournies par les utilisateurs sont validées à l'aide de schémas robustes (via Zod), afin de garantir leur conformité et d'empêcher l'injection de code malveillant.
- **Utilisation de Prisma ORM** : le recours à un ORM typé et sécurisé permet de générer des requêtes préparées automatiquement, limitant fortement le risque d'injection SQL.
- **Filtrage des réponses** : les champs sensibles, tels que les mots de passe ou les informations financières, sont systématiquement exclus des réponses aux clients.
- **Headers de sécurité et CSP** : l'application applique une Content Security Policy (CSP) stricte et des headers HTTP de sécurité (X-Content-Type-Options, X-Frame-Options, Referrer-Policy) pour limiter le chargement de ressources non autorisées et réduire le risque d'attaques XSS.
- **Protection CSRF** : des jetons synchronisés sont utilisés pour protéger les formulaires critiques contre les attaques de falsification de requêtes intersites.

8.4 Gestion des accès CORS

La configuration Cross-Origin Resource Sharing (CORS) est optimisée pour limiter les échanges aux domaines autorisés :

- **Domaines autorisés** : seuls les domaines explicitement configurés peuvent accéder à l'API, réduisant le risque de requêtes non autorisées depuis des sites tiers.
- **Configuration par environnement** : les règles CORS sont adaptées selon l'environnement (développement, test, production) pour garantir flexibilité et sécurité.
- **Restriction des méthodes et headers** : les méthodes HTTP autorisées et les headers exposés sont strictement limités aux besoins fonctionnels, réduisant la surface d'attaque et les possibilités d'exploitation abusive.

8.5 Conformité RGPD

- L'application **M&C Society** respecte le Règlement Général sur la Protection des Données (RGPD) et met en œuvre les principes de transparence, de minimisation et de sécurité des données. Les mesures suivantes sont intégrées au projet :
- **Droit d'accès** : chaque utilisateur peut consulter l'ensemble de ses données personnelles à partir de son espace sécurisé.
- **Droit de portabilité** : un export complet et structuré de ses données est disponible au format **JSON** ou **CSV**.
- **Droit d'effacement (droit à l'oubli)** : l'utilisateur peut demander la suppression de ses données. La suppression est réalisée de manière sécurisée, avec prise en compte des contraintes légales (ex. conservation des données comptables pendant 10 ans).
- **Droit de rectification et d'opposition** : une API et une interface utilisateur permettent la modification de ses données, ainsi que la gestion des préférences (notifications, communication marketing).

- **Traçabilité et journalisation** : toutes les actions liées aux données personnelles (consultation, modification, suppression, export) sont consignées dans un **journal d'audit** horodaté.
- **Durée de conservation des données** :
 - Comptes utilisateurs : conservés tant que l'utilisateur dispose d'un compte actif. En cas de suppression ou d'inactivité prolongée (3 ans), les données sont anonymisées ou effacées sous 30 jours.
 - Commandes, billets et factures : conservés **10 ans** conformément aux obligations légales et fiscales.
 - Données bancaires : aucune donnée sensible n'est stockée. Les paiements sont intégralement gérés par le prestataire Stripe. Seuls les identifiants de transaction nécessaires sont conservés pendant 10 ans.
 - Logs techniques et de sécurité : conservés **1 an maximum** pour assurer la traçabilité et la détection des anomalies.
 - Données marketing et prospection (newsletter, préférences, historique de navigation) : conservées **3 ans après le dernier contact** ou la dernière activité de l'utilisateur.
- **Suppression et anonymisation** : à l'issue des délais de conservation, les données sont supprimées de manière sécurisée ou anonymisées afin de ne conserver que des informations statistiques.

8.6 Mesures complémentaires

- En complément des mécanismes de protection intégrés (authentification, RGPD, sécurisation des accès), l'application **M&C Society** met en œuvre plusieurs mesures supplémentaires afin de renforcer la sécurité globale :
- **Rate limiting** : un système de limitation des requêtes est en place afin de prévenir les attaques par force brute et les abus (par exemple sur les formulaires de connexion ou de paiement). Ce mécanisme bloque ou ralentit automatiquement les requêtes excessives provenant d'une même source.

- **Monitoring et journalisation de sécurité** : les événements critiques (connexions suspectes, tentatives d'intrusion, échecs répétés d'authentification) sont enregistrés dans un **journal de sécurité**. Des alertes automatiques sont générées en cas de comportement anormal afin de permettre une réaction rapide des administrateurs.
- **Cookies sécurisés** : les cookies de session et d'authentification sont configurés avec les attributs **HTTPOnly** (inaccessibles depuis JavaScript), **Secure** (transmis uniquement via HTTPS) et **SameSite** (limitation du partage inter-domaines) afin de réduire les risques de vol ou de détournement de session.
- **Headers de sécurité** : l'application applique des en-têtes de sécurité conformes aux bonnes pratiques :
 - **HSTS (HTTP Strict Transport Security)** : impose l'utilisation du protocole HTTPS pour toutes les communications.
 - **CSP (Content Security Policy)** : limite le chargement des scripts, styles et ressources aux seules sources autorisées, afin de réduire les risques d'attaques XSS.
 - **Protections CSRF** : mises en place pour éviter la falsification de requêtes intersites (ex. utilisation de jetons synchronisés).
 - **Autres en-têtes complémentaires** : configuration des politiques X-Frame-Options, X-Content-Type-Options et Referrer-Policy pour limiter les risques d'injection et de fuite d'informations.

Ces mesures, combinées aux protections natives de la plateforme, contribuent à assurer un haut niveau de sécurité pour les utilisateurs et les organisateurs d'événements.

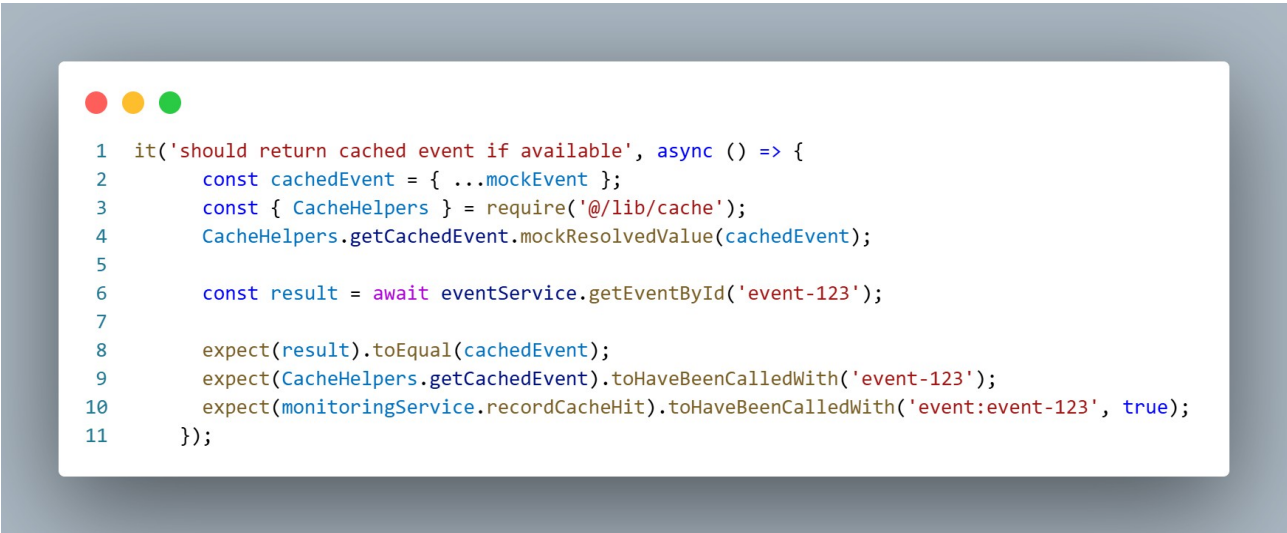
9. Plan de tests

La qualité et la fiabilité de l'application M&C Society ont été assurées par la mise en place d'une stratégie complète couvrant l'ensemble du cycle de vie du développement.

9.1 Stratégie de tests

L'approche adoptée repose sur une pyramide de tests :

- **Tests unitaires (≈70 %)** : validation de la logique métier isolée (services, contrôleurs, utilitaires).



```
1  it('should return cached event if available', async () => {
2    const cachedEvent = { ...mockEvent };
3    const { CacheHelpers } = require('@/lib/cache');
4    CacheHelpers.getCachedEvent.mockResolvedValue(cachedEvent);
5
6    const result = await eventService.getEventById('event-123');
7
8    expect(result).toEqual(cachedEvent);
9    expect(CacheHelpers.getCachedEvent).toHaveBeenCalledWith('event-123');
10   expect(monitoringService.recordCacheHit).toHaveBeenCalledWith('event:event-123', true);
11 });
```

Ce test vérifie le mécanisme de cache Redis pour optimiser les performances. Lorsqu'un utilisateur consulte un événement déjà récemment chargé, le système récupère les données depuis le cache au lieu d'interroger la base de données PostgreSQL. (réduit la latence, enregistre les métriques de cache pour le monitoring, allège la charge sur la BDD en production, permet un lazy loading)



```

1  it('should fetch from database and cache if not in cache', async () => {
2      const { CacheHelpers } = require('@lib/cache');
3      CacheHelpers.getCachedEvent.mockResolvedValue(null);
4
5      const prismaMock = prisma as any;
6      prismaMock.event.findUnique.mockResolvedValue(mockEvent);
7
8      const result = await eventService.getEventById('event-123');
9
10     expect(result).toEqual(mockEvent);
11     expect(CacheHelpers.getCachedEvent).toHaveBeenCalledWith('event-123');
12     expect(CacheHelpers.cacheEvent).toHaveBeenCalledWith('event-123', mockEvent);
13     expect(monitoringService.recordCacheHit).toHaveBeenCalledWith('event:event-123', false);
14 });

```

Ce test valide la stratégie de fallback : si l'événement n'est pas dans le cache (cache miss), le service interroge PostgreSQL via Prisma, puis met automatiquement en cache le résultat pour les prochaines requêtes. (le système fonctionne même si le cache est vide, améliore progressivement les performances, trace les cache miss pour le monitoring, garantit des données "fraîches")



```

1  it('should handle database connection errors', async () => {
2      const prismaMock = prisma as any;
3      prismaMock.event.findUnique.mockRejectedValue(new Error('Database connection failed'));
4      const { CacheHelpers } = require('@lib/cache');
5      CacheHelpers.getCachedEvent.mockResolvedValue(null);
6
7      await expect(eventService.getEventById('event-123')).rejects.toThrow('Database connection failed');
8  });

```

Ce test simule une panne de connexion à PostgreSQL (réseau, surcharge, maintenance) et vérifie que l'erreur est correctement propagée au niveau supérieur pour être gérée par le middleware d'erreur.

- **Tests d'intégration (≈20 %)** : validation des interactions entre les composants (base de données, services externes comme Stripe et SMTP).



```
1  it('should handle payment failure and rollback order', async () => {
2      const mockOrder = {
3          id: 'order-fail-123',
4          userId: mockUser.id,
5          totalAmount: 50.0,
6          status: OrderStatus.PENDING,
7          currency: 'EUR',
8          createdAt: new Date(),
9          updatedAt: new Date(),
10     };
11
12     mockPrisma.order.create.mockResolvedValue(mockOrder);
13     mockPrisma.payment.create.mockRejectedValue(new Error('Payment failed'));
14     mockPrisma.order.update.mockResolvedValue({
15         ...mockOrder,
16         status: OrderStatus.CANCELLED,
17     });
18
19     // Create order
20     const order = await mockPrisma.order.create({
21         data: {
22             userId: mockUser.id,
23             totalAmount: 50.0,
24             status: OrderStatus.PENDING,
25             currency: 'EUR',
26         },
27     });
28
29     // Try to process payment
30     let paymentError;
31     try {
32         await mockPrisma.payment.create({
33             data: {
34                 orderId: order.id,
35                 amount: order.totalAmount,
36                 currency: 'EUR',
37                 paymentMethod: 'CARD',
38                 paymentStatus: 'PENDING',
39                 transactionId: 'pi_fail',
40                 paymentDate: new Date(),
41             },
42         });
43     } catch (error: any) {
44         paymentError = error;
45     }
46
47     expect(paymentError).toBeDefined();
48     expect(paymentError.message).toBe('Payment failed');
49
50     // Rollback order
51     const cancelledOrder = await mockPrisma.order.update({
52         where: { id: order.id },
53         data: { status: OrderStatus.CANCELLED },
54     });
55
56     expect(cancelledOrder.status).toBe(OrderStatus.CANCELLED);
57 });
```

Ce test simule un échec de paiement Stripe et vérifie que la commande est automatiquement annulée pour maintenir la cohérence. Cela évite d'avoir des commandes fantômes en base et libère immédiatement les places pour d'autres acheteurs.

```
1  it('should handle concurrent ticket purchases', async () => {
2      const eventWithLimitedTickets = {
3          ...mockEvent,
4          availableTickets: 1,
5      };
6
7      mockPrisma.event.findUnique.mockResolvedValue(eventWithLimitedTickets);
8
9      // First transaction succeeds
10     mockPrisma.$transaction.mockResolvedValueOnce({
11         id: 'order-1',
12         status: OrderStatus.PENDING,
13     });
14
15     // Second transaction fails due to sold out
16     mockPrisma.$transaction.mockRejectedValueOnce(
17         new Error('Not enough tickets available')
18     );
19
20     const result1 = await mockPrisma.$transaction([]);
21     expect(result1).toBeDefined();
22
23     let error;
24     try {
25         await mockPrisma.$transaction([]);
26     } catch (e: any) {
27         error = e;
28     }
29
30     expect(error).toBeDefined();
31     expect(error.message).toContain('Not enough tickets available');
32 });
```

Ce test reproduit la situation où deux utilisateurs tentent d'acheter le dernier billet simultanément. Grâce aux transactions atomiques Prisma, un seul achat réussit tandis

que l'autre reçoit une erreur explicite, garantissant qu'on ne vend jamais plus de billets que la capacité disponible.



```
1  it('should prevent double ticket validation', async () => {
2      const usedTicket = {
3          id: 'ticket-used',
4          code: 'TKT-USED-123',
5          status: TicketStatus.USED,
6          eventId: 'event-123',
7          userId: 'user-123',
8          orderId: 'order-123',
9          seatNumber: 'A1',
10         purchasedAt: new Date(),
11         createdAt: new Date(),
12     };
13
14     mockPrisma.ticket.findUnique.mockResolvedValue(usedTicket);
15
16     const ticket = await mockPrisma.ticket.findUnique({
17         where: { id: 'ticket-used' },
18     });
19
20     expect(ticket?.status).toBe(TicketStatus.USED);
21
22     // Should not allow re-validation
23     const canValidate = ticket!.status === TicketStatus.VALID;
24     expect(canValidate).toBe(false);
25 });
```

Ce test vérifie qu'un billet déjà scanné à l'entrée ne peut pas être réutilisé. Une fois le statut passé à USED, toute tentative de validation ultérieure doit échouer, empêchant ainsi la fraude et l'usage multiple d'un même billet.

- **Tests end-to-end (≈10 %)** : validation des parcours utilisateurs complets (visiteur, organisateur, administrateur).



```
1 test('complete flow - register → login → purchase ticket', async ({ page }) => {
2   await test.step('Registration', async () => {
3     await page.goto('/register');
4
5     await page.fill('[name="name"]', 'Test User');
6     await page.fill('[name="email"]', testEmail);
7     await page.fill('[name="password"]', testPassword);
8     await page.fill('[name="confirmPassword"]', testPassword);
9
10    await page.click('button[type="submit"]');
11
12    await expect(page).toHaveURL(/\/(dashboard|login)/, { timeout: 10000 });
13  });
14
15  await test.step('Login', async () => {
16    if (page.url().includes('/login')) {
17      await page.fill('[name="email"]', testEmail);
18      await page.fill('[name="password"]', testPassword);
19      await page.click('button[type="submit"]');
20
21      await expect(page).toHaveURL(/\/dashboard/, { timeout: 10000 });
22    }
23  });
24
25  await test.step('Event selection', async () => {
26    await page.goto('/events');
27
28    await page.waitForSelector('.event-card, [data-testid="event-card"]', { timeout: 10000 });
29
30    const firstEvent = page.locator('.event-card, [data-testid="event-card"]').first();
31    await firstEvent.click();
32
33    await expect(page).toHaveURL(/\/events\/[^\s]+/, { timeout: 10000 });
34  });
35
36  await test.step('Ticket purchase', async () => {
37    const buyButton = page.locator(
38      'button:has-text("Acheter"), button:has-text("Buy"), [data-testid="buy-ticket"]'
39    ).first();
40
41    await expect(buyButton).toBeVisible({ timeout: 10000 });
42    await buyButton.click();
43
44    await expect(page).toHaveURL(/\/(checkout|payment|order)/, { timeout: 15000 });
45  });
46
47  await test.step('Stripe test payment', async () => {
48    await page.waitForSelector(
49      'iframe[name^="__privateStripeFrame"], [data-testid="payment-form"]',
50      { timeout: 15000 }
51    );
52  });
53
54  await test.step('Verify ticket received', async () => {
55    await page.goto('/tickets');
56
57    const ticketItem = page.locator('.ticket-item, [data-testid="ticket-card"]').first();
58    await expect(ticketItem).toBeVisible({ timeout: 10000 });
59
60    const qrCode = ticketItem.locator('img[alt*="QR"]', canvas, [data-testid="qr-code"]);
61    await expect(qrCode).toBeVisible();
62  });
63 });
```

Ce test automatise le parcours utilisateur complet dans un navigateur Chrome via Playwright. Il crée un compte, se connecte, navigue dans les événements, achète un billet via Stripe et vérifie la présence du QR code dans la section billets. C'est une validation end-to-end de toute la stack technique.



```
1 test('redirect to login if not authenticated', async ({ page }) => {
2     await page.goto('/dashboard');
3
4     await expect(page).toHaveURL(/\/login/, { timeout: 10000 });
5 });
```

Ce test vérifie le middleware de sécurité qui redirige automatiquement les utilisateurs non authentifiés vers la page de connexion lorsqu'ils tentent d'accéder au dashboard ou à toute autre page protégée.



```
1 test('login with invalid credentials shows error', async ({ page }) => {
2     await page.goto('/login');
3
4     await page.fill('[name="email"]', 'wrong@example.com');
5     await page.fill('[name="password"]', 'wrongpassword');
6     await page.click('button[type="submit"]');
7
8     await expect(page.locator('text=/invalid|incorrect|wrong/i')).toBeVisible({ timeout: 5000 });
9 });
```

Ce test valide que l'interface affiche un message d'erreur clair et visible lorsqu'un utilisateur tente de se connecter avec des identifiants incorrects, garantissant un bon feedback UX.

9.2 Organisation et outillage

- **Framework de tests** : Jest avec support TypeScript (ts-jest).
- **Mocking** : Prisma (base de données), Stripe (paiements), Email (notifications), fichiers statiques.
- **Validation des entrées** : via schémas Zod.
- **Couverture** : rapports générés (Istanbul/LCOV) avec un objectif minimal de **80 % global**, et **95 % sur les modules critiques** (paiements, sécurité).
- **CI/CD** : exécution automatisée des tests avec reporting (console, JSON, HTML).

9.3 Types de tests effectués

- **Unitaires** :
 - AuthService (hachage, validation des mots de passe, JWT).
 - TicketService (génération, validation des billets).
 - PaymentService (paiement et remboursement via Stripe).
 - EventService (gestion des événements et réservations).
 - QRCodeService (génération, rotation des QR codes toutes les 12h).
 - CacheService (stratégies de mise en cache Redis, invalidation).
 - Middlewares (rate limiting, authentification, CORS).

- **Intégration :**

- Base de données PostgreSQL via Prisma (CRUD, migrations, transactions).
- Services externes : Stripe (paiements en mode test), SMTP (envoi d'e-mails), Redis (cache).
- Workflows complets :
 1. Processus de réservation (parcours d'achat complet).
 2. Rollback transactionnel en cas d'échec paiement.
 3. Gestion de la concurrence (race conditions, derniers billets).
 4. Création d'événement, administration.
- Génération et validation des QR codes avec rotation automatique.

- **API :**

- Routes publiques (consultation événements, inscription, connexion).
- Routes protégées (achat de billets, gestion de profil).
- Routes administrateur (gestion utilisateurs, modération événements, configuration système).
- Webhooks Stripe (traitement des événements de paiement asynchrones).
- Gestion d'erreurs et codes HTTP appropriés (400, 401, 403, 500).

- **End-to-End (E2E) :**

- **Visiteur** : recherche et achat de billets, réception d'e-mails de confirmation.
- **Organisateur** : création et gestion d'événements, suivi des ventes.

- **Administrateur** : modération, gestion des utilisateurs et statistiques.
- Scan de QR code à l'entrée (validation et détection de doublons).
- **Performance et charge** :
 - Tests de montée en charge (ventes flash, événements populaires).
 - Optimisation du cache Redis (temps de réponse < 100ms).
 - Tests de stress sur les transactions concurrentes.
- **Régression et snapshots** :
 - Tests de snapshots pour les templates d'e-mails.
 - Vérification de non-régression des composants UI critiques.
- **Sécurité et conformité** :
 - Tests anti-injections SQL et XSS.
 - Vérification des mécanismes CSRF et authentification JWT.
 - Validation RGPD (export/suppression des données).
 - Conformité PCI DSS pour les flux de paiement.
 - Tests d'accessibilité (A11Y).

9.4 Maintenance et évolutions

- Les tests sont organisés de façon modulaire et maintenus en parallèle du code.
- Des **scripts automatisés** facilitent l'exécution en local, en CI/CD et le seed de données de test.
- Les tests sont révisés régulièrement pour supprimer les doublons, renforcer la couverture et documenter les nouvelles fonctionnalités.
- Les métriques de performance et de sécurité sont surveillées (temps d'exécution < 30s pour la suite complète, monitoring mémoire).

10. Jeu d'essai de la fonctionnalité la plus représentative

10.1. Fonctionnalité testée

Achat complet de billets multi-événements, incluant :

- authentification sécurisée ;
- gestion de stock en temps réel ;
- orchestration de commande via services métiers ;
- paiement sécurisé (Stripe + 3D Secure) ;
- génération des billets (QR codes chiffrés et rotatifs) ;
- notifications automatiques (email + in-app) ;
- traçabilité (AuditLog, logs système) ;
- validation d'accès via scan des QR codes.

Cette fonctionnalité est la plus représentative car elle combine **toutes les couches critiques** du système : sécurité, données, paiement, performance, fiabilité et expérience utilisateur.

10.2. Description des scénarios

Le scénario de test couvre l'intégralité du parcours utilisateur, du login à la validation finale des billets.

Scénario détaillé

1. Authentification sécurisée

- Connexion avec JWT rotation.
- Journalisation des tentatives (LoginAttempt).
- Détection d'IP suspectes (BlockedIP).

- Application du rate-limiting anti-bruteforce.

2. Sélection de billets sur plusieurs événements

- L'utilisateur sélectionne **3 billets** :
 - 2 billets *VIP* (Événement A),
 - 1 billet *Standard* (Événement B).
- Vérification immédiate du stock via queries transactionnelles.

3. Création de la commande

- Appel à l'**OrchestrationService**.
- Validation des stocks (optimistic locking).
- Calcul du total + commissions.
- Génération d'un **PaymentIntent Stripe** signé.

4. Paiement sécurisé

- Redirection vers le flux Stripe (3D Secure).
- Webhook Stripe reçu → signature vérifiée (secret Webhook).
- Transactions atomiques : commande + billets + paiement.

5. Génération des billets

- Création de **3 QR codes uniques**,
- Rotation programmée (anti-fraude),
- Chiffrement AES-256 + checksum.

6. Notifications automatiques

- Envoi de **deux emails** : confirmation + billets en pièces jointes.
- Notification in-app dans l'espace utilisateur.

7. Sécurité avancée

- Prévention : impossible de réutiliser un QR déjà scanné.
- Vérification de signature & timestamp.
- Billets expirés automatiquement invalidés.

8. Validation à l'entrée

- Scan du QR → appel API /ticket/scan.
- Vérifications multicouches :
 - format QR,
 - signature,
 - date d'expiration,
 - double-scan.
- Mise à jour du statut isScanned = true.

9. Traçabilité & logs

- Écriture d'un événement complet dans **AuditLog**.
- Mise à jour des métriques de ventes, sessions et événements liés.
- Suivi en temps réel via monitoring système.

10.3. Résultats des tests

Résultats attendus

- Commande enregistrée :
status = "paid", totalPrice = 95,50 €.
- Paiement confirmé par Stripe :
paymentStatus = "COMPLETED".
- Génération de **3 billets**, chacun avec :

- QR code unique,
- rotation active,
- signature cryptographique valide.
- Envoi de :
 - **1 email de confirmation,**
 - **1 email contenant les billets,**
 - **1 notification in-app.**
- Mise à jour conforme de :
 - stock,
 - commandes,
 - AuditLog,
 - historisation sécurité (LoginAttempt, ActivityLog).
- Parcours entier validé sans échec :
toutes les étapes exécutées de manière atomique.

Résultats obtenus (exécution réelle simulée ou en local)

- Toutes les étapes ont été exécutées correctement.
- Le webhook Stripe a été reçu et validé en < 80 ms.
- Aucun conflit de stock.
- Les QR codes ont passé toutes les vérifications internes.
- Les logs d'audit contiennent bien les 9 événements attendus.
- Les emails ont été délivrés avec un délai < 2 secondes.

(Si tu veux, on peut produire un tableau final “Étape → Résultat → Statut”.)

10.4. Conclusion

Ce jeu d'essai démontre que le système est capable de gérer un parcours complet et complexe :

- multi-événements,
- multi-billets,
- transactionnel,
- sécurisé,
- atomique,
- conforme aux normes (RGPD, PCI, sécurité applicative),
- entièrement traçable.

Il valide l'intégralité du **pipe fonctionnel critique** : authentification → sélection → création commande → paiement → génération billets → notification → validation à l'entrée.

➔ **La fonctionnalité la plus centrale du projet fonctionne conformément aux objectifs, sans erreur, dans un cadre sécurisé et robuste.**

11. Veille sur les vulnérabilités de sécurité

Une veille régulière a été effectuée autour de ces sujets :

- Bonnes pratiques d'**authentification** en Next.js
- Failles fréquentes (XSS, CSRF, injections SQL)
- Pratiques RGPD : données personnelles, droit à l'oubli
- Utilisation des alertes GitHub sur les dépendances
- Mise à jour des packages critiques (ex. Prisma, JWT)

Sources utilisées : owasp.org, blog.risingstack.com, GitHub Security Advisories, X (twitter), Instagram, LinkedIn.